Contents

Research Problems and Ranked Research Ideas **1**
Research problem 1 (P1) 1
Research problem 2 (P2) 45

Research Problems and Ranked Research Ideas

Base paper. Gong et al., *Scaling Diffusion Language Models via Adaptation from Autoregressive Models*, ICLR 2025.

Method. An implementation of **ResearchAgent** (Baek et al., arXiv:2404.07738). Problems, methods and experiment designs are generated with the paper's own prompts (Tables 6-8) over the base paper, the works that cite it, and entities retrieved from a knowledge store (Eq. 2). Each is reviewed by ReviewingAgents on five criteria per stage with the paper's human-induced rubrics (Tables 9-15), and refined from the reviews for 3 rounds. Deviations: `REPORT.md` §3.

Selection. Problems chosen by: team, 2026-09-28 (4->3: dropped P4, MASK-token, needs training); 2026-10-08 (3->2: dropped P3). The brief asks for one or two problems. P3 (candidate count x AR verifier) overlaps P2 (noise schedule) as a second quality-efficiency study, and no P3 work had started, so dropping it discards no cache and saves 384 calls. P1 kept despite the lowest problem score: it carries the anchor-specific question (AR retention vs. reranking headroom).

Novelty. Not yet checked for any idea below. Every idea must be checked against recent literature, starting with the base paper's own authors' later work, before it is pursued.

Research problem 1 (P1)

Does the degree to which a diffusion language model (DLM) preserves the autoregressive (AR) representations of its pretrained backbone determine how much a lightweight, training-free reranking step (using a small AR scorer) can recover the model's accuracy headroom, and is this recovery more efficient than simply increasing the denoising-step budget?

Rationale. The target paper shows that continual-pre-training of open-source AR models yields competitive DLMs (DiffuGPT, DiffuLLaMA) but leaves a sizable "accuracy headroom"—the gap between single-answer accuracy and the oracle performance achievable by selecting the best among multiple generation candidates (L6). Recent inference-time methods such as Jacobi Forcing and TESS 2's reward guidance demonstrate that quality-efficiency trade-offs can be shifted toward efficiency, yet it remains unclear whether a simple post-hoc selection mechanism can close this headroom more efficiently than additional denoising steps, and why some DLMs might benefit more than others.

To answer this, we will (1) quantify AR-representation preservation in each released DLM by computing centered kernel alignment (CKA) (or linear probing) between the DLM's hidden states and those of its AR base model on a held-out text corpus; (2) generate multiple candidates per prompt at several denoising-step budgets (e.g., 16, 32, 64, 128 steps) using the released checkpoints (DiffuGPT-S/M, DiffuLLaMA, Dream-7B, LLaDA-8B, DiffuCoder-7B); (3) rerank these candidates with a lightweight AR scorer (GPT-2-small, 124 M parameters, contributing <1 % of total FLOPs and negligible latency); (4) measure two kinds of quality gains: (a) the gain from reranking over the top-1 candidate at a fixed step budget, and (b) the gain from increasing the denoising-step budget (e.g., from 16 to 128 steps) without reranking; quality will be reported as a unified metric—the average of normalized scores across HumanEval pass@k (k=1,10), GSM8K accuracy, and the average of SIQA and WinoGrande commonsense scores, each normalized to [0,1] across all conditions; (5) test whether higher representation-preservation scores predict larger reranking gains *relative* to denoising-step gains, and whether this relationship differs between GPT-2-based and LLaMA-based DLMs or varies with step budget.

This study stays strictly within the inference-only constraint (no training, adaptation, or continual pre-training), uses only released checkpoints and publicly available base models for probing, and fits the ten-week timeline on a single RTX 6000 Pro Blackwell by limiting generation to ≤128 steps (where marginal returns are most informative), employing activation checkpointing and 8-bit quantization for the 6.74B DiffuLLaMA model, and parallelizing candidate generation, scoring, and evaluation across team members. By linking representation-preservation analysis to the comparative effectiveness of lightweight selection versus additional denoising, the work moves beyond an empirical optimization study to provide fundamental insight into whether the accuracy headroom stems from insufficient denoising or from suboptimal exploitation of AR-derived representations, thereby bridging representation-alignment techniques (REPR-ALIGN, PreDiff-LM) with pure inference-time trade-off methods (Jacobi Forcing, TESS 2).

ReviewingAgent scores: Clarity 3/5, Relevance 3/5, Originality 3/5, Feasibility 3/5, Significance 3/5.

All 8 ideas, ranked

Rank	Idea	Overall /5	Novelty	Feasibility	Clarity	Impact	Weakest
1	P1-M1	**4.9**	4	5	5.0	5	4
2	P1-M6	**4.9**	4	5	5.0	5	4
3	P1-M4	**4.7**	4	5	5.0	4	4
4	P1-M8	**4.7**	4	5	5.0	4	4
5	P1-M2	**4.5**	4	5	4.5	4	4
6	P1-M5	4.5	4	5	4.5	4	4
7	P1-M7	4.5	4	5	5.0	3	3
8	P1-M3	4.4	4	5	4.5	4	4

Top 5 submitted; the rest are listed so the cut is visible.

Idea 1.1 — P1-M1 (overall 4.9/5)

Method.

1. **Representation Preservation Quantification (Layer-Specific & Robust Baselines)** We quantify the preservation of AR representations in each DLM (M_{DLM}) relative to its base AR model (M_{AR}) using **Centered Kernel Alignment (CKA)**. To address the concern that averaging layers may mask specific preservation patterns, we compute CKA **per layer** and report both the per-layer profile and an aggregated score.

- **Data:** A fixed, held-out corpus of 5,000 diverse text segments, stratified to ensure coverage of the evaluation domains: 2,000 general text (WikiText-103), 1,500 code snippets (HumanEval/MBPP), and 1,500 reasoning/commonsense text (GSM8K/SIQA prompts). All segments are length-normalized to 128 tokens.
- **Feature Extraction:** For each model, we perform a single forward pass on the clean text (for AR) or the fully unmasked sequence (for DLM, at the final denoised state x_0). We extract hidden states from **all layers** $L = 1, \ldots, L_{total}$. Let $H_{AR}^{(l)} \in \mathbb{R}^{N \times d}$ and $H_{DLM}^{(l)} \in \mathbb{R}^{N \times d}$ be the hidden state matrices for layer l.
- **Metric:** We compute Linear CKA between corresponding layers:

$$S_{rep}^{(l)} = \text{CKA}(H_{AR}^{(l)}, H_{DLM}^{(l)})$$

  We define the **Aggregate Preservation Score** S_{rep} as the **mean** of $S_{rep}^{(l)}$ across all layers. This avoids the bias of focusing only on the last 4 layers, which may not capture mid-layer semantic shifts. We also report the standard deviation of $S_{rep}^{(l)}$ to assess consistency across depth.
- **Baseline Calibration & Normalization:** To establish a meaningful range for S_{rep}, we compute two baselines:

1. **Floor (Random):** We compute CKA between the DLM's hidden states and those of a **randomly initialized model** of the same architecture (weights sampled from the initialization distribution). This establishes the lower bound for "no preservation."
2. **Ceiling (Identity):** We compute CKA between the AR base model and itself (or a frozen copy of the AR base model). This establishes the upper bound for "perfect preservation" (theoretically 1.0, but serves as a sanity check for numerical stability and feature extraction consistency). We report S_{rep} as the raw CKA value, but also provide a **Normalized Preservation Score** $S_{rep}^{norm} = \frac{S_{rep} - S_{floor}}{S_{ceiling} - S_{floor}}$ to facilitate cross-architecture comparison.

2. Candidate Generation and Reranking Protocol For each DLM checkpoint ($M \in$ {DiffuGPT-S/M, DiffuLLaMA, Dream-7B, LLaDA-8B, DiffuCoder-7B}), we conduct inference on a standardized evaluation set (HumanEval, GSM8K, SIQA, WinoGrande).

- **Step Budgets:** We generate candidates at $K \in \{16, 32, 64, 128\}$ denoising steps.
- **Candidate Set:** For each prompt x, we generate $N = 10$ distinct candidates $\{y_1, \ldots, y_{10}\}$ by varying the random seed at the initial noise stage.
- **Standardized Top-1 Baseline:** The "Top-1" baseline is consistently defined as the **first generated candidate** (y_1) across all models and conditions.
- **Lightweight Reranker (Mismatched):** We use **GPT-2-small (124M)** as a frozen scorer for all models.
  - **Text-Bridge Scoring:** Candidates are decoded to raw text, then re-tokenized using the **GPT-2 tokenizer**. The score is the negative log-likelihood (NLL) of the re-tokenized sequence conditioned on the re-tokenized prompt.
  - **Truncation & Special Token Handling:** To ensure robustness, we strip all special tokens (e.g., `<|startoftext|>`, `<pad>`) from both the prompt and candidate text before re-tokenization. If the re-tokenized sequence exceeds the GPT-2 context window (1024 tokens), we truncate from the right, preserving the prefix. This ensures the scorer evaluates the most critical leading portion of the response.
  - $y^*_{GPT2} = \arg\min_{y_i} \text{Score}_{GPT2}(y_i)$.
- **Controlled Reranker (Matched):** To control for architectural bias, we use **TinyLlama-1.1B** as a frozen scorer for LLaMA-based DLMs (DiffuLLaMA, Dream-7B, LLaDA-8B, DiffuCoder-7B).
  - **Text-Bridge Scoring:** Candidates are decoded to text and re-tokenized using the **TinyLlama tokenizer**.
  - **Truncation & Special Token Handling:** Similar to GPT-2, we strip special tokens and truncate to the TinyLlama context window (2048 tokens).
  - $y^*_{TinyLlama} = \arg\min_{y_i} \text{Score}_{TinyLlama}(y_i)$.
  - *Note:* For GPT-2-based DLMs, the "matched" scorer is GPT-2-small itself. For LLaMA-based DLMs, we compute both mismatched (GPT-2) and matched (TinyLlama) reranking gains.
- **Non-AR Control (Encoder-Only):** To determine if the "recovery" is specifically tied to AR representations or general linguistic quality, we include a **BERT-base-uncased** (encoder-only) scorer as a control condition for a subset of prompts (50 per task) on one model (DiffuLLaMA).
  - **Scoring:** We compute the Mean Squared Error (MSE) between the candidate's [CLS] embedding and the prompt's [CLS] embedding, or simply the average token-level classification loss for a "coherence" proxy if available. Alternatively, we use the negative average token log-probability from the BERT MLM head on the candidate text. This serves as a baseline for "general linguistic plausibility" without causal ordering.
- **Oracle Baseline:** We explicitly report the **Oracle (Best-of-10)** performance, defined as the maximum score among the 10 candidates according to the ground-truth metric (e.g., pass@1 for code, exact match for math). This quantifies the total available headroom and how much of it is recovered by the reranker.
- **Compute Accounting & Latency:** We track both **FLOPs** and **Wall-Clock Time**, with explicit separation of phases.
  - **FLOPs:** $Cost_{steps} = K \times Cost_{forward}$. $Cost_{rerank} = K \times Cost_{forward} + N \times Cost_{scorer}$.

- **Wall-Clock:** We measure and report **three distinct timings**:
  1. T_{gen}: Time for DLM candidate generation.
  2. T_{bridge}: Time for decoding candidates to text and re-tokenizing for the scorer.
  3. T_{score}: Time for the AR scorer forward pass. Total Latency $T_{total} = T_{gen} + T_{bridge} + T_{score}$. This allows us to pinpoint if the "negligible" scorer cost is actually dominated by the Text-Bridge overhead.

3. Unified Quality Metric and Efficiency Analysis

- **Normalization:** We compute task-specific scores (Pass@1 for HumanEval, Accuracy for GSM8K/SIQA/WinoGrande). To create a unified metric $Q \in [0, 1]$, we normalize each task's score **per-task** across all models and conditions:

$$Q_{norm,task} = \frac{Q_{raw,task} - Q_{min,task}}{Q_{max,task} - Q_{min,task}}$$

  The final metric Q is the **average** of the four normalized task scores. This ensures that each task contributes equally to the final efficiency comparison, preventing high-variance tasks from dominating.
- **Efficiency Comparison:** We define **Marginal Efficiency** as the change in normalized quality ΔQ per unit of additional compute (measured in Wall-Clock Time).
  - $\Delta Q_{steps} = Q(K = 128) - Q(K = 16)$ (using Top-1 selection).
  - $\Delta Q_{rerank} = Q_{reranked}(K) - Q_{top1}(K)$ (at a fixed K, e.g., $K = 16$).
  - We compare ΔQ_{rerank} against the ΔQ_{steps} required to achieve an equivalent quality gain. Specifically, we ask: Does adding a lightweight reranker at $K = 16$ yield a larger ΔQ per second than increasing steps from 16 to 128?
  - **Step-Budget Sensitivity:** We compute the Reranking Advantage for multiple step-increase baselines:
    * $A^{16\to32}_{rerank} = \Delta Q_{rerank}(16) - \Delta Q_{steps}(16 \to 32)$
    * $A^{32\to64}_{rerank} = \Delta Q_{rerank}(32) - \Delta Q_{steps}(32 \to 64)$
    * $A^{64\to128}_{rerank} = \Delta Q_{rerank}(64) - \Delta Q_{steps}(64 \to 128)$ This tests if the reranking advantage is consistent or diminishes as the base denoising quality improves.

4. Statistical Correlation Analysis We test the hypothesis that S_{rep} (Aggregate CKA score) predicts the relative benefit of reranking over step-increase.

- **Reranking Advantage:** For each model M, we compute the **Reranking Advantage** using the *matched* scorer (to control for architectural bias):

$$A_{rerank}(M) = \Delta Q_{rerank,matched}(M) - \Delta Q_{steps}(M, 16 \to 32)$$

- **Correlation Analysis:** We perform a Pearson correlation analysis between $S_{rep}(M)$ and $A_{rerank}(M)$ across the 5 model families. Given the small sample size ($N = 5$), we focus on the **trend** and effect size rather than relying solely on p-values. We also report the Spearman rank correlation to check for monotonic relationships.
- **Statistical Robustness:** To address the small sample size, we perform **bootstrapping** (1,000 resamples with replacement) to estimate the confidence interval of the correlation coefficient. We also perform a **leave-one-out** analysis to check if the correlation is driven by a single outlier model.
- **Architectural Stratification:** We stratify the analysis by architecture family (GPT-2-based vs. LLaMA-based) to check if the relationship holds within families.
- **Scorer Mismatch Analysis:** For LLaMA-based models, we compute the difference between mismatched (GPT-2) and matched (TinyLlama) reranking gains. If the mismatched gain is significantly lower, it indicates that the GPT-2 scorer is a bottleneck for LLaMA-based DLMs, confounding the S_{rep} correlation.
- **Scorer Capacity Check:** To address the concern that small scorers may limit the observed effect, we perform a **subset analysis** using a larger scorer (e.g., LLaMA-2-7B) for a subset of

prompts (e.g., 50 prompts per task) on one LLaMA-based model (DiffuLLaMA). If the correlation between S_{rep} and reranking gain holds with the larger scorer, it suggests the effect is not merely a byproduct of the small scorers' limited capacity.
- **Domain-Specific CKA:** We analyze whether representation preservation (S_{rep}) is higher in specific domains (e.g., code vs. natural language) by computing CKA on the stratified subsets of the probing corpus. We then test if reranking efficiency is domain-dependent by correlating domain-specific S_{rep} with domain-specific reranking gains.

5. Experimental Execution Plan (10 Weeks)

- **Weeks 1-2:** Setup. Implement CKA extraction script (per-layer). Verify DLM checkpoints load correctly. Prepare evaluation datasets and scorers (GPT-2-small, TinyLlama-1.1B, LLaMA-2-7B, BERT-base). Implement Text-Bridge handling (tokenization, truncation).
- **Weeks 3-5:** Representation Probing. Run forward passes on the 5,000-sample corpus for all 5 DLMs and their AR bases. Compute per-layer CKA scores, aggregate scores, random baselines, and identity ceilings. (CPU/GPU mix, low compute).
- **Weeks 5-8:** Generation & Scoring.
  - *DiffuGPT-S/M:* Generate 10 candidates × 4 step budgets × ~1000 prompts. Score with GPT-2-small.
  - *DiffuLLaMA/Dream/LLaDA/DiffuCoder:* Use 8-bit quantization and activation checkpointing. Generate on a subset (e.g., 200 prompts per task) to manage 96GB VRAM and time. Score with both GPT-2-small and TinyLlama-1.1B.
  - *Subset Scoring with LLaMA-2-7B & BERT:* For DiffuLLaMA, score a subset (50 prompts) with LLaMA-2-7B and BERT-base to check scorer capacity and non-AR control.
  - Parallelize: 3 GPU users run different models simultaneously.
- **Weeks 9-10:** Analysis & Writing. Compute unified metrics (per-task normalization). Perform correlation analysis (Pearson, Spearman, Bootstrapping, Leave-One-Out), scorer mismatch analysis, scorer capacity check, and domain-specific CKA analysis. Write report.

Why this method.

1. **Addressing the "Why" via Layer-Specific Representation Geometry and Scorer Alignment** Existing literature (REPR-ALIGN, PreDiff-LM) suggests that AR-to-DLM adaptation is not just about learning a new decoding path but about preserving semantic representations. However, no study has linked the *degree* of this preservation (measured by CKA) to *inference-time efficiency gains*. By quantifying S_{rep} **per layer** and aggregating it, we move beyond "DLMs are slower" to a mechanistic explanation: If a DLM retains AR structure across layers (S_{rep} high), its output distribution likely remains closer to the AR manifold, making a small AR scorer a more effective "filter" for noise. Conversely, if S_{rep} is low, the DLM has diverged significantly, and the AR scorer may be less aligned. Crucially, by introducing a **matched scorer** (TinyLlama for LLaMA-based DLMs) and a **non-AR control** (BERT), we isolate the effect of representation preservation from the confounding variables of scorer-architecture mismatch and general linguistic plausibility. This allows us to test whether the "quality signal" is a general property of language representations, specific to the model's own weights, or specific to the causal (AR) structure. If the mismatched scorer (GPT-2) performs poorly on LLaMA-based DLMs even when S_{rep} is high, it indicates that scorer alignment is a critical factor in reranking efficiency, refining the hypothesis.

2. **Isolation of "Headroom" Sources with Rigorous Baselines and Oracle Quantification** The "accuracy headroom" is the gap between Top-1 and Oracle (Best-of-N). This headroom can be closed by:

   a) **Better Denoising:** Increasing steps refines the sample, reducing variance.
   b) **Better Selection:** Reranking exploits the existing variance in the candidate set. By measuring ΔQ_{steps} and ΔQ_{rerank} separately, we determine which component of the headroom is "fixable" by computation (steps) vs. which is "fixable" by selection (reranking). If ΔQ_{rerank} is large and ΔQ_{steps} is small, the headroom is due to suboptimal exploitation of the DLM's diverse samples, not insufficient denoising. If ΔQ_{steps} dominates, the DLM needs more time to converge. The use of a **standardized Top-1 baseline** (always the

first generated candidate) ensures that the reranking gain is measured against a consistent "no-selection" baseline. Explicitly reporting the **Oracle (Best-of-10)** performance allows us to quantify exactly how much of the total available headroom is recovered by the reranker versus additional denoising steps, providing a more complete picture of the efficiency trade-off.

3. **Efficiency of Lightweight Reranking with Controlled Variables and Real-World Latency** Using GPT-2-small (124M) and TinyLlama-1.1B is a critical innovation for efficiency. Larger AR scorers would be as expensive as the DLM itself, negating the efficiency benefit. These small scorers are small enough to be negligible in FLOPs but large enough to capture basic syntactic and semantic coherence. By using both **mismatched** and **matched** scorers, and a **non-AR control**, we test the robustness of the selection mechanism. If the matched scorer yields significantly higher gains than the mismatched scorer for LLaMA-based DLMs, it suggests that the "quality signal" is not universal but depends on architectural alignment. If the non-AR control (BERT) yields similar gains, it suggests the benefit is from general linguistic quality, not specifically AR representations. This insight is valuable for future work on inference-time optimization, as it highlights the importance of scorer selection. To address the concern about the "negligible latency" claim, we explicitly measure and report **Wall-Clock Time** for both the generation and scoring phases, **separating** the Text-Bridge overhead (decoding and retokenizing) from the scorer forward pass. This ensures that the "efficiency" claim is grounded in real-world performance, not just theoretical FLOPs, and allows us to identify if the Text-Bridge is a bottleneck.

4. **Rigor and Validity**
   - **Controlled Variables:** We fix the candidate count ($N = 10$) and step budgets $(16, 32, 64, 128)$ across all models. This ensures that differences in gain are due to the model's properties, not hyperparameter tuning.
   - **Unified Metric:** Normalizing **per-task** across all models and conditions prevents a single high-variance task (like HumanEval) from dominating the efficiency comparison, ensuring that each task contributes equally to the final average.
   - **Compute Accounting:** We explicitly state that the reranker cost is negligible in FLOPs, but we also report **Wall-Clock Time** to account for real-world latency, including the Text-Bridge overhead. This ensures that the "efficiency" claim is valid and comprehensive.
   - **CKA Baseline:** The inclusion of a **random baseline** (floor) and **identity baseline** (ceiling) ensures that the S_{rep} score is truly indicative of "preservation" rather than a general property of the architecture. The random baseline is more robust than layer shuffling, as it does not preserve any global distribution properties.
   - **Generalizability:** By including both GPT-2-based and LLaMA-based DLMs, and both coding (HumanEval) and commonsense (SIQA) tasks, the method generalizes across architectures and task types. The CKA metric is architecture-agnostic, making the method applicable to future DLMs.
   - **Scorer Capacity Check:** By testing a subset with a larger scorer (LLaMA-2-7B) and a non-AR control (BERT), we address the concern that the observed effect might be a byproduct of the small scorers' limited capacity or general linguistic quality. If the correlation holds with the larger AR scorer but not the non-AR control, it strengthens the claim that the effect is related to AR representation preservation.
   - **Statistical Robustness:** By using **bootstrapping** and **leave-one-out** analysis, we address the small sample size ($N = 5$) concern, providing confidence intervals and checking for outlier sensitivity. This makes the correlation analysis more rigorous and less prone to misinterpretation.

5. **Feasibility**
   - **Compute:** 96GB VRAM is sufficient for 8-bit quantized 7B models with activation checkpointing. Generating 10 candidates × 4 steps × 200 prompts is manageable in 3 weeks with 3 GPUs. The subset scoring with LLaMA-2-7B and BERT is limited to 50 prompts to keep compute costs low.
   - **Timeline:** 10 weeks is tight but feasible given the parallelization of CPU-bound analysis

(CKA, metrics) and GPU-bound generation.
- **Inference-Only:** No training is required. CKA is a forward pass. Reranking is a forward pass. This strictly adheres to the constraint.

This method provides a clear, testable, and rigorous answer to whether representation preservation dictates the efficiency of reranking vs. step-increase, offering a fundamental insight into the internal dynamics of diffusion language models while controlling for critical confounding variables such as scorer alignment, general linguistic quality, and real-world latency.

Experiment design.

1. Representation Preservation Quantification (CKA with Trajectory Verification)**

- **Corpus Construction:** Construct a fixed, held-out probing corpus of 5,000 text segments, stratified as follows: 2,000 general text (WikiText-103), 1,500 code (HumanEval/MBPP), and 1,500 reasoning/commonsense (GSM8K/SIQA prompts). All segments are tokenized and padded/truncated to 128 tokens using the respective model's tokenizer.
- **Feature Extraction (Clean State & Trajectory Check):**
  - **Primary State (x_0):** For each Autoregressive (AR) base model (M_{AR}) and its corresponding Diffusion Language Model (M_{DLM}), perform a single forward pass on the **ground-truth clean text** (teacher-forcing).
    - *DLM Implementation:* For DiffuLLaMA, this corresponds to setting the input state to the fully denoised sequence (no mask tokens) and performing a standard forward pass. This measures the representation capacity for clean language.
  - **Supplementary Trajectory Check:** To address the concern that x_0 may not reflect the internal state during active generation, we perform a supplementary CKA analysis on a subset of 500 samples (100 per domain) at **intermediate denoising steps** ($t = 0.2, 0.5, 0.8$ of the total step budget $K = 32$).
    - *Procedure:* We run the DLM sampling loop for these 500 samples, capturing the hidden states at the specified intermediate steps. We compute CKA between the DLM's intermediate hidden states and the AR model's clean hidden states.
    - *Purpose:* This confirms whether the "preservation" measured at x_0 is maintained during the generative process or if representations drift significantly as noise is introduced. We report the correlation between $S_{rep}^{x_0}$ and $S_{rep}^{t=0.5}$ to validate the primary metric.
  - Extract hidden states $H_{DLM}^{(l)} \in \mathbb{R}^{N \times d}$ and $H_{AR}^{(l)} \in \mathbb{R}^{N \times d}$ for all layers $l = 1, \ldots, L_{total}$.
- **CKA Computation:**
  - Compute Linear CKA between corresponding layers: $S_{rep}^{(l)} = \text{CKA}(H_{AR}^{(l)}, H_{DLM}^{(l)})$.
  - Calculate the **Aggregate Preservation Score** S_{rep} as the mean of $S_{rep}^{(l)}$ across all layers.
  - Calculate **Baselines**:
    - *Floor (Random):* CKA between M_{DLM} hidden states and a **randomly initialized model** of the same architecture.
      - *Initialization Specification:* The random model is initialized using the **exact same weight initialization distribution** (e.g., `nn.init.normal_` with specific mean/std as defined in the HuggingFace model config) as the original AR base model. Crucially, this random model is **frozen** (requires_grad=False) and processed in the same inference mode as the DLM to avoid any accidental normalization shifts or gradient updates. We provide the exact initialization script in the supplementary materials.
    - *Ceiling (Identity):* CKA between M_{AR} and itself (sanity check, should be ~1.0).
  - Report Normalized Preservation Score: $S_{rep}^{norm} = \frac{S_{rep} - S_{floor}}{S_{ceiling} - S_{floor}}$.

2. Candidate Generation Protocol

- **Models:** DiffuGPT-S, DiffuGPT-M, DiffuLLaMA (6.74B), Dream-7B, LLaDA-8B, DiffuCoder-7B.
- **Tasks:** HumanEval (coding), GSM8K (math), SIQA (commonsense), WinoGrande (commonsense).
- **Prompts:**

  - **Standardized Subset:** Use a **fixed subset of 100 prompts per task** for **all** models.
  - **Reproducibility:** The prompt subset is selected via random sampling from the official test sets using a **fixed global seed (`seed=42`)**. The list of prompt IDs is logged to ensure exact replication.
  - *Feasibility Note:* For 7B+ models, generation is performed with 8-bit quantization and activation checkpointing. If VRAM constraints force a reduction for specific 8B models, reduce to 50 prompts but **report the standard error of the mean** to quantify uncertainty. We prioritize consistent sample size for cross-model comparability.
- **Generation Parameters:**
  - Step Budgets $K \in \{16, 32, 64, 128\}$.
  - Candidate Count $N = 10$ per prompt, generated by varying the random seed at the initial noise stage.
  - **Seed Management:** To ensure reproducibility, the seed for the j-th candidate of the i-th prompt is defined as `seed = i * 10 + j` (where i is the prompt index and $j \in \{0, \ldots, 9\}$).
  - **Top-1 Baseline:** The first generated candidate (y_1, where $j = 0$) for each prompt is designated as the "Top-1" reference.
  - **Oracle Baseline:** The best candidate among the 10 based on ground-truth evaluation metrics (pass@1 for code, exact match for math, accuracy for commonsense).
- **Compute Accounting & VRAM Safety:**
  - Measure Wall-Clock Time for generation (T_{gen}) for each ($Model, Task, K$) configuration.
  - **VRAM Safety:** For the largest models (LLaDA-8B, Dream-7B), implement a **strict sequential loop** (batch size = 1) for candidate generation to avoid OOM errors.
  - **OOM Handling:** Implement a `try-except` block around the generation loop. If an OOM error occurs, the system automatically clears the cache, reduces the batch size (if >1, otherwise retries with smaller sequence length if applicable, or skips and logs the failure), and resumes from the next prompt. This ensures the 100-prompt loop does not crash entirely.
  - **Timing Integrity:** The reported T_{gen} must reflect the **actual production cost** of the batch-size-1 sequential loop. We do not optimize the timing by batching for the sake of the metric, ensuring the efficiency comparison against reranking is fair and conservative.

3. Lightweight Reranking Protocol

- **Scorers:**
  - *Mismatched Scorer:* GPT-2-small (124M) for all models.
  - *Matched Scorer:* TinyLlama-1.1B for LLaMA-based DLMs (DiffuLLaMA, Dream-7B, LLaDA-8B, DiffuCoder-7B). GPT-2-small serves as the matched scorer for DiffuGPT.
  - *Control Scorer:* BERT-base-uncased (encoder-only) for a subset (10 prompts per task) of DiffuLLaMA only.
- **Text-Bridge Scoring Process:**
  1. Decode DLM token IDs to raw text.
  2. **Normalization:** Strip all special tokens (e.g., `<|startoftext|>`, `<pad>`, `<|eot_id|>`). **Normalize whitespace**: collapse multiple spaces/newlines into single spaces. **Crucially, apply this normalization consistently to both the prompt and the candidate** before re-tokenization to avoid shifting the alignment of the sequence.
  3. Re-tokenize text and prompt using the *Scorer's* tokenizer.
  4. Truncate to scorer's max context length (1024 for GPT-2, 2048 for TinyLlama).
  5. Compute Negative Log-Likelihood (NLL) of the candidate sequence conditioned on the prompt.
  6. **Scoring Definition:** The NLL is calculated as the **sum of log-probabilities of the candidate tokens only**, conditioned on the prompt tokens. The prompt is treated as a fixed prefix (teacher-forced) and its probability is not included in the score. This isolates the quality of the generated response.
  7. Select candidate with minimum NLL: $y^*_{scorer} = \arg\min_{y_i} \text{NLL}(y_i|\text{prompt})$.
- **BERT Control Metric (Specific & Validated):**
  - To maintain validity, the BERT scorer uses the **average masked token log-probability** (pseudo-log-likelihood, PLL).

  - **Procedure:**
    1. Concatenate prompt and candidate text.
    2. **Masking Strategy:** Mask 15% of the tokens **only in the candidate portion** of the sequence. The prompt tokens remain unmasked to serve as context, mirroring the AR scorer's conditional setup.
    3. Use a fixed random seed for mask selection to ensure consistency.
    4. Compute the log-probability of the *original* tokens at the masked positions using the BERT MLM head.
    5. **Normalization:** Calculate the **average** log-probability per masked token: $\text{Score}_{BERT} = \frac{1}{N_{masked}} \sum_{i \in \text{masked}} \log p(x_i | \text{context})$. This prevents longer candidates from being penalized/rewarded simply due to the summation of log-probabilities.
  - **Control Validation:** Before using BERT scores to contrast with AR scorers, we compute the **Spearman rank correlation** between the BERT scores and the ground-truth quality metrics (pass@1, accuracy) for the 10 candidates. We report this correlation to ensure the BERT control is a valid baseline for "general linguistic quality." If the correlation is negligible, we interpret the BERT results with caution, noting it may not capture the specific quality dimensions relevant to the tasks.
- **Latency Accounting:**
  - Measure T_{bridge} (decoding + re-tokenization).
  - Measure T_{score} (scorer forward pass).
  - Total Rerank Latency: $T_{rerank} = T_{bridge} + T_{score}$.
  - *Optimization:* Implement vectorized batch processing for T_{bridge} and T_{score} to minimize overhead, but report the un-optimized single-sample latency as the primary metric for "negligibility" claims to be conservative.

4. Evaluation and Metrics

- **Task Metrics:**
  - HumanEval: pass@1.
  - GSM8K: Exact Match Accuracy.
  - SIQA/WinoGrande: Multiple Choice Accuracy.
- **Unified Quality Metric (Q):**
  - Normalize each task's score to $[0, 1]$ across all models and conditions: $Q_{norm,task} = \frac{Q_{raw} - Q_{min}}{Q_{max} - Q_{min}}$.
  - $Q = \text{Average}(Q_{norm,HumanEval}, Q_{norm,GSM8K}, Q_{norm,SIQA}, Q_{norm,WinoGrande})$.
- **Efficiency Metrics (Compute-Matched):**
  - **Reranking Gain:** $\Delta Q_{rerank} = Q_{reranked} - Q_{top1}$ (at fixed K).
  - **Step Gain:** $\Delta Q_{steps} = Q_{top1}(K_{high}) - Q_{top1}(K_{low})$.
  - **Marginal Efficiency:** Compare ΔQ_{rerank} per second of T_{rerank} vs. ΔQ_{steps} per second of additional T_{gen}.
  - **Fair Comparison:** To ensure a fair efficiency comparison, we calculate the **Total Compute** for each strategy:
    * Strategy A (Rerank at $K = 16$): $Cost_A = T_{gen}(16) + T_{rerank}$.
    * Strategy B (Increase Steps to K'): $Cost_B = T_{gen}(K')$.
    * We compare the quality gain ΔQ achieved for these two specific compute budgets. If $Cost_A \approx Cost_B$, we directly compare ΔQ_{rerank} and ΔQ_{steps}. If not, we report the **Quality per Second** ($\Delta Q / T_{total}$) for both.
  - **Reranking Advantage:** $A_{rerank}(M) = \Delta Q_{rerank,matched}(M) - \Delta Q_{steps}(M, 16 \to 32)$.
  - **Scorer Comparison Metric:** For comparing AR vs. BERT scorers, we use the **Spearman rank correlation** between the scorer's ranking of the 10 candidates and the ground-truth quality ranking, rather than raw score differences, to account for scale differences between PLL and NLL.

5. Statistical Analysis

- **Correlation:** Compute Pearson and Spearman correlation between S_{rep} and A_{rerank} across

the 5-6 model families.

- **Robustness & Interpretation:**
  - Bootstrap (1,000 resamples) for correlation confidence intervals.
  - Leave-one-out analysis to check for outlier sensitivity.
  - Stratify by architecture (GPT-2 vs. LLaMA).
  - **Interpretation Note:** Given $N = 5 - 6$, we focus on the **directionality** and **trend** of the correlation rather than strict statistical significance (p-values). We explicitly state that the correlation is exploratory and indicative of a potential relationship, not a definitive causal proof.
- **Scorer Analysis:**
  - Compare Gains from Mismatched vs. Matched scorers for LLaMA-based models.
  - Compare Gains from AR Scorers vs. BERT Control for DiffuLLaMA, using rank correlation as the primary comparison metric.

Why this design. This revised experiment addresses the specific feedback from the review while maintaining the core scientific goal of linking representation preservation to inference-time efficiency.

1. **Enhanced Validity of Representation Measurement:**
   - **Trajectory Verification:** We added a supplementary CKA analysis at intermediate denoising steps ($t = 0.2, 0.5, 0.8$) to confirm that the "preservation" measured at the clean state x_0 is actually maintained during the generative process. This addresses the concern that x_0 might be a static capacity measure that doesn't reflect the dynamic state during inference.
   - **CKA Baseline Precision:** We specified that the random initialization for the CKA floor baseline must use the **exact same weight initialization distribution** and that the model is **frozen** to avoid normalization shifts. We also committed to providing the initialization script for reproducibility.
2. **Improved Control Validity (BERT):**
   - **Masking Strategy:** We clarified that the 15% masking for the BERT pseudo-log-likelihood is applied **only to the candidate tokens**, leaving the prompt as context. This mirrors the conditional setup of the AR scorer.
   - **Normalization:** We specified that the BERT score is the **average** log-probability per masked token, preventing length bias.
   - **Control Validation:** We added a step to compute the rank correlation between BERT scores and ground-truth quality to validate that the BERT control is a meaningful proxy for "general linguistic quality" before using it to contrast with AR scorers.
   - **Comparison Metric:** We shifted the comparison between AR and BERT scorers to use **Spearman rank correlation** rather than raw score differences, acknowledging that PLL and NLL have different scales and distributions.
3. **Feasibility and Stability Enhancements:**
   - **VRAM Safety & OOM Handling:** We mandated a strict sequential loop (batch size = 1) for 8B models and implemented a `try-except` block to gracefully handle OOM errors, ensuring the 10-week timeline is protected.
   - **Timing Integrity:** We explicitly stated that the reported T_{gen} must reflect the actual production cost of the batch-size-1 loop, ensuring the efficiency comparison is fair and conservative.
4. **Clarity and Reproducibility:**
   - **Whitespace Normalization:** We clarified that whitespace normalization is applied consistently to both prompt and candidate to avoid alignment shifts.
   - **Interpretation Caution:** We added a note in the statistical analysis section to focus on directionality and trends rather than p-values, given the small sample size, to prevent over-interpretation of the correlation.

This design provides a rigorous, feasible, and reproducible framework to answer whether representation preservation dictates the efficiency of lightweight reranking, offering fundamental insights into the trade-offs between denoising depth and selection quality in diffusion language models.

Method	Clarity	Validity	Rigorousness	Innovativeness	Generalizability
rating	5	5	5	4	5

Experiment	Clarity	Validity	Robustness	Feasibility	Reproducibility
rating	5	5	5	5	5

Weakest point per the reviewers (Innovativeness, 4/5). The method is exceptionally well-described and logically sound. To further enhance the impact:

- **CKA Sensitivity:** While per-layer CKA is a strong choice, the authors might consider if "representation preservation" is more critical in the early layers (semantic) or late layers (syntactic/token-level) for the reranker's success.
- **Sample Size:** The correlation analysis relies on $N = 5$ model families. While bootstrapping and leave-one-out analysis are included to mitigate this, the authors should be cautious about over-claiming a "law" of DLMs and instead frame it as a strong empirical trend.
- **BERT Baseline:** For the BERT-base control, using the MLM pseudo-log-likelihood (sum of log-probs of tokens given context) would be a more direct and fair comparison to the AR scorer's NLL than MSE of [CLS] embeddings.

Idea 1.2 — P1-M6 (overall 4.9/5)

Method.

1. Core Concept: Attention-Symmetry Reranking Advantage (ASRA)**

Previous methods (CKA, DTD, SGIG) attempted to measure representation preservation by comparing hidden states of the DLM and AR model on *clean* text or by analyzing the denoising trajectory. These approaches suffer from a fundamental confound: **the difference in attention masks**.

- The AR model uses **causal** attention (left-to-right).
- The DLM uses **bidirectional** (or masked) attention.

Even if a DLM perfectly preserves the *semantic* content of the AR model, its hidden states will differ structurally because the DLM has access to future context. Consequently, global CKA or Procrustes alignment may measure the "price of bidirectionality" rather than the "loss of AR structure." This makes it difficult to determine if reranking is efficient because of preserved *semantics* or simply because the scorer is also an AR model (sharing the causal prior).

We propose **Attention-Symmetry Reranking Advantage (ASRA)**, a method that isolates the **semantic preservation** of the AR backbone from the **structural change** of bidirectional attention. We do this by measuring the **Asymmetry Gap**: the difference in how much the DLM "uses" future context versus how much the AR model "relies on" past context.

Hypothesis:

- **High Semantic Preservation / Low Asymmetry:** If a DLM has been successfully adapted from an AR model (e.g., via REPR-ALIGN or strong continued pre-training), its representations should remain largely driven by the *past* context (the AR prior), with the *future* context providing only minor refinement. In this case, the "accuracy headroom" is due to stochastic sampling among valid, AR-consistent continuations. A lightweight AR scorer, which naturally models the past-to-future dependency, will be highly effective at reranking.
- **Low Semantic Preservation / High Asymmetry:** If the DLM has drifted significantly from the AR prior, its representations may rely heavily on *future* context to resolve ambiguity (true bidirectional modeling). In this case, an AR scorer (which cannot see the future) may be "blind" to the DLM's internal logic, making reranking less effective. Additional denoising steps, which allow the DLM to refine its bidirectional view, may be more efficient.

2. Metric Definition: Attention Asymmetry Index (AAI)

Instead of comparing DLM and AR hidden states directly, we quantify how much the DLM's representation at a given token position depends on the **future** context relative to the **past** context. We compare this to the AR model's inherent dependency on the past.

Step 1: Causal and Anticausal Decomposition of DLM Representations For a DLM M_{DLM} and a sequence $x = (x_1, \ldots, x_L)$:

1. **Full Bidirectional Hidden State:** Compute $H_{full} \in \mathbb{R}^{L \times d}$ using standard bidirectional attention (the DLM's native mode).
2. **Causal-Only Hidden State:** Compute $H_{causal} \in \mathbb{R}^{L \times d}$ by modifying the attention mask of the DLM to be strictly causal (upper-triangular zero). This simulates the DLM acting as an AR model.
3. **Anticausal-Only Hidden State:** Compute $H_{anticausal} \in \mathbb{R}^{L \times d}$ by modifying the attention mask to be strictly anticausal (lower-triangular zero, looking only to the right). This isolates the "future" information.

Note: This requires three forward passes per sequence. Since these are single forward passes (not denoising loops), the cost is negligible.

Step 2: Quantifying the "Future" Contribution Define the **Future Sensitivity** of the DLM at position t as the cosine distance between the full bidirectional state and the causal-only state:

$$D_{future}(t) = 1 - \cos(H_{full}[t], H_{causal}[t])$$

A high $D_{future}(t)$ indicates that the DLM's representation at position t changes significantly when future tokens are removed, implying strong reliance on bidirectional context.

Define the **Past Reliance** of the AR model M_{AR} at position t as the cosine similarity between the AR model's hidden state and a "shuffled" context (to establish a baseline of non-past reliance), or more simply, we use the AR model's own causal structure as the reference for "standard" language processing.

Step 3: Attention Asymmetry Index (AAI) We define the AAI for a model M as the average difference in how much the model's representations are "disturbed" by the removal of future context, normalized by the model's total representational variance.

$$\text{AAI}(M_{DLM}) = \frac{1}{L} \sum_{t=1}^{L} D_{future}(t)$$

- **Low AAI:** The DLM's representations are stable even without future context. This suggests the DLM is effectively an AR model with a bidirectional interface (high AR preservation). The "extra" information from the future is minimal.
- **High AAI:** The DLM's representations are highly sensitive to future context. This suggests the DLM has developed distinct bidirectional capabilities that diverge from the AR prior.

Why this is different from CKA/Procrustes:

- CKA compares H_{DLM} (bidirectional) vs H_{AR} (causal). The difference is confounded by the mask.
- AAI measures $H_{DLM}(bidirectional)$ vs $H_{DLM}(causal)$. It measures **how much the DLM *itself* uses the bidirectionality it was trained to use**. If the DLM preserves the AR prior, it should *not* need the future context as much as a true bidirectional model would.

3. Efficiency Comparison: Reranking vs. Denoising

We test whether **AAI** predicts the **Compute-Normalized Efficiency Ratio (CNER)** for reranking.

Quality Gains:

- $Q_{rerank}(M, K = 16)$: Accuracy of the best candidate (by AR scorer NLL) among $N = 10$ candidates at $K = 16$.

- $Q_{top1}(M, K = 16)$: Accuracy of the first candidate at $K = 16$.
- $Q_{top1}(M, K = 128)$: Accuracy of the first candidate at $K = 128$.
- $\Delta Q_{rerank} = Q_{rerank}(16) - Q_{top1}(16)$.
- $\Delta Q_{steps} = Q_{top1}(128) - Q_{top1}(16)$.

Compute Costs (FLOPs):

- $C_{gen}(K)$: FLOPs for one candidate at K steps. For a Transformer, this is approx $K \times (2 \cdot P \cdot L)$, where P is parameter count and L is sequence length. *Correction from previous reviews:* We explicitly multiply by K.
- C_{score}: FLOPs for scoring one candidate with the AR scorer. Approx $2 \cdot P_{scorer} \cdot L$.
- $Cost_{rerank} = N \cdot C_{gen}(16) + N \cdot C_{score}$.
- $Cost_{steps} = C_{gen}(128)$.

CNER Definition:

$$\text{CNER}(M) = \frac{\Delta Q_{rerank}/Cost_{rerank}}{\Delta Q_{steps}/Cost_{steps}}$$

Hypothesis:

- **Low AAI (High AR Preservation):** The DLM's candidates are semantically consistent with the AR prior. The AR scorer, which shares this causal geometry, can easily identify the best candidate. Reranking is efficient. CNER > 1.
- **High AAI (Low AR Preservation):** The DLM relies on future context to generate coherent text. The AR scorer, lacking this future view, may rank candidates poorly (e.g., selecting a locally plausible but globally incoherent continuation). Reranking is less effective. Denoising steps, which refine the bidirectional view, are more efficient. CNER < 1.

4. Experimental Protocol

- **Models:** DiffuGPT-S/M, DiffuLLaMA, Dream-7B, LLaDA-8B.
- **AR Scorers:**
  - GPT-2-based DLMs: GPT-2-Small (124M).
  - LLaMA-based DLMs: LLaMA-2-350M (350M).
- **Benchmarks:** HumanEval (code), GSM8K (math), WinoGrande (commonsense).
- **Step Budgets:** $K \in \{16, 128\}$.
- **Candidates:** $N = 10$ per prompt at $K = 16$.

5. Procedure

1. **AAI Computation (Weeks 1-3):**
   - **Data:** A held-out corpus of $N = 2000$ segments from the Pile dataset (book, github, arxiv subsets to cover diverse domains).
   - **For each DLM:**
     - For each segment x:
       - Run 3 forward passes: Bidirectional, Causal-Masked, Anticausal-Masked.
       - Compute $D_{future}(t)$ for each position.
       - Average to get AAI(M).
   - **Control:** Compute AAI for the original AR base models (GPT-2, LLaMA-2). Since they are inherently causal, $H_{causal} = H_{full}$, so $D_{future} = 0$ and AAI(AR) $= 0$. This provides a baseline of "perfect AR preservation" (zero future reliance).
   - **Validation:** Correlate AAI with global CKA. We expect a negative correlation (High CKA → Low AAI), but AAI should be more specific to the *mechanism* of preservation.
2. **Efficiency Measurement (Weeks 4-7):**
   - **Generation:**
     - For 200 prompts per benchmark:
       - Generate 10 candidates at $K = 16$.
       - Generate 1 candidate at $K = 128$.
   - **Scoring:**
     - Score 10 candidates at $K = 16$ with the AR scorer.

- Select best candidate.
- **Evaluation:**
  - Compute accuracies for $Q_{rerank}(16)$, $Q_{top1}(16)$, $Q_{top1}(128)$.
  - Compute ΔQ_{rerank} and ΔQ_{steps}.
- **Compute Accounting:**
  - Calculate $Cost_{rerank}$ and $Cost_{steps}$ using the explicit K multiplier.
  - Compute CNER.

3. **Statistical Analysis (Weeks 8-9):**
   - **Primary Test:** Correlate **AAI** with **CNER** across the 4 models.
     - Hypothesis: **Negative Correlation**. Low AAI (high AR preservation) → High CNER (reranking is efficient).
     - Perform Pearson correlation and linear regression: $\text{CNER} = \beta_0 + \beta_1 \text{AAI} + \epsilon$.
   - **Stratified Analysis:**
     - Test if the relationship holds within GPT-2-based and LLaMA-based families separately.
     - Test if the relationship varies by task type (Code vs. Math vs. Commonsense). We hypothesize that Code (structured, local dependencies) will show stronger AAI-CNER links than Commonsense (global dependencies).
   - **Sensitivity Analysis:**
     - Vary N (5, 10, 20) to check stability of CNER.
     - Vary K_{steps} (16$→64$vs16→$128) to see if the efficiency advantage of reranking persists at higher step budgets.

6. Why This is Different and Better

- **Isolates Semantic from Structural Change:** Unlike CKA, which conflates semantic drift with the causal/bidirectional mask difference, AAI measures the *functional* reliance on future context. It directly tests if the DLM is "acting like an AR model" (low AAI) or "acting like a true bidirectional model" (high AAI).
- **Mechanistic Insight:** It provides a clear mechanistic explanation for why reranking works. If the DLM preserves the AR prior, it doesn't need the future context to be coherent, so an AR scorer (which lacks future context) is a valid proxy for quality. If the DLM relies on future context, the AR scorer is a poor proxy.
- **Compute-Rigorous:** Explicitly accounts for the K multiplier in denoising costs, addressing the previous feedback on FLOPs approximation.
- **Generalizable:** The method is model-agnostic. It can be applied to any DLM/AR pair by simply running the three masked forward passes.

7. Execution Plan (10 Weeks)

- **Weeks 1-2:** Setup. Implement masked forward passes (Causal, Anticausal) for DLMs. Verify AAI computation on a small subset.
- **Weeks 3-4:** AAI Computation. Run AAI analysis for all 4 DLMs on the 2000-sample corpus.
- **Weeks 5-7:** Generation & Scoring.
  - *GPU 1:* DiffuGPT-S/M.
  - *GPU 2:* DiffuLLaMA (8-bit).
  - *GPU 3:* Dream/LLaDA (8-bit).
  - *CPU:* Scoring with GPT-2-small and LLaMA-2-350M.
- **Weeks 8-9:** Analysis. Compute CNER. Perform regression of CNER on AAI. Stratify by family and task.
- **Week 10:** Writing.

Why this method. The research problem asks whether AR representation preservation determines the efficiency of reranking vs. denoising. Previous methods failed to rigorously separate *semantic* preservation from *structural* (attention mask) differences.

This method, **ASRA**, addresses this by:

1. **Defining AAI:** A metric that measures the DLM's functional reliance on future context. This isolates the "AR-like" behavior (low future reliance) from "bidirectional" behavior (high future reliance).

2. **Linking to Efficiency:** It hypothesizes that DLMs with low AAI (high AR preservation) will benefit more from AR-based reranking because the scorer's causal view is a valid proxy for the DLM's internal logic.
3. **Rigorous Compute Accounting:** It explicitly includes the denoising step multiplier in FLOPs, ensuring the efficiency comparison is fair.
4. **Mechanistic Clarity:** It provides a clear, testable hypothesis about *why* reranking works: it works when the DLM's quality is determined by causal (past-to-future) coherence, which the AR scorer can evaluate. It fails when quality depends on bidirectional (global) coherence, which the AR scorer cannot see.

Experiment design.

Title:** Decoupling Semantic Preservation from Attention Structure: An Attention-Symmetry Analysis of Reranking Efficiency in Diffusion Language Models

1. Objective To test whether the **Attention Asymmetry Index (AAI)**—a metric quantifying a Diffusion Language Model's (DLM) functional reliance on future context—predicts the relative efficiency of lightweight AR-based reranking versus increasing denoising steps. Specifically, we aim to validate the hypothesis that DLMs with low AAI (high AR semantic preservation) exhibit a higher **Compute-Normalized Efficiency Ratio (CNER)** for reranking, whereas DLMs with high AAI (strong bidirectional dependence) benefit more from additional denoising steps.

2. Experimental Design & Variables

- **Independent Variable (Model Property):**
  - **Attention Asymmetry Index (AAI):** Calculated for each DLM. AAI measures the cosine distance between the DLM's bidirectional hidden states and its causal-masked hidden states, isolating the "future context" contribution from structural mask differences.
- **Independent Variable (Inference Condition):**
  - **Reranking Strategy:** Generate $N = 10$ candidates at $K = 16$ steps; select the best using an AR scorer's negative log-likelihood (NLL).
  - **Denoising Strategy:** Generate 1 candidate at $K = 128$ steps (no reranking).
- **Dependent Variable:**
  - **Quality Gain (ΔQ):**
    - $\Delta Q_{rerank} = Q_{rerank}(K = 16) - Q_{top1}(K = 16)$
    - $\Delta Q_{steps} = Q_{top1}(K = 128) - Q_{top1}(K = 16)$
  - **Compute-Normalized Efficiency Ratio (CNER):**

$$\text{CNER} = \frac{\Delta Q_{rerank}/Cost_{rerank}}{\Delta Q_{steps}/Cost_{steps}}$$

    - $Cost_{rerank} = N \cdot K_{low} \cdot F_{fwd} + N \cdot F_{score}$
    - $Cost_{steps} = K_{high} \cdot F_{fwd}$
    - *(Note: F_{fwd} is the FLOPs of one forward pass for the DLM; F_{score} is the FLOPs of one forward pass for the AR scorer over the full sequence length L.)*
- **Control Variables:**
  - **Models:** 4 distinct DLMs with varying architectures and training regimes:
    1. **DiffuGPT-S/M** (GPT-2 based, ~124M/355M params)
    2. **DiffuLLaMA** (LLaMA-2 based, 6.74B params)
    3. **Dream-7B** (LLaMA-2 based, 7B params)
    4. **LLaDA-8B** (LLaMA-2 based, 8B params)
  - **AR Scorers:**
    - GPT-2-Small (124M) for GPT-2-based DLMs.
    - LLaMA-2-350M (350M) for LLaMA-based DLMs.
  - **Benchmarks:**
    - **Code:** HumanEval (200 prompts)
    - **Math:** GSM8K (200 prompts)
    - **Commonsense:** WinoGrande (200 prompts)
  - **Sequence Length:** Fixed maximum length L per benchmark to ensure comparable FLOPs (e.g., $L = 512$ for Code/Math, $L = 256$ for Commonsense).

- **Step Budgets:** $K \in \{16, 128\}$.

3. Procedure

Phase 1: AAI Computation (Weeks 1-3)

1. **Data Preparation:** Select 2,000 text segments from the **Pile dataset v2** (specifically the `book`, `github`, and `arxiv` subsets) with length $L = 512$. Use a fixed random seed (e.g., `seed=42`) for sample selection to ensure reproducibility.
2. **Forward Passes:** For each DLM M_{DLM} and each segment x:
   - Compute H_{full}: Standard forward pass with bidirectional attention mask.
   - Compute H_{causal}: Forward pass with strictly causal attention mask (upper-triangular zero, i.e., token t attends to $1 \dots t$).
   - Compute $H_{anticausal}$: Forward pass with strictly anticausal attention mask (lower-triangular zero, i.e., token t attends to $t \dots L$).
   - *Implementation Note:* These passes are performed on the **clean, original ground-truth tokens** from the Pile dataset. The input IDs remain constant across all three passes; only the attention mask changes.
   - *Positional Embedding Handling:* For the anticausal pass, positional embeddings are **kept absolute** (i.e., the embedding for token t remains the embedding for position t). We do not shift the sequence to make the rightmost token the "start". This ensures that $H_{anticausal}[t]$ represents the representation of token t when it only attends to tokens to its right, without introducing artifacts from re-indexing.
   - *Mask Specification:* The attention masks are implemented as additive tensors of shape (L, L). Masked positions (where attention is disallowed) are set to $-\infty$ (or a large negative value like $-1e9$ in FP16/FP32) before the softmax operation. Unmasked positions are set to 0.
3. **AAI Calculation:**
   - Compute per-position future sensitivity: $D_{future}(t) = 1 - \cos(H_{full}[t], H_{causal}[t])$.
   - Compute AAI: $\text{AAI}(M_{DLM}) = \frac{1}{L} \sum_{t=1}^{L} D_{future}(t)$.
   - *Layer Analysis:* Additionally, compute AAI per layer to identify if "AR-preservation" is stronger in early or later layers.
4. **Core Noise-Level Sensitivity Analysis:**
   - *Elevation from Exploratory:* To address the gap between clean-text structure and noisy-inference behavior, we compute AAI at two specific noise levels for a subset of 500 sequences per model.
   - For a given clean sequence x, generate a noisy version x_{noisy} by masking a fraction p of tokens (where p corresponds to diffusion time $t = 0.5$).
   - Compute H_{full}, H_{causal}, and $H_{anticausal}$ on x_{noisy}.
   - Compute AAI_{noisy} and compare it to AAI_{clean}.
   - *Hypothesis:* If AAI is a stable structural property, AAI_{noisy} should correlate strongly with AAI_{clean}. If the DLM relies on future context only to resolve noise, AAI might change significantly. We report the correlation coefficient $\rho(\text{AAI}_{clean}, \text{AAI}_{noisy})$ as a validity check.
5. **Control & Baseline Checks:**
   - **AR Baseline:** Verify that AAI for the original AR base models (GPT-2, LLaMA-2) is ≈ 0 (since $H_{full} \approx H_{causal}$ for causal models).
   - **Bidirectional Baseline:** Compute AAI for **BERT-base** (a purely bidirectional model) to establish the "High AAI" upper bound.
   - **Quantization Sensitivity:** Perform a sensitivity check on a subset of 100 samples comparing AAI values computed in FP16 vs. INT8 to verify that quantization does not introduce noise that masks the asymmetry gap. *Note: The primary AAI results for 7B+ models will be reported using FP16/FP32 weights if VRAM allows during the single-forward-pass phase; if 8-bit is required for memory, the quantization sensitivity check is mandatory to interpret the results.*

Phase 2: Generation & Scoring (Weeks 4-7)

1. **Candidate Generation (GPU):**

- For each of the 600 prompts (200 per benchmark) and each DLM:
  - Generate 10 candidates at $K = 16$ steps.
  - Generate 1 candidate at $K = 128$ steps.
- *Sampling:* Use nucleus sampling with temperature $T = 1.0$ and top-p 0.9.
- *Optimization:* Use 8-bit quantization for 6B+ models (DiffuLLaMA, Dream, LLaDA) to fit within 96GB VRAM. Implement dynamic batching with a maximum batch size of 4 candidates per forward pass for 7B+ models. Parallelize generation across the 3 GPU-enabled team members.

2. **Scoring (CPU/GPU):**
   - **Tokenizer Handling:** Decode the DLM output to text and then **re-tokenize it using the scorer's own native tokenizer** before computing NLL. This prevents alignment errors due to vocabulary mismatches.
   - For the 10 candidates at $K = 16$, compute NLL using the corresponding AR scorer (GPT-2-Small or LLaMA-2-350M).
   - Select the candidate with the lowest NLL as Q_{rerank}.
   - Use the first generated candidate as Q_{top1}.
   - *Scorer Sanity Check:* For a small subset (20 prompts), also score with a larger scorer (e.g., LLaMA-7B) to confirm that CNER trends are a property of the DLM's representations and not a limitation of the small scorer's capacity.
3. **Evaluation:**
   - **HumanEval:** Execute code in a sandboxed **Docker environment** (Python 3.10, standard libraries only) to ensure consistent "pass@1" metrics.
   - **GSM8K:** Extract final numerical answer; exact match metric.
   - **WinoGrande:** Multiple-choice accuracy.
   - **Normalization:** Normalize scores to [0,1] **per benchmark** using Min-Max scaling: $Q_{norm} = \frac{Q_{raw} - Q_{min}}{Q_{max} - Q_{min}}$, where Q_{min} and Q_{max} are the minimum and maximum raw scores across *all* models, step budgets, and reranking conditions for that specific benchmark.
   - **Diversity & Headroom Analysis:**
     - Compute self-BLEU and token-level entropy for the $N = 10$ candidate pools.
     - **Oracle vs. Scorer Gap:** For each prompt, compute the **Oracle Score** (the best score among the 10 candidates, regardless of scorer ranking) and the **Scorer Score** (Q_{rerank}).
     - Calculate **Scorer Recall**: The fraction of prompts where the AR scorer selected the Oracle-best candidate.
     - *Analysis:* If ΔQ_{rerank} is low, we must distinguish whether it is because the Oracle Score is close to Top-1 (low headroom) or because the Scorer Recall is low (poor selection). We will report both metrics to disentangle "generation diversity" from "scorer effectiveness."

Phase 3: Statistical Analysis (Weeks 8-9)

1. **Compute CNER:** Calculate ΔQ_{rerank}, ΔQ_{steps}, and FLOPs costs for each model-benchmark pair.
2. **Primary Hypothesis Test:**
   - Perform Pearson correlation and linear regression: $\text{CNER} = \beta_0 + \beta_1 \text{AAI} + \epsilon$.
   - **Expected Result:** Negative correlation ($\beta_1 < 0$). Low AAI $\rightarrow$ High CNER (reranking is more efficient).
   - Report the **standard error** of the quality gain ΔQ across the 200 prompts per benchmark to add statistical weight to the CNER.
3. **Stratified Analysis:**
   - Separate models by family (GPT-2 vs. LLaMA) to check for architecture-specific effects.
   - Separate by task type (Code vs. Math vs. Commonsense) to test if local dependency (Code) shows stronger AAI-CNER links than global dependency (Commonsense).
   - Analyze if the "accuracy headroom" varies by task (e.g., functionally identical but syntactically different candidates in HumanEval vs. logically divergent candidates in GSM8K).
4. **Sensitivity Analysis:**
   - Vary N (5, 10, 20) to ensure CNER stability.

- Compare CNER at $K_{high} = 64$ vs. $K_{high} = 128$ to check if the efficiency advantage of reranking persists at higher step budgets.
- **NLL Variance Check:** Correlate the **variance of NLLs** across the 10 candidates with ΔQ_{rerank}. If NLL variance is near zero, reranking gain is likely noise. We expect a positive correlation between NLL variance and reranking gain, confirming that the scorer is making meaningful distinctions.

4. Feasibility & Resource Management

- **Compute:**
  - **AAI Phase:** 3 forward passes per sequence. 2000 sequences × 4 models × 3 passes = 24,000 forward passes. For 7B models, this is $\approx 24,000 \times 14$ GFLOPs ≈ 336 TFLOPs. On RTX 6000 Pro (~100 TFLOPS FP16), this is $<$ 1 hour per model. Total AAI phase: $<$ 4 hours GPU time. *Note: AAI is computed on FP16 weights where possible to ensure geometric fidelity; if 8-bit is used, the sensitivity check validates the result.*
  - **Generation Phase:**
    - DiffuGPT-S/M: Small models, fast. Negligible time.
    - DiffuLLaMA (6.74B): $600 \times 10 \times 16 + 600 \times 1 \times 128 = 96,000 + 76,800 \approx 173,000$ forward passes. $173,000 \times 14$ GFLOPs ≈ 2.4 PFLOPs. At 100 TFLOPS, ≈ 24 hours. With 8-bit quantization and parallelization, feasible within 3 weeks.
    - Dream/LLaDA (7-8B): Similar to DiffuLLaMA. Total generation time $\approx 3-4$ weeks with parallelization.
- **Team:**
  - 3 GPU users: Handle generation for the 4 large models in parallel batches. **Priority:** Generation is the bottleneck; start immediately in Week 4.
  - 4 CPU users: Handle AAI computation (CPU is sufficient for small models, GPU for large), scoring (GPT-2/LLaMA-350M fit in CPU/RAM), evaluation harnesses, and statistical analysis.

5. Deliverables

- A dataset of AAI scores (global, per-layer, and noise-sensitive) for 4 DLMs.
- A dataset of CNER scores for 4 DLMs across 3 benchmarks.
- Regression analysis linking AAI to CNER.
- Analysis of Scorer Recall vs. Oracle Headroom.
- A paper section detailing the "Attention-Symmetry Reranking Advantage" and its implications for inference-time compute allocation in DLMs.

Why this design. This experiment design directly addresses the research problem by isolating the **semantic preservation** of AR representations from the **structural change** of attention masks, which is a critical confound in previous methods like CKA. By introducing the **Attention Asymmetry Index (AAI)**, we create a mechanistic metric that quantifies how much a DLM actually *uses* its bidirectional capability. If a DLM has a low AAI, it implies that the model's representations are largely driven by the past context (the AR prior), meaning the "accuracy headroom" is due to stochastic sampling among AR-consistent continuations. In this case, a lightweight AR scorer, which shares this causal geometry, should be highly effective at reranking, leading to a high CNER. Conversely, a high AAI suggests the DLM relies on future context for coherence, making an AR scorer (which lacks future view) a poor proxy, and thus additional denoising steps (which refine the bidirectional view) should be more efficient, leading to a low CNER.

The design is **robust** because it controls for model architecture by using family-specific scorers and stratifies analysis by task type. It is **feasible** because the AAI computation requires only single forward passes (negligible cost), and the generation phase is limited to $K \leq 128$ steps, which is manageable on a single high-end GPU with quantization and parallelization. The **compute accounting** is rigorous, explicitly including the K multiplier in FLOPs calculations, ensuring a fair comparison between reranking and denoising. This experiment moves beyond empirical optimization to provide **fundamental insight** into why certain DLMs benefit more from inference-time selection than others, bridging the gap between representation alignment techniques and inference-time trade-off methods.

Revisions Addressing Review Feedback:

1. **Clarity & Reproducibility:**
   - **Anticausal Mask Precision:** Explicitly defined the attention mask implementation as an additive tensor with $-\infty$ (or $-1e9$) for masked positions and 0 for allowed positions. This removes ambiguity regarding "lower-triangular zero" (additive vs. multiplicative).
   - **Positional Embeddings:** Clarified that positional embeddings are **absolute** and not shifted for the anticausal pass. This prevents artificial variance introduced by re-indexing the sequence start.
   - **Dataset Specificity:** Retained the use of **Pile dataset v2** and a fixed random seed (`seed=42`).
   - **Normalization Scope:** Retained per-benchmark Min-Max scaling to prevent scale differences from skewing the unified metric.
2. **Validity & Depth of Analysis:**
   - **Noise-Level Sensitivity (Core):** Elevated the noise-level AAI check from "exploratory" to a **core component**. We now compute AAI on noisy inputs ($t = 0.5$) for 500 sequences to validate that the structural "AR-likeness" persists during the actual denoising process, addressing the gap between clean-text analysis and inference behavior.
   - **Headroom Disentanglement:** Added **Oracle vs. Scorer Gap** analysis. By computing the Oracle Score (best of 10) and Scorer Score, we can distinguish between a lack of diversity (Oracle $\approx$ Top-1) and poor scorer effectiveness (Oracle $\gg$ Scorer). This directly addresses the concern that low ΔQ_{rerank} might be due to generation limits rather than representation mismatch.
   - **NLL Variance Check:** Added a check correlating NLL variance across candidates with reranking gain to ensure that selection is not occurring in a flat NLL landscape (noise).
3. **Robustness & Statistical Rigor:**
   - **Quantization Fidelity:** Clarified that AAI is computed in FP16 where possible. If 8-bit is used for memory constraints, the **Quantization Sensitivity** check (FP16 vs. INT8 on 100 samples) is mandatory to ensure the metric is not an artifact of quantization noise.
   - **Standard Error Reporting:** Retained the requirement to report the standard error of ΔQ.
   - **AAI Baseline Calibration:** Retained BERT-base (High AAI) and AR models (Zero AAI) as calibration points.
4. **Feasibility & Implementation Safety:**
   - **Resource Prioritization:** Explicitly noted that **Generation is the bottleneck** and should be prioritized immediately in Week 4 to avoid GPU contention delays in the final analysis weeks.
   - **Sandbox Environment:** Retained the specific Docker environment for HumanEval.
   - **Clean Text Definition:** Retained the use of ground-truth tokens for the primary AAI calculation, with the noisy check serving as a validity complement.

Method	Clarity	Validity	Rigorousness	Innovativeness	Generalizability
rating	5	5	5	4	5

Experiment	Clarity	Validity	Robustness	Feasibility	Reproducibility
rating	5	5	5	5	5

Weakest point per the reviewers (Innovativeness, 4/5). 1. **Theoretical Strength:** The distinction between "semantic preservation" and "structural change" is well-reasoned and addresses a known confound in the analysis of adapted models.

2. **Methodological Rigor:** The inclusion of the K multiplier in FLOPs calculations and the use of a held-out corpus for AAI computation demonstrate a high level of technical precision.

3. **Potential Refinement:** While the AAI uses cosine distance to measure "Future Sensitivity," it would be beneficial to analyze whether this sensitivity is uniform across layers. It is possible that lower layers preserve AR structure while higher layers develop bidirectional logic; a layer-wise AAI could provide even deeper insight.
4. **Baseline Validation:** The plan to correlate AAI with global CKA is a strong validation step to ensure the new metric provides complementary, rather than redundant, information.

Idea 1.3 — P1-M4 (overall 4.7/5)

Method.

Spectral-Alignment Reranking Efficiency (SARE): A Mechanistic Link Between AR-Representation Geometry and Inference-Time Selection**

1. Core Concept: From "Distance" to "Spectral Overlap" Previous methods (CKA, PASD, LGS) treat representation preservation as a scalar distance or a static geometric property. However, the efficacy of a lightweight AR scorer in reranking DLM candidates depends not on *how close* the DLM is to the AR model globally, but on **how well the DLM's stochastic variance aligns with the AR scorer's sensitivity to quality**.

We propose **Spectral-Alignment Reranking Efficiency (SARE)**. The core hypothesis is that the "accuracy headroom" recoverable via reranking is determined by the **spectral overlap** between:

1. The **Covariance Structure of the DLM's Candidate Variance** (how the DLM explores the latent space).
2. The **Sensitivity Spectrum of the AR Scorer** (which directions in the latent space correspond to high-likelihood/high-quality continuations).

If the DLM's stochasticity lies primarily in directions where the AR scorer is "blind" (low sensitivity), reranking fails. If the DLM's stochasticity lies in directions where the AR scorer is "sharp" (high sensitivity), reranking is highly efficient. SARE quantifies this alignment directly, bypassing the need for global distance metrics or unstable covariance inversions.

2. Metric Definition: Spectral Alignment Score (SAS)

Let M_{DLM} be the diffusion model and M_{AR} be the lightweight AR scorer (e.g., GPT-2-small or LLaMA-2-350M).

Step A: Extract the "Quality Sensitivity Spectrum" of the AR Scorer We characterize how the AR scorer's log-likelihood changes with respect to perturbations in the hidden state space.

1. **Reference Corpus:** $\mathcal{C}_{ref}$ (1,000 text segments, $L = 128$, from WikiText-103 validation set, ensuring domain neutrality).
2. **Perturbation Analysis:** For each segment $x \in \mathcal{C}_{ref}$, we perform a single forward pass of M_{AR} to obtain the final-layer hidden states $H_{AR} \in \mathbb{R}^{L \times d}$.
3. **Gradient-Based Sensitivity:** We compute the gradient of the negative log-likelihood (NLL) of the sequence with respect to the hidden states. Specifically, we approximate the sensitivity vector $\mathbf{g}_t \in \mathbb{R}^d$ at each position t as:

$$\mathbf{g}_t = \frac{\partial \text{NLL}(x)}{\partial \mathbf{h}_t}$$

   Note: This requires a backward pass on the AR scorer, which is feasible for small models (124M-350M) and computationally negligible compared to DLM inference.
4. **Sensitivity Covariance:** We compute the covariance matrix of these sensitivity vectors across the corpus:

$$\Sigma_{sens} = \frac{1}{|\mathcal{C}_{ref}|L} \sum_{x,t} (\mathbf{g}_t - \bar{\mathbf{g}})(\mathbf{g}_t - \bar{\mathbf{g}})^T$$

   where $\bar{\mathbf{g}}$ is the mean sensitivity vector.

5. **Eigen-Decomposition:** We compute the top $K = 50$ eigenvectors $\{\mathbf{u}_1, \dots, \mathbf{u}_{50}\}$ of Σ_{sens}. These eigenvectors represent the directions in latent space where the AR scorer is most "sensitive" to changes in representation (i.e., directions that strongly affect likelihood/quality). Let $\mathbf{U}_{sens} \in \mathbb{R}^{d \times 50}$ be the matrix of these eigenvectors.

Step B: Extract the "Stochastic Exploration Spectrum" of the DLM We characterize how the DLM varies its outputs for a given prompt.

1. **Candidate Generation:** For each prompt x_p in the evaluation set, generate $N = 16$ candidates $\{y_1, \dots, y_{16}\}$ at a low step budget $K = 16$.
2. **Hidden State Extraction:** For each candidate y_i, extract the final-layer hidden states $H_i \in \mathbb{R}^{L \times d}$ during the final denoising step (or via a separate forward pass on the generated text).
3. **Position-Wise Variance:** Instead of averaging over the sequence (which washes out position-specific structure), we compute the variance of the hidden states *across candidates* at each position t:
$$\mathbf{v}_t = \frac{1}{N-1} \sum_{i=1}^{N} (\mathbf{h}_{i,t} - \bar{\mathbf{h}}_t)(\mathbf{h}_{i,t} - \bar{\mathbf{h}}_t)^T$$
where $\bar{\mathbf{h}}_t = \frac{1}{N} \sum_{i=1}^{N} \mathbf{h}_{i,t}$.
4. **Aggregate Exploration Covariance:** We average the position-wise variance matrices across all prompts and positions to get the global exploration covariance:
$$\Sigma_{exp} = \frac{1}{PL} \sum_{p=1}^{P} \sum_{t=1}^{L} \mathbf{v}_{t,p}$$
Note: This captures the "shape" of the DLM's stochastic exploration without requiring alignment to a global AR mean.
5. **Eigen-Decomposition:** We compute the top $K = 50$ eigenvectors $\{\mathbf{w}_1, \dots, \mathbf{w}_{50}\}$ of Σ_{exp}. Let $\mathbf{U}_{exp} \in \mathbb{R}^{d \times 50}$ be the matrix of these eigenvectors.

Step C: Compute Spectral Alignment Score (SAS) We measure the alignment between the DLM's exploration directions and the AR scorer's sensitivity directions.

$$\text{SAS}(M_{DLM}, M_{AR}) = \frac{1}{K^2} \sum_{i=1}^{K} \sum_{j=1}^{K} \left(\mathbf{w}_i^T \mathbf{u}_j\right)^2 \cdot \lambda_{exp,i} \cdot \lambda_{sens,j}$$

Where:

- $\lambda_{exp,i}$ is the i-th eigenvalue of Σ_{exp} (variance explained by DLM exploration direction $\mathbf{w}_i$).
- $\lambda_{sens,j}$ is the j-th eigenvalue of Σ_{sens} (sensitivity explained by AR direction $\mathbf{u}_j$).
- $(\mathbf{w}_i^T \mathbf{u}_j)^2$ measures the cosine similarity squared between the DLM exploration direction and the AR sensitivity direction.

Interpretation:

- **High SAS:** The DLM's variance is concentrated in directions where the AR scorer is highly sensitive. This implies that the AR scorer can effectively distinguish "good" from "bad" candidates based on their latent representation, making reranking efficient.
- **Low SAS:** The DLM's variance is concentrated in directions where the AR scorer is insensitive (or orthogonal). The AR scorer cannot effectively rank candidates, implying that additional denoising steps (which reduce variance globally) may be more effective than reranking.

3. Hypothesis and Efficiency Link

- **Hypothesis 1 (Spectral Alignment Predicts Reranking Gain):** Models with **high SAS** will exhibit **higher Reranking Gains** (ΔQ_{rerank}) relative to Step Gains (ΔQ_{steps}).
- **Hypothesis 2 (SAS is Superior to CKA/PASD):** SAS will explain more variance in the Reranking Advantage ($A_{adv} = \Delta Q_{rerank} - \Delta Q_{steps}$) than static metrics like CKA or dynamic metrics like PASD, because SAS directly models the *interaction* between the generator's variance and the scorer's sensitivity.

4. Experimental Protocol

- **Models:** DiffuGPT-S/M, DiffuLLaMA, Dream-7B, LLaDA-8B.
- **AR Scorers:** GPT-2-small (124M) for GPT-2 family; LLaMA-2-350M (350M) for LLaMA family.
- **Benchmarks:** HumanEval, GSM8K, WinoGrande.
- **Step Budgets:** $K = 16$ (for SAS and Reranking), $K = 128$ (for Step Gain).
- **Candidates:** $N = 16$ per prompt at $K = 16$ (increased from 10 to improve eigenvalue stability).
- **Compute Accounting:**
  - **SAS Computation:** Requires backward passes on the AR scorer (negligible) and forward passes on the DLM (same cost as generation).
  - **Generation:** 16 candidates × 16 steps × 200 prompts × 3 tasks × 4 models.
  - **Scoring:** 2 scorers × 16 candidates × 200 prompts × 3 tasks × 4 models.

5. Procedure

1. **Setup & Sensitivity Spectrum (Weeks 1-2):**
   - Load AR scorers.
   - Compute Σ_{sens} and $\mathbf{U}_{sens}$ for GPT-2-small and LLaMA-2-350M on $\mathcal{C}_{ref}$.
   - *Validation:* Visualize the top eigenvectors of Σ_{sens} to ensure they correspond to meaningful linguistic directions (e.g., via probing on simple tasks).
2. **SAS Computation (Weeks 3-4):**
   - Generate $N = 16$ candidates at $K = 16$ for 200 prompts per task.
   - Extract hidden states.
   - Compute Σ_{exp} and $\mathbf{U}_{exp}$ for each DLM.
   - Compute SAS for each DLM-Scorer pair.
3. **Quality & Efficiency Measurement (Weeks 5-7):**
   - **Reranking Gain:** Score the 16 candidates with the AR scorer.
     - $\Delta Q_{rerank} = Q^{scorer}_{best-of-16} - Q^{scorer}_{top-1}$.
   - **Step Gain:** Generate a single candidate at $K = 128$.
     - $\Delta Q_{steps} = Q^{128}_{top-1} - Q^{16}_{top-1}$.
   - **Reranking Advantage:** $A_{adv} = \Delta Q_{rerank} - \Delta Q_{steps}$.
4. **Correlation Analysis (Weeks 8-9):**
   - Compute Pearson correlation between **SAS** and A_{adv}.
   - **Baseline Comparison:** Compute CKA (static) and PASD (dynamic distance) for the same models.
   - **Regression:**

$$A_{adv}(M) = \beta_0 + \beta_1 \text{SAS}(M) + \epsilon_1$$
$$A_{adv}(M) = \alpha_0 + \alpha_1 \text{CKA}(M) + \epsilon_2$$
$$A_{adv}(M) = \gamma_0 + \gamma_1 \text{PASD}(M) + \epsilon_3$$

   - Test if $|\beta_1|$ is significantly larger than $|\alpha_1|$ and $|\gamma_1|$, and if SAS explains more variance (R^2) in the Reranking Advantage.
   - *Expectation:* β_1 should be **positive** (Higher SAS → Higher Advantage).

6. Why This is Different and Better

- **Addresses Reviewer Concerns:**
  - **No Global Averaging:** SAS uses position-wise variance matrices, preserving token-specific structure. This directly addresses the critique that averaging hides critical errors.
  - **No Unstable Inversion:** SAS uses eigenvalue decomposition of covariance matrices, which is numerically stable and computationally feasible for $d = 4096$.
  - **Mechanistic Link:** SAS directly models the *interaction* between the generator's variance and the scorer's sensitivity, rather than just measuring distance. This provides a deeper mechanistic insight into *why* reranking works.
- **Innovative Metric:** The "Spectral Alignment Score" is a novel metric that bridges representation geometry and inference-time selection efficacy. It is generalizable to any DLM-AR pair and any lightweight scorer.

- **Rigorous Validation:** The method includes a sensitivity analysis of the AR scorer, ensuring that the metric is grounded in the actual behavior of the reranker.

7. Execution Plan (10 Weeks)

- **Weeks 1-2:** Setup. Implement sensitivity spectrum extraction for AR scorers. Validate on small models.
- **Weeks 3-4:** SAS Computation. Run $K = 16$ generations, extract hidden states, compute SAS.
- **Weeks 5-7:** Quality Experiments. Run $K = 16$ (reranking) and $K = 128$ (step gain) generations. Score with AR scorers.
- **Weeks 8-9:** Analysis. Correlate SAS with A_{adv}. Compare with CKA and PASD baselines. Stratify by architecture.
- **Week 10:** Writing.

Why this method. The research problem asks whether AR representation preservation determines the efficiency of reranking vs. step-increase. Previous methods measured preservation as a static property (CKA) or a dynamic distance property (PASD). While PASD fixed the circularity issue, it did not directly measure the *interaction* between the DLM's variance and the AR scorer's sensitivity.

SARE resolves this by:

1. **Measuring Interaction, Not Just Distance:** It uses spectral analysis to quantify the alignment between the DLM's stochastic exploration and the AR scorer's quality sensitivity. This directly links to the ability of a lightweight scorer to rank candidates.
2. **Numerical Robustness:** By using eigenvalue decomposition of covariance matrices, the method is computationally stable and efficient.
3. **Mechanistic Insight:** It tests the hypothesis that reranking is efficient when the DLM's stochasticity is "aligned" with the AR scorer's sensitivity, rather than just "small" (close to the mean). This provides a deeper understanding of why some DLMs benefit from reranking: not just because they are "AR-like," but because their "AR-likeness" aligns their variance with the scorer's decision boundary.

Experiment design.

1. Experimental Design & Protocol**

A. Models, Data & Dimensionality Alignment

- **Diffusion Language Models (DLMs):**
  - *GPT-2 Family:* DiffuGPT-S (127M, $d = 768$), DiffuGPT-M (355M, $d = 1024$).
  - *LLaMA Family:* DiffuLLaMA (6.74B, $d = 4096$), Dream-7B (7B, $d = 4096$), LLaDA-8B (8B, $d = 4096$).
  - *Note:* All 7B+ models will be loaded in 8-bit quantization (`bitsandbytes`, `load_in_8bit=True`) for inference to fit within the 96GB VRAM constraint.
- **AR Scorers (Lightweight Rerankers):**
  - *For GPT-2 Family:* GPT-2-small (124M parameters, hidden dim $d_{AR} = 768$).
  - *For LLaMA Family:* LLaMA-2-350M (350M parameters, hidden dim $d_{AR} = 1024$).
  - *Rationale:* Matching architectural families isolates the effect of diffusion adaptation. The scale gap is intentional to test the "lightweight" constraint, but we include a sensitivity check with a larger scorer (LLaMA-2-7B) on a subset of prompts to verify if SAS correlates with scorer capacity.
- **Dimensionality Alignment Strategy (Critical Fix with Baseline):**
  - The SAS metric requires eigenvectors $\mathbf{w}_i$ (DLM) and $\mathbf{u}_j$ (AR) to be in the same space. Since DLMs and Scorers have different hidden dimensions, we cannot directly compute $\mathbf{w}^T\mathbf{u}$.
  - *Solution:* We project both sets of hidden states into the **AR Scorer's native space** using a **Frozen Linear Projection Probe**.
    1. **Calibration Set:** 500 text segments ($L = 128$) from the WikiText-103 *train* split, disjoint from the validation set used for sensitivity and evaluation.
    2. **Probe Training:** For each DLM-Scorer pair, train a linear layer $W_{proj} \in \mathbb{R}^{d_{AR} \times d_{DLM}}$ to predict the AR scorer's final-layer hidden states from the DLM's

final-layer hidden states on the calibration set.
    * *Loss:* Mean Squared Error (MSE) between $W_{proj} h_{DLM}$ and h_{AR}.
    * *Optimizer:* AdamW, learning rate $1e-3$, 50 epochs.
    * *Target:* The hidden state of the AR model for the **same** text segment at the **final layer** only.
    * *Constraint:* This is a representation alignment probe, not DLM training. The DLM weights remain frozen.
3. **Probe Validation & Bias Control:**
    * Report the R^2 of this linear probe on a **held-out test set** (100 segments from WikiText-103 train, disjoint from calibration) for each model pair.
    * **Random Projection Baseline:** To ensure the linear probe captures meaningful semantic alignment rather than just distribution matching, we also compute SAS using a **random orthogonal projection** $W_{rand} \in \mathbb{R}^{d_{AR} \times d_{DLM}}$ (fixed seed). If the SAS from the learned probe is not significantly higher than the SAS from the random projection, the result is flagged as artifact-driven by projection failure.
    * *Threshold:* If $R^2_{heldout} < 0.5$ or $SAS_{learned} \approx SAS_{random}$, flag the SAS result for that pair as unreliable.
4. **Projection:** Compute $\Sigma^{AR}_{exp} = W_{proj} \Sigma^{DLM}_{exp} W^T_{proj}$. Now $\mathbf{w}_i$ and $\mathbf{u}_j$ are both in $\mathbb{R}^{d_{AR}}$.

- **Benchmarks:**
  - *Coding:* HumanEval (164 problems).
  - *Math:* GSM8K (1319 problems, subset 200 for generation).
  - *Commonsense:* WinoGrande (4400 problems, subset 200 for generation).
  - *Reference Corpus for Sensitivity/Calibration:* WikiText-103 Validation (1,000 segments, $L = 128$).

B. Phase 1: Spectral Alignment Score (SAS) Computation (Weeks 1-2)

1. **Sensitivity Spectrum Extraction (Σ_{sens}):**
    - Load AR Scorers (GPT-2-small, LLaMA-2-350M).
    - For each segment in $\mathcal{C}_{ref}$, perform a forward pass to get final-layer hidden states H_{AR}.
      - *Layer Specification:* "Final-layer hidden states" are defined as the output of the **last Transformer block**, **after** the final LayerNorm but **before** the LM Head (linear projection to vocabulary). This ensures the representation is fully processed by the backbone.
    - Compute the gradient of the sequence NLL with respect to these hidden states $\mathbf{h}_t$.
    - **Multi-Layer Verification:** As a diagnostic, compute Σ_{sens} using the average of the last 3 layers' hidden states. If the top-50 eigenvectors of the final-layer-only spectrum are highly correlated (cosine similarity > 0.9) with the multi-layer spectrum, we proceed with final-layer-only for efficiency. If not, we average the last 3 layers for the primary analysis.
    - **Baseline Sensitivity:** Compute a baseline covariance matrix Σ_{random} using gradients with respect to **random perturbations** of the hidden states (zero-mean Gaussian noise, σ matched to the standard deviation of the true sensitivity gradients). This allows us to verify that the eigenvectors of Σ_{sens} capture specific "quality" directions rather than general manifold variance.
    - **Shrinkage Estimation:** Apply the **Ledoit-Wolf shrinkage estimator** to Σ_{sens} to stabilize the covariance matrix.
    - **Eigen-Decomposition:** Perform eigen-decomposition to obtain top $K = 50$ eigenvectors $\mathbf{U}_{sens}$ and eigenvalues λ_{sens}.
    - *Validation:* Check the eigenvalue decay of Σ_{sens}. If the top 50 eigenvalues do not account for a significant portion of the total variance compared to Σ_{random}, increase K or reconsider the sensitivity definition.
2. **Exploration Spectrum Extraction (Σ_{exp}) with Dimensionality Fix:**
    - For each DLM, select 200 prompts from the benchmark sets.
    - Generate $N = 32$ candidates per prompt at $K = 16$ denoising steps. *Increased N to 32 to improve covariance rank stability.*
    - **Hidden State Extraction:** For each candidate, perform a **separate forward pass** through the DLM on the **clean generated text** (not the noisy final step). This ensures

the hidden states represent the model's "clean" understanding of the output, consistent with how the AR scorer will evaluate the text.
    - *Layer Specification:* Extract hidden states from the **last Transformer block, after final LayerNorm, before LM Head**.
    - *Quantization Consistency:* The DLM must be loaded with the **same 8-bit quantization** settings used during the forward pass for hidden state extraction to ensure consistency with the generation process.
  - Compute position-wise variance matrices $\mathbf{v}_t$ across the 32 candidates for each prompt/position.
  - Average across prompts and positions to get $\Sigma_{exp}^{DLM} \in \mathbb{R}^{d_{DLM} \times d_{DLM}}$.
  - **Shrinkage Estimation:** Apply the **Ledoit-Wolf shrinkage estimator** to Σ_{exp}^{DLM} before projection, as $N = 32$ is much smaller than d_{DLM}, making the raw covariance singular and noisy.
  - **Projection:** Project Σ_{exp}^{DLM} into AR space using the frozen linear probe W_{proj}: $\Sigma_{exp}^{AR} = W_{proj} \Sigma_{exp}^{DLM} W_{proj}^T$.
  - **Eigen-Decomposition:** Perform eigen-decomposition of Σ_{exp}^{AR} to obtain top $K = 50$ eigenvectors $\mathbf{U}_{exp}$ and eigenvalues λ_{exp}.
  - *Numerical Stability:* Zero out any eigenvalues $\lambda_{exp,i}$ that are less than $1e-8$ to prevent division by zero or instability in the SAS calculation.
3. **SAS Calculation:**
  - Compute $\text{SAS}(M_{DLM}, M_{AR})$ using the formula:

$$\text{SAS} = \frac{1}{K^2} \sum_{i=1}^{K} \sum_{j=1}^{K} (\mathbf{w}_i^T \mathbf{u}_j)^2 \lambda_{exp,i} \lambda_{sens,j}$$

  - Compute baseline metrics for comparison:
    - **CKA:** Centered Kernel Alignment between DLM and AR base model hidden states on the reference corpus.
    - **PASD:** Proximal Alignment Score (dynamic distance) computed on the same 200 prompts.

C. Phase 2: Quality & Efficiency Measurement (Weeks 3-5)

1. **Standardized Baseline Definition (Q_{top-1}^{16}):**
  - To ensure robustness and reproducibility, Q_{top-1}^{16} is defined as the **average accuracy of the first 4 candidates** generated with a fixed seed (seed=42) at $K = 16$ steps. This reduces variance from a single sample while remaining a "no-reranking" baseline.
  - *Rationale:* Using the mean of 4 candidates is more stable than a single "first" candidate and does not require internal probability access, which varies across DLM implementations.
  - *Comparison:* We will also report the single-candidate Top-1 accuracy to ensure the averaging does not artificially deflate ΔQ_{rerank} by raising the baseline too high. In the final results, we will clearly distinguish between the "Stable Baseline" (avg of 4) and the "Single-Sample Baseline".
2. **Reranking Gain Measurement (ΔQ_{rerank}):**
  - Using the 32 candidates generated in Phase 1 (at $K = 16$ steps):
  - Score each candidate using the corresponding AR Scorer (sequence log-likelihood).
  - Select the best candidate based on scorer score.
  - Evaluate the best candidate on the specific benchmark task.
  - Calculate $Q_{scorer_selected}$.
  - $\Delta Q_{rerank} = Q_{scorer_selected} - Q_{top-1}^{16}$.
  - *Note:* The reranking gain is compared against the stable baseline (avg of 4) to maintain consistency with the definition of Q_{top-1}^{16}.
3. **Step Gain Measurement (ΔQ_{steps}):**
  - Generate a single candidate per prompt at $K = 128$ denoising steps (seed=42).

- Evaluate this candidate on the benchmark.
- Calculate Q_{top-1}^{128}.
- $\Delta Q_{steps} = Q_{top-1}^{128} - Q_{top-1}^{16}$.

4. **Unified Quality Metric:**
   - **Normalization Formula:** For each task (HumanEval, GSM8K, WinoGrande), let $S_{m,c}$ be the score of model m under condition c. The normalized score is:

$$\hat{S}_{m,c} = \frac{S_{m,c} - \min_{m',c'} S_{m',c'}}{\max_{m',c'} S_{m',c'} - \min_{m',c'} S_{m',c'}}$$

     where the min and max are computed across **all** models and **all** conditions (including $K = 16$ and $K = 128$) for that specific task.
   - Compute average normalized score $\bar{Q}$ for each model/condition as the mean of the normalized scores across the three tasks.
   - Compute $A_{adv} = \Delta\bar{Q}_{rerank} - \Delta\bar{Q}_{steps}$.

D. Phase 3: Correlation & Regression Analysis (Weeks 6-7)

1. **Increase Sample Size for Regression:**
   - To address the small $N = 5$ issue, we add **two more data points** by evaluating **DiffuCoder-7B** (using GPT-2 scorer as a cross-architecture control) and **LLaDA-8B-Base** (if distinct from Instruct, or use a different scorer variant).
   - Alternatively, we perform **stratified regression** within the LLaMA family (3 models) and GPT-2 family (2 models) separately, acknowledging the small N but focusing on the *direction* and *consistency* of the effect rather than global R^2 superiority.
   - *Primary Analysis:* We report Pearson correlations and linear regression coefficients for the full set of 5-7 models. We use **bootstrapping** (1000 resamples) to estimate confidence intervals for β_1, α_1, and γ_1 to handle small N.
2. **Correlation Analysis:**
   - Compute Pearson correlation between SAS and A_{adv}.
   - Compute Pearson correlation between CKA and A_{adv}.
   - Compute Pearson correlation between PASD and A_{adv}.
3. **Regression Modeling:**
   - Fit linear regression models:
     - $A_{adv} = \beta_0 + \beta_1 \text{SAS} + \epsilon_1$
     - $A_{adv} = \alpha_0 + \alpha_1 \text{CKA} + \epsilon_2$
     - $A_{adv} = \gamma_0 + \gamma_1 \text{PASD} + \epsilon_3$
   - Compare R^2 and coefficient significance using bootstrap confidence intervals.
   - Test Hypothesis 1: Is $\beta_1 > 0$ and significant?
   - Test Hypothesis 2: Is $R^2_{SAS} > R^2_{CKA}$ and $R^2_{SAS} > R^2_{PASD}$?
4. **Stratified Analysis:**
   - Repeat correlation analysis separately for GPT-2 family and LLaMA family.
   - **Scorer Capacity Check:** For the LLaMA family, rerun the reranking gain calculation using LLaMA-2-7B as the scorer on a subset of 50 prompts. Check if the correlation between SAS (computed with LLaMA-2-350M) and the gain from the 7B scorer holds. This tests if SAS is a property of the DLM's variance structure or an artifact of the small scorer's limitations.

E. Feasibility & Compute Budget

- **SAS Computation:**
  - AR Scorer Backward Passes: 1,000 segments × 128 positions × 2 scorers. Negligible compute (<1 hour).
  - DLM Forward Passes for Hidden States: 200 prompts × 32 candidates × 5 models. This is equivalent to 32,000 forward passes. With batch size 8, this is 4,000 batches. On RTX 6000 Pro, this is feasible in ~8-12 hours for 7B models (8-bit).
  - Projection Probe Training: Negligible (CPU/GPU, <10 mins).
- **Generation:**

  - $K = 16$ steps: 200 prompts × 32 candidates × 5 models.
  - $K = 128$ steps: 200 prompts × 1 candidate × 5 models.
  - Total generations: ~32,200 sequences.
  - With 8-bit quantization and batch processing, this fits within 2 weeks on a single GPU, especially with parallelization across the 3 GPU-accessible team members.
- **Scoring:**
  - AR Scorer Forward Passes: 32,000 candidates × 2 scorers. Negligible compute.
  - LLaMA-2-7B Scoring (Subset): 50 prompts × 32 candidates × 3 models. Negligible compute.

F. Software Environment & Reproducibility

- **PyTorch:** 2.1.0+cu121
- **CUDA:** 12.1
- **Quantization:** bitsandbytes 0.41.1
- **Seeds:** All random seeds (generation, dropout, etc.) fixed to 42.
- **Code:** All scripts for probe training, SAS computation, and evaluation will be released on GitHub.

Why this design.

1. **Addressing Validity and Reproducibility Gaps**
   - **Dimensionality Alignment with Bias Control:** The original design failed to specify how to compare eigenvectors from models with different hidden dimensions. The revised design introduces a **frozen linear probe** to project DLM hidden states into the AR scorer's space. To address the concern that the probe might artificially inflate SAS by projecting variance into prominent AR directions, we introduce a **random projection baseline**. If the learned probe's SAS is not significantly higher than the random projection's SAS, the result is flagged as unreliable. This ensures that the spectral alignment is meaningful and not an artifact of the projection method.
   - **Probe Validation:** The probe is now validated on a **held-out test set** (separate from calibration) to check for overfitting. The R^2 on this held-out set is reported, and a threshold of $R^2 < 0.5$ triggers a flag.
   - **Layer Specification:** The "final-layer hidden states" are explicitly defined as the output of the last Transformer block, **after** the final LayerNorm but **before** the LM Head. This eliminates ambiguity regarding whether the representation includes the language modeling head's projection.
   - **Normalization Details:** The "Unified Quality Metric" normalization formula is explicitly defined as Min-Max scaling across **all** models and conditions for each task. This ensures that the A_{adv} calculation is consistent and reproducible.
2. **Enhancing Robustness**
   - **Sensitivity Spectrum Validation:** The design now includes a **random perturbation baseline** for the sensitivity spectrum. By comparing the eigenvectors of Σ_{sens} (gradient of NLL) against those of Σ_{random} (gradient of random perturbations), we can verify that the sensitivity spectrum captures specific "quality" directions rather than general manifold variance.
   - **Multi-Layer Verification:** We add a diagnostic step to check if sensitivity is concentrated in the final layer or distributed across the last few layers. If the final-layer spectrum is not representative, we average the last 3 layers. This ensures that the "Quality Sensitivity Spectrum" is a robust measure of the AR scorer's behavior.
   - **Scorer Capacity Check:** For the LLaMA family, we add a subset analysis using a larger scorer (LLaMA-2-7B). This helps determine if the SAS-Reranking correlation is a function of the scorer's capacity or a fundamental property of the DLM's variance structure. If the correlation holds with the larger scorer, it strengthens the mechanistic claim.
   - **Quantization Consistency:** The DLM is loaded with the **same 8-bit quantization** settings for both generation and hidden state extraction. This ensures that the representations analyzed in SAS are consistent with the representations used during the actual generation process, avoiding discrepancies due to quantization noise.

3. **Maintaining Feasibility**
   - The added steps (random projection, held-out probe validation, multi-layer diagnostic, LLaMA-2-7B subset scoring) are computationally negligible compared to the DLM generation and forward passes. The total compute budget remains within the single RTX 6000 Pro Blackwell constraint.
   - The design remains strictly **inference-only**, using only released checkpoints and standard forward/backward passes for analysis. No training, adaptation, or continual pre-training is performed on the DLMs.
4. **Expected Outcomes & Interpretation**
   - If β_1 (SAS coefficient) is positive and significant, and R^2_{SAS} is higher than baselines (within bootstrap confidence intervals), it supports the hypothesis that spectral alignment is the key predictor of reranking efficiency.
   - If the SAS from the learned probe is significantly higher than the SAS from the random projection, it confirms that the linear probe is capturing meaningful semantic alignment. If not, the SAS results are discarded as artifacts.
   - If the sensitivity spectrum eigenvectors are distinct from the random perturbation eigenvectors, it confirms that the "Quality Sensitivity Spectrum" is a valid measure of the AR scorer's sensitivity to quality.
   - If the SAS-Reranking correlation holds with the larger LLaMA-2-7B scorer, it suggests that the effect is robust to scorer capacity and is a fundamental property of the DLM's representation geometry.
   - Stratified analysis will reveal if the effect holds across architectures, providing insight into whether the "AR-likeness" required for efficient reranking is architecture-dependent.

Method	Clarity	Validity	Rigorousness	Innovativeness	Generalizability
rating	5	5	4	4	4

Experiment	Clarity	Validity	Robustness	Feasibility	Reproducibility
rating	5	5	5	5	5

Weakest point per the reviewers (Rigorousness, 4/5). 1. **Address Representation Alignment:** Explicitly define how you will handle the dimensionality and manifold mismatch between the DLM's hidden states and the AR scorer's hidden states. If the models are not identical, you must implement a projection layer or a mapping function; otherwise, the spectral alignment score is mathematically undefined.

2. **Clarify "Final-Layer" Extraction:** You mention extracting hidden states during the "final denoising step." Be precise about whether these are the states *before* or *after* the final linear head, as the sensitivity of the NLL is highly dependent on the specific layer's role in the probability distribution.
3. **Baseline Rigor:** While comparing SAS to CKA and PASD is excellent, ensure that the "Step Gain" (ΔQ_{steps}) is measured using the same number of total FLOPs as the "Reranking Gain" to ensure a fair efficiency comparison, as reranking $N = 16$ candidates involves multiple forward passes.
4. **Sample Size Validation:** $N = 16$ candidates per prompt may be insufficient to produce a stable covariance matrix Σ_{exp} for a high-dimensional space ($d = 4096$). Consider using a larger N or a shrinkage estimator (e.g., Ledoit-Wolf) to ensure the eigenvalues are not noise-driven.

Idea 1.4 — P1-M8 (overall 4.7/5)

Method.

1. Core Concept: Conditional Divergence at Semantic Pivots (CDSP)**

Previous methods (CKA, TSAR, DTD, SGIG, RCHA, PRD, PDL) attempt to correlate representation alignment with quality gains, but they often suffer from **aggregation bias** (averaging over trivial tokens) or **computational inefficiency** (requiring multiple forward passes per token for leave-one-out masking). While PDL correctly identified the need to focus on "pivot" positions, its implementation of computing DLM conditional probabilities via iterative masking is computationally prohibitive for large models (e.g., 6.7B parameters) within the 10-week timeline.

We propose **Conditional Divergence at Semantic Pivots (CDSP)**, a metric that measures the agreement between the DLM and the AR scorer *only at the positions where the task outcome is determined*, but computed via a **single, efficient forward pass**. Instead of masking each pivot individually, we leverage the fact that the DLM is a bidirectional model. We compute the DLM's "self-confidence" (log-likelihood) for the *entire* generated sequence in one pass, and the AR scorer's log-likelihood in one pass. We then isolate the divergence specifically at the "semantic pivots" (high-entropy, non-trivial tokens). This directly tests the hypothesis that reranking efficiency depends on the scorer's ability to distinguish valid continuations at the exact points of semantic divergence, while remaining computationally lightweight.

2. Metric Definition: CDSP (Conditional Divergence at Semantic Pivots)

For a DLM M_{DLM} and an AR scorer M_{AR}:

1. **Candidate Generation and Semantic Pivot Identification:**
   - For each prompt x from the evaluation set (HumanEval, GSM8K, WinoGrande), generate $N = 10$ candidates $\{y_1, \dots, y_N\}$ from M_{DLM} at a low step budget $K = 16$.
   - **Identify Semantic Pivots:** For each candidate y_i, we define the set of **Semantic Pivot Positions** $\mathcal{P}(y_i)$ as the token indices t that satisfy two criteria:
     1. **High AR Uncertainty:** The AR scorer's entropy $H_{AR}(t) = -\sum_v p_{AR}(v|y_{<t}) \log p_{AR}(v|y_{<t})$ is in the top α% of the sequence (sensitivity analysis on $\alpha \in \{5\%, 10\%, 20\%\}$).
     2. **Non-Trivial Token:** The token $y_{i,t}$ is not a punctuation mark, whitespace, or a very common stop-word (based on a standard frequency list). This filters out "noise" positions where divergence is uninformative.
2. **Efficient Likelihood Extraction:**
   - **AR Scorer Likelihoods:** For each candidate y_i, compute the AR scorer's log-probability for each token t:
   $$\log p_{AR}(t) = \log P_{M_{AR}}(y_{i,t} \mid x, y_{i,<t})$$
   This is obtained in a single causal forward pass of M_{AR} on y_i.
   - **DLM Likelihoods:** For the same candidate y_i, compute the DLM's log-probability for each token t:
   $$\log p_{DLM}(t) = \log P_{M_{DLM}}(y_{i,t} \mid x, y_{i,\setminus t})$$
   Crucial Implementation Detail: Unlike PDL, we do **not** perform leave-one-out masking for each pivot. Instead, we perform a **single bidirectional forward pass** of M_{DLM} on the complete sequence y_i. In standard discrete diffusion inference, the model predicts the next state (or the clean token) given the current noisy state. To evaluate the likelihood of the *final* generated tokens, we treat the final generated sequence as the "clean" target. We compute the log-probability of the actual token $y_{i,t}$ at each position t from the model's output distribution. *Note on Compatibility:* For DiffuLLaMA, the forward pass at the final step (or a "score" step) predicts the categorical distribution over the vocabulary for each position. We extract the log-probability of the *sampled* token $y_{i,t}$ from this distribution. This is a standard evaluation technique for bidirectional LMs (similar to computing perplexity for BERT-style models) and requires only **one** forward pass per candidate, not one per pivot.
3. **CDSP Computation (Divergence at Pivots Only):**
   - We define the **Local Divergence** at position t as the absolute difference in log-probabilities:
   $$\Delta \log p(t) = |\log p_{DLM}(t) - \log p_{AR}(t)|$$

- **Aggregate CDSP:** We compute the mean divergence *only* over the semantic pivot positions $\mathcal{P}(y_i)$, averaged over all candidates and prompts:

$$\text{CDSP}(M_{DLM}) = \frac{1}{N \cdot P} \sum_{i=1}^{N} \sum_{p=1}^{P} \left(\frac{1}{|\mathcal{P}(y_i)|} \sum_{t \in \mathcal{P}(y_i)} \Delta \log p(t) \right)$$

- *Low CDSP:* High agreement at critical decision points. The DLM's bidirectional context leads to the same "hard decisions" as the AR scorer's causal context.
- *High CDSP:* Low agreement at critical decision points. The DLM may be "confident" in a token that the AR scorer finds "unlikely" (or vice versa) at the exact positions that determine task correctness.

3. Hypothesis: CDSP Predicts Reranking Efficiency via "Critical-Point Alignment"

We hypothesize that **lower CDSP** (higher agreement at semantic pivots) predicts **higher Reranking Advantage** (ΔQ_{adv}).

- **Mechanism:** Reranking works only if the AR scorer's ranking correlates with the true quality of the candidates. Quality in these tasks is determined by the pivot tokens. If the DLM's probability distribution at these pivot tokens aligns with the AR scorer's distribution (low CDSP), the AR scorer's NLL is a valid proxy for the DLM's "correctness" at the critical points. If they diverge (high CDSP), the AR scorer may penalize valid DLM-specific outputs (e.g., leveraging future context to disambiguate a pivot) or fail to penalize invalid ones, making reranking ineffective.
- **Efficiency Link:**
  - **Low CDSP:** The DLM's candidate set is "scorable" at the points that matter. The AR scorer can effectively distinguish the best candidate. ΔQ_{rerank} is high.
  - **High CDSP:** The DLM's candidate set is "unscoreable" at the critical points. The AR scorer is out-of-distribution for the DLM's specific semantic choices. ΔQ_{rerank} is low. In this regime, increasing denoising steps (which may refine the trajectory towards a more consistent state) is the only effective path, making $\Delta Q_{steps} > \Delta Q_{rerank}$.

4. Why This is Substantively Different and Addresses Feedback

- **Efficiency via Single Forward Pass:** Unlike PDL, which required a separate forward pass for *each* pivot position (linear scaling with pivot count), CDSP requires only **one** forward pass per candidate for the DLM and **one** for the AR scorer. This makes it computationally feasible for 7B models within the 10-week timeline, addressing the "Computational Cost" critique directly.
- **Semantic Filtering:** We refine the pivot definition by adding a "Non-Trivial Token" filter (excluding punctuation/stop-words). This addresses the "Innovativeness" feedback that high entropy might occur at formatting tokens, ensuring the metric focuses on *semantic* pivots.
- **Sensitivity Analysis:** We explicitly plan a sensitivity analysis on the entropy threshold α (5%, 10%, 20%) to ensure the correlation with reranking efficiency is robust, addressing the "Validity" and "Generalizability" critiques regarding the arbitrary 10% threshold.
- **Random Pivot Baseline:** To prove that the "pivot" nature of the selected tokens drives the predictive power, we will compute a **Random Pivot Divergence (RPD)** metric, where pivots are randomly sampled from non-trivial tokens. We expect CDSP to be a significantly stronger predictor of ΔQ_{adv} than RPD, validating the "critical-point" hypothesis.

5. Experimental Protocol

- **Models:** DiffuGPT-S/M, DiffuLLaMA, Dream-7B, LLaDA-8B.
- **Scorers:** GPT-2-small (124M) for GPT-2-based; LLaMA-2-350M (350M) for LLaMA-based.
- **Benchmarks:** HumanEval, GSM8K, WinoGrande.
- **Step Budgets:** $K \in \{16, 128\}$.
- **Candidates:** $N = 10$ per prompt at $K = 16$.
- **Pivot Thresholds:** $\alpha \in \{5\%, 10\%, 20\%\}$.

6. Procedure

1. **CDSP Computation (Weeks 1-3):**

- For each DLM, generate $N = 10$ candidates at $K = 16$ for 200 prompts per task.
- For each candidate y_i:
  - Run **one** forward pass of M_{AR} to get $\log p_{AR}(t)$ for all t.
  - Run **one** forward pass of M_{DLM} (bidirectional, on clean y_i) to get $\log p_{DLM}(t)$ for all t.
  - Identify Semantic Pivots $\mathcal{P}(y_i)$ using α and the non-trivial filter.
  - Compute $\Delta \log p(t)$ for $t \in \mathcal{P}(y_i)$.
- Calculate CDSP for each model-scorer pair and each α.
- Calculate RPD (Random Pivot Divergence) for comparison.

2. **Quality & Efficiency Measurement (Weeks 4-6):**
   - Generate $N = 1$ candidate at $K = 128$ for all models.
   - Compute:
     - $Q_{top1}(16)$: Accuracy of the first candidate at $K = 16$.
     - $Q_{rerank}(16)$: Accuracy of the best-scored candidate at $K = 16$ (using M_{AR}).
     - $Q_{top1}(128)$: Accuracy of the single candidate at $K = 128$.
   - Calculate:
     - $\Delta Q_{rerank} = Q_{rerank}(16) - Q_{top1}(16)$.
     - $\Delta Q_{steps} = Q_{top1}(128) - Q_{top1}(16)$.
     - $A_{adv} = \Delta Q_{rerank} - \Delta Q_{steps}$.
3. **Statistical Analysis (Weeks 7-8):**
   - Perform Pearson correlation between **CDSP** and A_{adv} across the 4-5 models.
   - Perform multiple regression:

$$A_{adv}(M) = \beta_0 + \beta_1 \text{CDSP}(M) + \beta_2 \text{CKA}(M) + \epsilon$$

   - Test if β_1 is significantly **negative** (lower CDSP → higher reranking advantage).
   - **Sensitivity Analysis:** Repeat the correlation for $\alpha \in \{5\%, 10\%, 20\%\}$. The metric is robust if the correlation remains significant across these thresholds.
   - **Baseline Comparison:** Compare the R^2 of the regression using CDSP vs. RPD. CDSP should explain significantly more variance.

7. Execution Plan (10 Weeks)

- **Weeks 1-2:** Setup. Implement CDSP extraction (single forward pass logic). Verify DLM forward pass on clean text yields valid token log-probabilities. Prepare datasets and stop-word lists.
- **Weeks 3-4:** CDSP Computation. Generate candidates at $K = 16$. Compute $\log p_{AR}$ and $\log p_{DLM}$. Calculate CDSP and RPD for all α values.
- **Weeks 5-7:** Quality Experiments. Generate candidates at $K = 128$. Compute accuracies and gains.
- **Weeks 8-9:** Analysis. Correlate CDSP with Reranking Advantage. Perform sensitivity analysis and regression. Compare CDSP vs. RPD vs. CKA.
- **Week 10:** Writing.

Why this method. The research problem asks whether AR-representation preservation determines the efficiency of lightweight reranking versus additional denoising steps. Existing proposals (CKA, TSAR, DTD, SGIG, RCHA, PRD, PDL) attempt to answer this by measuring geometric alignment or aggregate token-level statistics. However, PDL, while innovative in focusing on "pivots," proposed a computationally expensive leave-one-out masking strategy that is infeasible for 7B models within the 10-week constraint.

CDSP addresses this by:

1. **Focusing on Critical Points:** It measures agreement only at the semantic pivot positions (high-entropy, non-trivial tokens), avoiding the dilution effect of averaging over trivial tokens. This directly addresses the "Aggregation Bias" critique.
2. **Computationally Efficient:** It uses a **single forward pass** per candidate for both the DLM and the AR scorer to extract log-probabilities. This is orders of magnitude cheaper than PDL's iterative masking, ensuring feasibility within the resource constraints.

3. **Directly Linking to Scorer Effectiveness:** The hypothesis is that if the DLM's predictions at the pivot points are "AR-consistent" (low CDSP), the AR scorer is a valid proxy for quality. If they are "AR-inconsistent" (high CDSP), the scorer is unreliable at the critical points, and denoising steps are needed. This provides a mechanistic explanation for why some DLMs benefit more from reranking than others.
4. **Rigorous Validation:** By including a sensitivity analysis on the entropy threshold and a Random Pivot Baseline, the method rigorously tests whether the "pivot" nature of the tokens is what drives the predictive power, addressing the validity and generalizability concerns raised in the reviews.

This method is innovative because it shifts the focus from "how similar are the models globally?" to "how well does the scorer understand the DLM's specific critical decisions?" It is rigorous because it uses standard probabilistic metrics on the actual outputs. It is valid because it respects the discrete nature of the checkpoints and the inference-only constraint. It is generalizable because it can be applied to any DLM-AR scorer pair, and the pivot definition is task-agnostic and robust to threshold variations.

Experiment design.

Experiment Name:** Comparative Analysis of Reranking Efficiency vs. Denoising Budget via Conditional Divergence at Semantic Pivots (CDSP) with Cross-Tokenizer Alignment and Trajectory-Smoothed Confidence

1. Experimental Setup & Resources

- **Hardware:** 1x NVIDIA RTX 6000 Pro Blackwell (96 GB VRAM).
  - *Memory Management:* Utilize `torch.cuda.empty_cache()` and a dedicated process manager (e.g., Accelerate or a custom Python subprocess orchestrator) to strictly enforce sequential loading of the large DLMs and small AR scorers, preventing OOM errors during context switching.
  - *Quantization:* Use **bitsandbytes NF4 (4-bit)** for large DLMs (DiffuLLaMA, Dream-7B, LLaDA-8B) to maximize VRAM headroom for batch processing and KV cache. Use **FP16** for smaller models (DiffuGPT-S/M) and all AR scorers.
- **Models (Generators):**
  - *GPT-2 Family:* DiffuGPT-S (127M), DiffuGPT-M (355M).
  - *LLaMA Family:* DiffuLLaMA (6.74B), Dream-7B (7B), LLaDA-8B (8B).
  - *Note:* DiffuCoder-7B is excluded from the main statistical analysis due to its specialized coding focus, which would skew the unified commonsense/reasoning metrics, but may be used for a coding-specific subset if time permits.
- **Scorers (AR):**
  - *For GPT-2 DLMs:* GPT-2-small (124M).
  - *For LLaMA DLMs:* LLaMA-2-350M (350M).
  - *Constraint:* Scorers are run in FP16. They are executed sequentially with the generator to manage VRAM.
- **Datasets:**
  - *HumanEval:* 164 prompts (Coding).
  - *GSM8K:* 500 prompts (Math/Reasoning).
  - *WinoGrande:* 500 prompts (Commonsense).
  - *Total Prompts:* ~1164 unique prompts.
- **Step Budgets (K):** $K \in \{16, 128\}$.
- **Candidates (N):** $N = 10$ for $K = 16$; $N = 1$ for $K = 128$.
- **Pivot Thresholds (α):** $\{5\%, 10\%, 20\%$ of sequence length.
- **Smoothing Window (S):** Primary $S = 4$. Sensitivity check on $S \in \{2, 4, 8\}$ in Week 1.
- **Stop-Word List: NLTK English Stop Words** (standard corpus) + custom list of punctuation/whitespace tokens specific to each tokenizer. This list will be provided as a supplementary file for reproducibility.

2. Procedure

Phase 0: Technical Validation & Tokenizer Alignment Framework (Week 1)

- **Objective:** Resolve the critical ambiguity regarding DLM scoring and establish a robust mapping between DLM token indices and AR scorer token indices.
- **Protocol:**
  1. **Tokenizer Mapping Strategy:**
     - Since DLMs and AR scorers may use different tokenizers (e.g., LLaMA vs. GPT-2, or even different BPE splits within the same family if versions differ), direct index-to-index comparison is invalid.
     - **Implementation:** Develop a **Character-Offset Mapping Function**.
       1. Decode the generated candidate y_i into a raw string s.
       2. Tokenize s using the **AR Scorer's** tokenizer to get tokens T_{AR} and their character offsets $[start_{AR}, end_{AR}]$.
       3. Tokenize s using the **DLM's** tokenizer to get tokens T_{DLM} and their character offsets $[start_{DLM}, end_{DLM}]$.
       4. For each AR token t_{AR}, identify the corresponding DLM token(s) T_{DLM}^{map} that cover the same character span.
       5. **Logit Aggregation:** If one AR token maps to multiple DLM tokens (sub-word mismatch), compute the **mean log-probability** of the DLM tokens over that span to represent the DLM's confidence for that AR token. If one DLM token maps to multiple AR tokens, assign the same DLM logit to all corresponding AR positions.
       6. **Edge Case Handling:** If an AR token maps to a null or empty span (e.g., special tokens or whitespace differences), exclude that position from the CDSP calculation to avoid undefined divergence.
  2. **DLM Scoring Verification:**
     - Select 10 representative prompts. Generate candidates.
     - **Test A (Final Step Logits):** Capture logits from the last denoising step. Check entropy.
     - **Test B (Smoothed Confidence):** Capture logits from the last $S = 4$ denoising steps. Compute the **Time-Averaged Log-Probability**: $\log p_{DLM}^{smooth}(t) = \frac{1}{S}\sum_{k=K-S+1}^{K} \log p_{DLM}^{step k}(t)$.
     - **Decision:** Use **Smoothed Confidence** as the primary metric for CDSP, as it mitigates noise in the final step. If smoothing causes significant latency issues, fall back to Final Step Logits.
     - **Smoothing Sensitivity:** Briefly check $S \in \{2, 4, 8\}$ to ensure the pivot signal is not washed out.

Phase 1: Candidate Generation & Likelihood Extraction (Weeks 2-4)

1. **Generation at Low Budget ($K = 16$):**
   - For each DLM M_{DLM} and each prompt x, generate $N = 10$ candidates $\{y_1, \ldots, y_{10}\}$ using $K = 16$ denoising steps.
   - **Logit Capture:** Modify the inference script to **save the logits** from the **last 4 denoising steps** (steps 13-16) for each candidate.
   - Compute $\log p_{DLM}^{smooth}(t)$ for all DLM positions t by averaging the log-probabilities of the sampled token across these 4 steps.
2. **AR Scorer Likelihoods:**
   - For each generated candidate y_i, run a single causal forward pass of the corresponding AR scorer M_{AR} on $[x, y_i]$.
   - Extract $\log p_{AR}(t)$ and entropy $H_{AR}(t)$ for every AR token position t in y_i.
   - **FLOP Counting:** Record the FLOPs for this forward pass, including the **prompt prefill** (processing the input context x) and the candidate generation span.
3. **Semantic Pivot Identification & Alignment:**
   - For each candidate y_i:
     1. Identify **Semantic Pivot Positions** $\mathcal{P}_{AR}(y_i)$ in the **AR Token Space**:
        - Calculate AR entropy $H_{AR}(t)$ for all t.
        - Select tokens where $H_{AR}(t)$ is in the top α% ($\alpha \in \{0.05, 0.10, 0.20\}$).
        - Filter out "trivial" tokens (punctuation, whitespace, stop-words from NLTK list).
     2. **Map to DLM Space:** Using the Character-Offset Mapping Function, determine the

corresponding DLM logit values $\log p_{DLM}^{smooth}(t')$ for each AR pivot t.

4. **CDSP Calculation:**
   - For each model M_{DLM}, calculate the Conditional Divergence at Semantic Pivots:

$$\text{CDSP}(M_{DLM}, \alpha) = \frac{1}{|\text{Prompts}| \cdot N} \sum_{x} \sum_{i=1}^{N} \left(\frac{1}{|\mathcal{P}_{AR}(y_i)|} \sum_{t \in \mathcal{P}_{AR}(y_i)} |\log p_{DLM}^{smooth}(t) - \log p_{AR}(t)| \right)$$

   - Calculate **Random Pivot Divergence (RPD)**: Repeat the calculation above but replace $\mathcal{P}_{AR}(y_i)$ with a random set of AR token positions of the same size, sampled from the non-trivial tokens (excluding the high-entropy filter).

Phase 2: Quality & Efficiency Measurement (Weeks 5-7)

1. **Generation at High Budget ($K = 128$):**
   - For each DLM and each prompt, generate $N = 1$ candidate using $K = 128$ denoising steps.
2. **Quality Metrics Calculation:**
   - $Q_{top1}(16)$**:** Accuracy of the *first* generated candidate (y_1) from the $K = 16$ batch.
   - $Q_{rerank}(16)$**:** Identify the best candidate among the $N = 10$ candidates from $K = 16$ based on the AR scorer's total log-likelihood $\sum_t \log p_{AR}(t)$. Calculate the accuracy of this best-scoring candidate.
   - $Q_{random}(16)$**: New Baseline:** Randomly select one candidate from the $N = 10$ batch. Calculate the average accuracy over 100 random selections per prompt to estimate the "Best-of-N" ceiling without intelligence.
   - $Q_{self_rerank}(16)$**: New Baseline:** Identify the best candidate among the $N = 10$ candidates based on the DLM's own smoothed log-likelihood $\sum_t \log p_{DLM}^{smooth}(t)$. Calculate accuracy. This tests if the external AR scorer adds value over internal confidence.
   - $Q_{top1}(128)$**:** Accuracy of the single candidate from $K = 128$.
   - *Metrics:*
     - HumanEval: pass@1.
     - GSM8K: Exact Match Accuracy.
     - WinoGrande: Accuracy.
   - *Normalization:* Normalize each task's score to [0,1] across all models and conditions to create a unified Quality Score Q.
     - *Floor Effect Mitigation:* If a task has a "floor effect" (many models score 0), report the raw accuracy separately and use a rank-based normalization for the unified metric to avoid artificial inflation of gains.
3. **Efficiency Gains Calculation:**
   - $\Delta Q_{rerank} = Q_{rerank}(16) - Q_{top1}(16)$.
   - $\Delta Q_{steps} = Q_{top1}(128) - Q_{top1}(16)$.
   - $\Delta Q_{random} = Q_{random}(16) - Q_{top1}(16)$.
   - $\Delta Q_{self} = Q_{self_rerank}(16) - Q_{top1}(16)$.
   - **Reranking Advantage:** $A_{adv} = \Delta Q_{rerank} - \Delta Q_{steps}$.
   - **Net Reranking Gain:** $G_{net} = \Delta Q_{rerank} - \Delta Q_{random}$.
   - **External Scorer Utility:** $U_{ext} = \Delta Q_{rerank} - \Delta Q_{self}$.
4. **Computational Accounting:**
   - Record wall-clock time and FLOPs for:
     1. **Reranking Path:** Generating 10 candidates at $K = 16$ + Scoring 10 candidates with the AR scorer (including prefill).
     2. **Denoising Path:** Generating 1 candidate at $K = 128$.
   - Compare the total compute of (1) vs. (2) to validate the "efficiency" claim quantitatively.

Phase 3: Statistical Analysis (Weeks 8-9)

1. **Correlation Analysis:**
   - Compute **Spearman's rank correlation** coefficient ρ between **CDSP** and A_{adv} across the 5 models.
   - Perform the same correlation for **RPD** and A_{adv}.

- Perform the same correlation for **CKA** (Centered Kernel Alignment between DLM and AR base hidden states, computed on a separate 1000-token validation set *stratified by task domain* to avoid domain-shift artifacts) and A_{adv} as a control metric.
- **Bootstrap Confidence Intervals:** Use bootstrap resampling (10,000 iterations) on the model-level data points to estimate 95% CIs for the correlation coefficients.
- **Leave-One-Out Cross-Validation:** Perform a "leave-one-model-out" analysis to check if the correlation is driven by a single outlier model.

2. **Regression Analysis:**
   - Fit a multiple linear regression model:
   $$A_{adv}(M) = \beta_0 + \beta_1 \text{CDSP}(M) + \beta_2 \text{CKA}(M) + \epsilon$$
   - Test the significance of β_1. Hypothesis: $\beta_1 < 0$ (Lower CDSP → Higher Reranking Advantage).
   - Compare the R^2 of the model with CDSP vs. the model with only CKA.
3. **Sensitivity Analysis:**
   - Repeat the correlation analysis for $\alpha \in \{5\%, 10\%, 20\%$.
   - Check if the sign and significance of the correlation remain consistent across thresholds.
4. **Family-Specific Analysis:**
   - Separate the analysis for GPT-2-based DLMs vs. LLaMA-based DLMs to check for architecture-specific effects.
   - Report per-task correlations (HumanEval, GSM8K, WinoGrande) to ensure the signal is not driven by a single task.
5. **Scorer Utility Analysis:**
   - Analyze $U_{ext} = \Delta Q_{rerank} - \Delta Q_{self}$. If $U_{ext} > 0$, the external AR scorer provides a distinct advantage over the DLM's internal confidence, supporting the hypothesis that AR representations are a better proxy for quality in these tasks.

Phase 4: Reporting (Week 10)

- Summarize findings: Does CDSP (based on smoothed final-step logits) predict reranking efficiency better than global alignment (CKA) or random baselines (RPD)?
- Determine if lightweight reranking is a viable substitute for increased denoising steps for specific model families.
- Discuss implications for inference-time compute allocation, backed by the FLOP/latency counts.
- Address the "Net Reranking Gain" (G_{net}) and "External Scorer Utility" (U_{ext}) to confirm that the AR scorer adds value beyond random selection and internal confidence.

Why this design. This experiment is designed to rigorously test the hypothesis that **local alignment at critical decision points (CDSP)** is a better predictor of reranking efficiency than **global representation alignment (CKA)**, while addressing the critical technical feasibility and validity constraints identified in the reviews.

1. **Addressing the Core Problem & Technical Validity:**
   - **Tokenizer Alignment:** The previous design's ambiguity regarding cross-tokenizer comparison is resolved by implementing a **Character-Offset Mapping Function**. This ensures that $\log p_{DLM}$ and $\log p_{AR}$ are compared at semantically equivalent positions, even if the tokenizations differ. This is crucial for the validity of the CDSP metric when using different scorer families. Edge cases (null spans) are explicitly handled to maintain mathematical integrity.
   - **Logit Smoothing:** The "Final Step Logits" are replaced with **Time-Averaged Log-Probabilities** over the last 4 steps. This addresses the robustness concern that the final step might be noisy or degenerate. Smoothing provides a more stable estimate of the DLM's confidence, making the CDSP metric more reliable. A sensitivity check on the smoothing window S ensures this choice is robust.
   - **Efficiency Validation:** The experiment now includes **Computational Accounting** (FLOPs and latency) to quantitatively verify that the reranking step is indeed more efficient than increasing denoising steps. The FLOP count for the AR scorer explicitly includes prompt prefill, ensuring an accurate comparison.

2. **Feasibility & Efficiency:**
    - **Compute:** By using the **Smoothed Final Step Logits** already generated during inference, we eliminate the need for any additional DLM forward passes for CDSP calculation. This makes the experiment extremely feasible on a single 96GB GPU, even for 7B-8B models.
    - **Memory:** Explicit use of `torch.cuda.empty_cache()` and process management ensures seamless transition between large generators and small scorers, preventing OOM errors. The use of NF4 quantization for large models maximizes VRAM headroom.
    - **Timeline:** The 10-week plan is feasible. Week 1 is dedicated to verifying the tokenizer mapping and logit smoothing logic.
3. **Validity & Robustness:**
    - **Random Pivot Baseline (RPD):** By comparing CDSP to RPD, we control for the possibility that the *number* of high-entropy tokens drives the correlation, rather than the *specific location* of semantic pivots.
    - **Random Selection Baseline (Q_{random}):** We added a new baseline, $Q_{random}(16)$, to distinguish the *scorer's* contribution (ΔQ_{rerank}) from the *best-of-N* effect (ΔQ_{random}).
    - **Self-Reranking Baseline (Q_{self_rerank}):** This new baseline tests if the external AR scorer is truly necessary. If ΔQ_{self} is close to ΔQ_{rerank}, the DLM's internal confidence is sufficient, and the "AR representation preservation" hypothesis might be less relevant than "internal confidence calibration." If $\Delta Q_{rerank} > \Delta Q_{self}$, it supports the hypothesis that AR-specific representations are superior for quality assessment.
    - **Sensitivity Analysis:** Testing multiple α values ensures the metric is not an artifact of an arbitrary threshold.
    - **Statistical Rigor:** With only 5 models, we use **Spearman's rank correlation** and **Bootstrap Confidence Intervals** to provide more robust statistical evidence than simple Pearson correlation. Additionally, **Leave-One-Out Cross-Validation** is included to check for outlier-driven results, addressing the small sample size concern.
    - **Normalization:** We address "floor effects" by using rank-based normalization for the unified metric and reporting raw accuracies separately.
4. **Novelty:**
    - This experiment moves beyond previous work by focusing on **inference-time alignment** and its practical impact on efficiency.
    - It provides a mechanistic explanation for why some DLMs benefit from reranking (low CDSP) while others do not (high CDSP).
    - It bridges the gap between representation geometry (CKA) and practical inference trade-offs (Reranking vs. Steps).
5. **Risk Mitigation:**
    - *DLM Scoring Compatibility:* The use of "Smoothed Final Step Logits" is a standard technique in diffusion sampling. The Week 1 verification step ensures that these logits are meaningful.
    - *Small Sample Size:* The use of multiple benchmarks, sensitivity analyses, bootstrap CIs, and leave-one-out validation helps mitigate the limitations of $n = 5$. The directional consistency across metrics and thresholds will be the key evidence.
    - *Tokenizer Compatibility:* The Character-Offset Mapping Function is a robust solution to cross-tokenizer issues, ensuring that the CDSP metric is well-defined regardless of the specific tokenizer pairs used.
    - *Reproducibility:* The specific stop-word list (NLTK) and quantization method (NF4) are explicitly stated to ensure full reproducibility.

Method	Clarity	Validity	Rigorousness	Innovativeness	Generalizability
rating	5	5	4	4	4

Experiment	Clarity	Validity	Robustness	Feasibility	Reproducibility
rating	5	5	5	5	5

Weakest point per the reviewers (Rigorousness, 4/5). 1. **Address the "Clean Input" Distribution Shift:** To enhance rigorousness, the author should clarify or test if the DLM's forward pass on a fully clean sequence is numerically stable and meaningful. A more rigorous approach would be to use a "pseudo-masking" or "soft-masking" strategy (e.g., replacing the target token with a mask token for a single pass) to align the input with the model's training objective, while still maintaining the efficiency of a single pass (by batching the masked positions).

2. **Clarify the "Non-Trivial Token" List:** The method mentions a "standard frequency list" for stop-words. To ensure replication, the specific list (e.g., NLTK stop-words) or the frequency threshold should be explicitly defined.
3. **Scorer-Model Pairing:** The protocol uses GPT-2-small for GPT-2-based DLMs and LLaMA-2-350M for LLaMA-based DLMs. The author should explicitly state if they will test "cross-family" scorers (e.g., LLaMA-scorer on GPT-2-DLM) to determine if CDSP measures *model-specific* alignment or *general linguistic* alignment.

Idea 1.5 — P1-M2 (overall 4.5/5)

Method.

1. **Representation Preservation via Procrustes Alignment and Latent Distortion** We quantify the preservation of AR representations in each DLM (M_{DLM}) relative to its base AR model (M_{AR}) using **Procrustes Analysis**, which explicitly measures the "cost" of the linear transformation required to align the DLM's latent space to the AR base, addressing the review's concern that CCA might mask the native misalignment.

- **Data:** A fixed, held-out corpus of 5,000 diverse text segments (from WikiText-103 and OpenWebText subsets), length-normalized to 128 tokens.
- **Feature Extraction:** For each model, we perform a single forward pass on the clean text (for AR) or the fully unmasked sequence (for DLM, at the final denoised state x_0). We extract hidden states from the final layer. Let $H_{AR} \in \mathbb{R}^{N\times d}$ and $H_{DLM} \in \mathbb{R}^{N\times d}$ be the centered hidden state matrices.
- **Metric:** We solve the Procrustes problem to find the optimal orthogonal matrix Q that minimizes $\|H_{DLM} - H_{AR}Q\|_F^2$. The **Latent Distortion** D_{latent} is defined as the normalized residual:

$$D_{latent} = \frac{\|H_{DLM} - H_{AR}Q^*\|_F}{\|H_{AR}\|_F}$$

where Q^* is the optimal rotation. A low D_{latent} indicates that the DLM's representations are a simple rotation of the AR base, implying high preservation. A high D_{latent} indicates significant "twisting" or non-linear deformation. We also compute the singular values of the optimal linear map to detect scaling distortions.

2. **Candidate Generation and Reranking Protocol** For each DLM checkpoint ($M \in$ {DiffuGPT-S/M, DiffuLLaMA, Dream-7B, LLaDA-8B, DiffuCoder-7B}), we conduct inference on a standardized evaluation set (HumanEval, GSM8K, SIQA, WinoGrande).

- **Step Budgets:** We generate candidates at $K \in \{16, 32, 64, 128\}$ denoising steps.
- **Candidate Set:** For each prompt x, we generate $N = 10$ distinct candidates $\{y_1, \ldots, y_{10}\}$ by varying the random seed at the initial noise stage.
- **Lightweight Reranker:** We use **GPT-2-small (124M)** as a frozen scorer. The score for a candidate y_i given prompt x is the negative log-likelihood (NLL) of the sequence y_i conditioned on x:

$$\text{Score}(y_i) = -\sum_t \log P_{\text{GPT2}}(y_{i,t} \mid x, y_{i,<t})$$

The reranked candidate is $y^* = \arg\min_{y_i} \text{Score}(y_i)$.

- **Compute Accounting:** We strictly track FLOPs. The cost of the reranker is negligible (<1% of DLM generation cost). The "Reranking Gain" is measured at a *fixed* step budget K, comparing Top-1 (first generated) vs. Best-of-10 (by GPT-2 NLL). The "Step Gain" is measured by comparing Top-1 accuracy at $K = 16$ vs. $K = 128$.

3. **Unified Quality Metric and Efficiency Analysis**

- **Normalization:** We compute task-specific scores (Pass@1 for HumanEval, Accuracy for GSM8K/SIQA/WinoGrande). To create a unified metric $Q \in [0, 1]$, we normalize each task's score across all models and conditions:

$$Q_{norm} = \frac{Q_{raw} - Q_{min}}{Q_{max} - Q_{min}}$$

  The final metric is the average of the four normalized task scores.
- **Efficiency Comparison:** We define **Marginal Efficiency** as the change in normalized quality ΔQ per unit of additional compute.
  - $\Delta Q_{steps} = Q(K = 128) - Q(K = 16)$ (using Top-1 selection).
  - $\Delta Q_{rerank} = Q_{reranked}(K) - Q_{top1}(K)$ (at a fixed K, e.g., $K = 16$ or $K = 32$).
  - We compare ΔQ_{rerank} against the ΔQ_{steps} required to achieve an equivalent quality gain. Specifically, we ask: Does adding a GPT-2 reranker at $K = 16$ yield a larger ΔQ than increasing steps from 16 to 64 or 128?

4. **Statistical Correlation Analysis** We test the hypothesis that D_{latent} (Procrustes distortion) predicts the relative benefit of reranking over step-increase.

- For each model M, we compute the **Reranking Advantage**:

$$A_{rerank}(M) = \Delta Q_{rerank}(M) - \Delta Q_{steps}(M, 16 \to 32)$$

- We perform a Pearson correlation analysis between $D_{latent}(M)$ and $A_{rerank}(M)$ across the 5 model families.
- We also stratify by architecture family (GPT-2-based vs. LLaMA-based) to check if the relationship holds within families or if architecture confounds the result.

5. **Experimental Execution Plan (10 Weeks)**

- **Weeks 1-2:** Setup. Implement Procrustes alignment script. Verify DLM checkpoints load correctly. Prepare evaluation datasets and GPT-2-small scorer.
- **Weeks 3-5:** Representation Probing. Run forward passes on the 5,000-sample corpus for all 5 DLMs and their AR bases. Compute Procrustes distortion scores.
- **Weeks 5-8:** Generation & Scoring.
  - *DiffuGPT-S/M:* Generate 10 candidates × 4 step budgets × ~1000 prompts.
  - *DiffuLLaMA/Dream/LLaDA/DiffuCoder:* Use 8-bit quantization and activation checkpointing. Generate on a subset (e.g., 200 prompts per task).
  - Parallelize: 3 GPU users run different models simultaneously.
- **Weeks 9-10:** Analysis & Writing. Compute unified metrics. Perform correlation analysis. Write report.

Why this method.

1. **Procrustes over CCA:** The review highlighted that CCA finds the *best* linear projection, which might mask the fact that the DLM's native space is misaligned. Procrustes Analysis directly measures the residual error after the optimal rotation, providing a more direct metric of "distortion" or "twisting" that is not invariant to the specific orientation of the latent space. This better captures the "cost" of the transformation that an AR scorer might struggle with.
2. **Efficiency Metric:** The method retains the clear comparison between reranking gain and step-increase gain, normalized by compute, which directly addresses the research question of whether reranking is more efficient than additional denoising.
3. **Rigorousness:** By using Procrustes, we avoid the numerical instability of CCA in high dimensions and provide a more interpretable metric of representation preservation. The experimental design remains comprehensive, with clear baselines and compute accounting.

4. **Innovativeness:** The shift from CKA/CCA to Procrustes offers a new angle on representation alignment, focusing on the geometric "cost" of the AR-to-DLM transition rather than just the existence of a linear mapping.
5. **Generalizability:** The method is applicable to any DLM adapted from an AR backbone, and the use of multiple models and architectures ensures the findings are not specific to one implementation.

Experiment design.

1. **Experimental Objective** To determine whether the geometric preservation of autoregressive (AR) representations in diffusion language models (DLMs), quantified by Procrustes Latent Distortion (D_{latent}), predicts the relative efficacy of a lightweight AR-based reranking strategy compared to increasing the denoising step budget. Specifically, we aim to quantify whether reranking is a more compute-efficient method for recovering "accuracy headroom" than additional denoising steps, and to identify architectural conditions (GPT-2 vs. LLaMA base) where this efficiency holds.

2. Hypothesis

- **H1 (Correlation):** There is a significant negative correlation between Procrustes Latent Distortion (D_{latent}) and the Reranking Advantage (A_{rerank}). Models that preserve AR geometry (low D_{latent}) will benefit more from AR-scoring reranking relative to step-increase gains.
- **H2 (Efficiency):** At low step budgets ($K = 16, 32$), the quality gain from GPT-2-small reranking (ΔQ_{rerank}) exceeds the quality gain from increasing denoising steps to $K = 128$ (ΔQ_{steps}) for models with low D_{latent}, but not for models with high D_{latent}.

3. Models and Checkpoints We utilize the following released DLM checkpoints and their corresponding AR base models. All models are evaluated in inference-only mode.

DLM Checkpoint	Parameters	Base AR Model	Architecture Family	Quantization Strategy
DiffuGPT-S	124M	GPT-2 Small	GPT-2	FP16 (Full Precision)
DiffuGPT-M	350M	GPT-2 Medium	GPT-2	FP16 (Full Precision)
DiffuLLaMA	6.74B	LLaMA-2 7B	LLaMA	8-bit (bitsandbytes NF4)
Dream-7B	7B	LLaMA-2 7B	LLaMA	8-bit (bitsandbytes NF4)
LLaDA-8B	8B	LLaMA-3 8B	LLaMA	8-bit (bitsandbytes NF4)
DiffuCoder-7B	7B	CodeLlama-7B	LLaMA	8-bit (bitsandbytes NF4)

Note on Scorers:

1. **Primary Scorer:** A frozen **GPT-2 Small (124M)** is used exclusively for the main reranking analysis. It serves as a lightweight proxy for "AR plausibility."
2. **Secondary Scorer (Robustness Check):** A frozen **GPT-2 Medium (350M)** is used for a subset of LLaMA-based models (specifically DiffuLLaMA and Dream-7B) to verify if the "Reranking Advantage" is a function of representation preservation or simply scorer capacity limitations.

4. Representation Preservation Metric (Procrustes Analysis)

- **Corpus:** 5,000 disjoint text segments from WikiText-103 and OpenWebText (held-out from all training sets). Each segment is tokenized to exactly 128 tokens using the *generator's* tokenizer.
- **Feature Extraction:**
  - **AR Models:** Perform a single causal forward pass on the 128-token sequence. Extract hidden states $H_{AR} \in \mathbb{R}^{N \times d}$ from the **final transformer layer** (before the LM head) and the **middle layer** (e.g., layer $L/2$). Center H_{AR} (subtract column means).
  - **DLMs:**
    * **Primary State ($t = 0$):** Initialize the sequence with the clean 128-token text (no masking). Perform a single forward pass using the DLM's specific attention mask configuration (bidirectional or hybrid) as defined in its inference loop at the final denoised state ($t = 0$). Extract hidden states $H_{DLM} \in \mathbb{R}^{N \times d}$ from the **same final and middle layer indices** as the AR base. Center H_{DLM}.
    * **Sensitivity State ($t = K/2$):** To address the concern that $t = 0$ may not reflect denoising behavior, we perform a secondary probe. For a subset of 500 samples, we initialize with 50% random masking (simulating the state at $K/2$ for $K = 100$), run one denoising step, and extract hidden states H_{DLM}^{mid}. We compute D_{latent}^{mid} to check consistency with $D_{latent}^{t=0}$.
  - *Constraint Check:* Per verified repository context, DLMs operate on discrete token IDs. We use the standard forward pass on the *clean* input IDs to obtain the "native" representation, avoiding the need to simulate the full denoising trajectory for the primary probing.
- **Procrustes Calculation:** Solve for the optimal orthogonal matrix Q^* that minimizes $\|H_{DLM} - H_{AR}Q\|_F^2$ using `scipy.linalg.orthogonal_procrustes`. Calculate **Latent Distortion**:

$$D_{latent} = \frac{\|H_{DLM} - H_{AR}Q^*\|_F}{\|H_{AR}\|_F}$$

  - $D_{latent} \approx 0$: High preservation (DLM space is a rotation of AR space).
  - $D_{latent} \approx 1$: Low preservation (DLM space is orthogonal or highly distorted).
  - *Dimensionality:* Hidden dimensions match for all selected pairs (768 for GPT-2-S, 1024 for GPT-2-M, 4096 for LLaMA-2/3). No projection is required.
- **Layer Aggregation:** The final D_{latent} used for correlation analysis is the **average** of the final-layer and middle-layer distortions: $D_{latent}^{final} = \frac{1}{2}(D_{latent}^{L} + D_{latent}^{L/2})$. We report separate correlations for each layer in the appendix to verify that the trend is not driven by a single depth.
- **Baselines:**
  1. **Zero-Distortion Baseline:** Compute D_{latent} between the AR base and itself (expected ≈ 0).
  2. **Random Baseline:** Shuffle the rows of H_{AR} to establish the upper bound of D_{latent} for noise-level distortion.
  3. **Unrelated Model Baseline:** Compute D_{latent} between two unrelated AR models of similar size (e.g., GPT-2-S vs. GPT-2-M, or LLaMA-2 vs. LLaMA-3) to establish the "natural" distortion between different pretrained spaces.
  - *Normalization for Analysis:* The final D_{latent} used in correlation analysis will be centered relative to the "Unrelated Model Baseline" to account for inherent high-dimensional noise and architectural differences.

5. Candidate Generation and Reranking Protocol

- **Evaluation Tasks & Sample Size:** To ensure statistical comparability across model families, we use a **consistent subset of 800 prompts** for all models:
  1. **HumanEval** (Coding): 200 problems. Metric: Pass@1.
  2. **GSM8K** (Math): 200 subset. Metric: Exact Match Accuracy.
  3. **SIQA** (Social IQ): 200 subset. Metric: Multiple Choice Accuracy.
  4. **WinoGrande** (Commonsense): 200 subset. Metric: Multiple Choice Accuracy. *Total Prompts:* 800 per model.
  - *Prompting Strategy:* We use the standard prompt templates provided in the respective model repositories. To ensure reproducibility, we pin the exact commit hashes of the

`DiffuLLaMA`, `Dream-7B`, `LLaDA`, and `DiffuCoder` repositories used for template extraction (documented in Appendix A).

- **Generation Settings:**
  - **Step Budgets (K):** 16, 32, 64, 128 denoising steps.
  - **Candidate Count (N):** 10 distinct candidates per prompt.
  - **Randomness:** For each prompt, generate $N = 10$ candidates by varying the initial noise seed.
  - **Sampling Hyperparameters:** Temperature $T = 0.8$, Top-p 0.95, Top-k 50 (unless specified otherwise by the model's default inference script). These settings are fixed to ensure candidate diversity is consistent across models.
  - **Top-1 Baseline:** The first generated candidate (y_1) is the "Top-1" reference.
- **Reranking Pipeline (Decode-Retokenize-Scoring):** To handle vocabulary mismatches between LLaMA-based generators and the GPT-2 scorer, we implement a strict **Decode-Retokenize** pipeline:
  1. **Decode:** Decode the generated token IDs of candidate y_i into raw text using the *generator's* tokenizer.
  2. **Retokenize:** Re-tokenize the raw text using the **GPT-2 tokenizer**.
  3. **Score:** Compute the Negative Log-Likelihood (NLL) of the re-tokenized sequence conditioned on the prompt (also re-tokenized by GPT-2) using the frozen GPT-2-small model.

$$\text{Score}(y_i) = -\sum_t \log P_{\text{GPT2}}(y_{i,t}^{gpt2} \mid x^{gpt2}, y_{i,<t}^{gpt2})$$

  4. **Select:** $y^* = \arg\min_i \text{Score}(y_i)$.
  - *Compute Accounting:* GPT-2-small NLL is computed via a single teacher-forced forward pass per candidate. Total FLOPs for reranking are logged and verified to be <1% of the DLM generation FLOPs.
- **Compute Budget & Feasibility:**
  - **DiffuGPT-S/M:** Run in FP16. Generate 10 candidates × 4 steps × 800 prompts.
  - **7B/8B Models:** Use **8-bit quantization (NF4)** and **Activation Checkpointing**. Generate 10 candidates × 4 steps × 800 prompts.
  - **Parallelization:**
    - GPU User 1: DiffuGPT-S & M (Fast, can run full set early).
    - GPU User 2: DiffuLLaMA & Dream-7B.
    - GPU User 3: LLaDA-8B & DiffuCoder-7B.
  - **CPU Tasks:** GPT-2-small scoring and evaluation metrics (AST checking for HumanEval, regex for GSM8K) are offloaded to CPU or run on GPU during DLM idle time.

6. Metrics and Analysis

- **Unified Quality Metric (Q):** For each task, normalize the raw score S_{raw} across **all models and all conditions** (all step budgets, Top-1 vs Reranked) to $[0, 1]$:

$$Q_{task} = \frac{S_{raw} - S_{min}}{S_{max} - S_{min}}$$

  Note: S_{min} and S_{max} are calculated dynamically from the complete dataset of all runs to ensure a consistent baseline. To mitigate outlier sensitivity, we also report the raw scores in the final table. The final quality score for a model/condition is:

$$Q = \frac{1}{4} \sum_{t \in \{HumanEval, GSM8K, SIQA, WinoGrande\}} Q_t$$

- **Efficiency Gains:**
  1. **Step Gain:**

$$\Delta Q_{steps} = Q_{Top1}(K = 128) - Q_{Top1}(K = 16)$$

  2. **Reranking Gain:**

$$\Delta Q_{rerank}(K) = Q_{Reranked}(K) - Q_{Top1}(K)$$

We primarily evaluate $\Delta Q_{rerank}(16)$ and $\Delta Q_{rerank}(32)$.

3. **Reranking Advantage (A_{rerank}):** To address the feedback on baseline robustness, we define the advantage relative to the **minimum step budget required to match the reranked quality**. Let $Q_{target} = Q_{Reranked}(K = 16)$. Find the smallest $K_{match} \in \{16, 32, 64, 128\}$ such that $Q_{Top1}(K_{match}) \geq Q_{target}$. If no such K exists in our budget, set $K_{match} = 128 + \infty$ (indicating reranking is superior to any tested step budget).

$$A_{rerank}(M) = (K_{match} - 16) \times \text{UnitCompute} - \text{RerankCompute}$$

Simplification for Correlation: For the primary correlation analysis with $N = 6$, we use the simpler proxy $A^{simple}_{rerank} = \Delta Q_{rerank}(16) - \Delta Q_{steps}(16 \to 32)$ as defined in the method, but we report the "Step Equivalent" (K_{match}) in the results table to provide the rigorous efficiency comparison.

- **Statistical Analysis:**
  1. **Correlation:** Compute Pearson and Spearman rank correlation coefficients between centered D_{latent} and A^{simple}_{rerank} across the 6 models.
  2. **Stratified Analysis:** Separate correlations for GPT-2-based models ($N = 2$) and LLaMA-based models ($N = 4$).
  3. **Significance:** Use bootstrap confidence intervals (10,000 resamples) for the correlation coefficients and mean quality gains. Given the small $N = 6$, results will be interpreted as indicative trends rather than definitive causal proofs, with explicit reporting of confidence intervals.
  4. **Scorer Utility Check:** Compute the Pearson correlation between GPT-2-small NLL and actual task accuracy for each model family. If this correlation is weak (e.g., $|r| < 0.2$) for LLaMA models, it reinforces the interpretation that the lack of reranking benefit is due to representation misalignment or scorer capacity, not just representation misalignment.

7. Experimental Execution Plan (10 Weeks)

- **Weeks 1-2: Setup & Verification**
  - Setup environment (PyTorch, bitsandbytes, DiffuLLaMA repo).
  - Verify checkpoint loading for all 6 DLMs and 3 AR bases.
  - Implement Procrustes alignment script (CPU-based).
  - Implement the Decode-Retokenize-Scoring pipeline for GPT-2-small and GPT-2-medium.
  - Prepare evaluation datasets (800 prompts total) and verify tokenization consistency.
  - *Milestone:* All models load and generate 1 sample correctly; Scorer pipeline verified on a small subset.
- **Weeks 3-4: Representation Probing**
  - Run forward passes on the 5,000-sample corpus for all AR bases and DLMs (final and middle layers) at $t = 0$.
  - Run sensitivity probing on 500 samples at $t = K/2$ for the 6 DLMs.
  - Compute H_{AR} and H_{DLM} matrices.
  - Calculate Procrustes Q^* and D_{latent} for each DLM-AR pair.
  - Compute random and unrelated model baselines.
  - *Milestone:* D_{latent} scores computed for all 6 models; consistency check between $t = 0$ and $t = K/2$ probes.
- **Weeks 5-8: Generation & Scoring (Parallelized)**
  - **Group 1 (GPU 1):** DiffuGPT-S & DiffuGPT-M (800 prompts, 4 steps, 10 candidates).
  - **Group 2 (GPU 2):** DiffuLLaMA & Dream-7B (800 prompts, 4 steps, 10 candidates, 8-bit).
  - **Group 3 (GPU 3):** LLaDA-8B & DiffuCoder-7B (800 prompts, 4 steps, 10 candidates, 8-bit).
  - *CPU Tasks:* GPT-2-small scoring (asynchronous), evaluation metrics computation.
  - *Secondary Scorer Run:* For DiffuLLaMA and Dream-7B, run GPT-2-medium scoring on the same candidates.
  - *Checkpointing:* Save generation progress every 100 prompts to mitigate GPU contention risks.
  - *Milestone:* All candidates generated and scored.

- **Weeks 9-10: Analysis & Writing**
  - Compute Unified Quality Metrics Q.
  - Calculate ΔQ_{steps}, ΔQ_{rerank}, and A_{rerank}.
  - Perform correlation analysis between D_{latent} and A_{rerank}.
  - Draft report, visualizations (Scatter plot of D_{latent} vs A_{rerank} with CI, Bar charts of Q vs Steps).
  - *Milestone:* Final report with conclusions on efficiency and representation preservation.

8. Potential Risks and Mitigations

- **Risk:** GPU contention or failure.
  - *Mitigation:* Checkpointing of generation progress every 100 prompts. Use of 8-bit quantization reduces memory pressure. If GPU fails, CPU-based scoring can continue, and generation can be resumed on a different machine if available, or batch sizes reduced.
- **Risk:** GPT-2-small is too weak to effectively rerank LLaMA-generated candidates.
  - *Mitigation:* The use of GPT-2-medium as a secondary scorer for LLaMA models helps distinguish between "scorer capacity" and "representation preservation." If GPT-2-medium also fails to rerank effectively, it supports H1 (high $D_{latent} \rightarrow$ low A_{rerank}).
- **Risk:** Procrustes alignment is ill-posed if the "clean" forward pass does not reflect the denoising trajectory.
  - *Mitigation:* We include the $t = K/2$ sensitivity probe. If D_{latent} at $t = 0$ and $t = K/2$ are highly correlated, the $t = 0$ metric is validated as representative. If not, we will report the $t = K/2$ metric as the primary predictor.
- **Risk:** Statistical power with $N = 6$ models.
  - *Mitigation:* Use of Spearman rank correlation and bootstrap CIs. Interpretation will be cautious, focusing on directional trends and architectural stratification rather than p-values alone. We explicitly frame results as "indicative trends" due to the small sample size.

9. Data and Code Availability

- All scripts for Procrustes alignment, generation, and evaluation will be released.
- The 5,000-sample probing corpus will be released as a JSONL file.
- The 800-prompt evaluation subset and prompt templates (with pinned commit hashes) will be released to allow exact reproducibility.
- Results will be tabulated in the final report, including both normalized Q and raw scores.

Why this design.

1. **Alignment with Research Problem** The experiment directly addresses the core question: *Does AR representation preservation determine reranking efficacy?* By quantifying preservation via Procrustes D_{latent} and measuring efficacy via the Reranking Advantage A_{rerank}, we establish a direct link between the geometric property of the model and its inference-time optimization potential. The comparison against ΔQ_{steps} ensures we answer the secondary question: *Is reranking more efficient than adding steps?* The use of a consistent 800-prompt subset across all models ensures that the correlation analysis is not biased by sample size variance, directly addressing the robustness feedback.

2. **Methodological Rigor & Validity**

- **Procrustes over CKA:** Procrustes provides a more interpretable "distortion" metric than CKA, which can be inflated by high-dimensional noise. It explicitly measures the cost of the linear transformation, aligning with the hypothesis that "twisting" in latent space hinders AR-scoring. The addition of random and unrelated model baselines provides a complete range for D_{latent} normalization and sanity checks.
- **Probing State Validity:** Addressing the review's concern, we add a sensitivity analysis probing at $t = K/2$. This verifies that the $t = 0$ "native" representation is a valid proxy for the representations encountered during the critical denoising phase. If the metrics diverge, we switch to the $t = K/2$ metric, ensuring the predictor is faithful to the generation process.
- **Decode-Retokenize Pipeline:** The explicit implementation of the decode-then-retokenize

strategy for the GPT-2 scorer resolves the vocabulary mismatch risk identified in the reviews. This ensures that the NLL is computed on semantically valid text for the scorer, rather than failing due to token ID mismatches. This is crucial for the validity of the "AR scorer" assumption across LLaMA-based models.
- **Robust Efficiency Baseline:** We refine the "Reranking Advantage" by reporting the "Step Equivalent" (K_{match}). This directly answers "how many steps does this reranking save?" which is a more intuitive and robust measure of efficiency than a simple difference in quality scores at fixed steps.

3. **Feasibility and Resource Optimization**

- **Single GPU Constraint:** The plan respects the 96GB single-GPU limit by using 8-bit quantization for 7B+ models and a consistent, manageable prompt subset (800) for all models. This trade-off ensures that all models are evaluated on the same data, facilitating direct comparison, while keeping the total compute within the 10-week timeline.
- **Parallelization:** The 3-person GPU split ensures that the bottleneck (generation) is parallelized across team members. CPU-heavy tasks (scoring, evaluation) are assigned to non-GPU team members or run asynchronously, maximizing throughput.
- **Timeline:** The 10-week schedule is realistic. Weeks 1-4 are setup and probing (low compute). Weeks 5-8 are the heavy generation phase, which is parallelized. Weeks 9-10 are analysis.

4. **Novelty and Impact** This experiment bridges two previously separate areas: representation alignment (REPR-ALIGN) and inference-time efficiency (Jacobi Forcing, TESS 2). By showing that *how much* a DLM preserves AR geometry predicts *how well* it can be reranked by an AR model, we provide a new diagnostic tool for DLM developers. If D_{latent} is high, investing in more denoising steps is better than investing in reranking. This is an actionable insight for the field.

5. **Validity Checks**

- **Control Group:** The GPT-2-based DLMs (DiffuGPT-S/M) serve as a control. They should theoretically have lower D_{latent} (higher preservation) and thus higher A_{rerank} when scored by GPT-2-small. If this trend is observed, it validates the Procrustes metric and the reranking hypothesis.
- **Negative Control:** If D_{latent} shows no correlation with A_{rerank}, it suggests that representation geometry is *not* the primary driver of reranking efficacy, pointing instead to other factors (e.g., token distribution, diversity of candidates). This is also a valuable scientific outcome.
- **Scorer Utility Check:** We explicitly measure the correlation between GPT-2 NLL and task accuracy per model family. If this correlation is weak for LLaMA models, it reinforces the interpretation that the lack of reranking benefit is due to representation misalignment, not just scorer weakness. The addition of GPT-2-medium as a secondary scorer further strengthens this check.

Method	Clarity	Validity	Rigorousness	Innovativeness	Generalizability
rating	4	4	4	4	4

Experiment	Clarity	Validity	Robustness	Feasibility	Reproducibility
rating	5	5	5	5	5

Weakest point per the reviewers (Clarity, 4/5). While the method is overall very clear, two minor points could be further specified to ensure absolute replication:

1. **Tokenizer Handling:** Since the method uses a variety of DLMs (LLaMA-based, GPT-2-based, etc.) and a GPT-2-small reranker, the method should explicitly state that the reranker will use its own tokenizer to process the decoded text of the candidates, rather than assuming a shared vocabulary across all models.

2. **Normalization Baseline:** In the unified quality metric, the Q_{min} and Q_{max} are described as being "across all models and conditions." It would be helpful to clarify if these are the absolute observed min/max in the experiment or predefined theoretical bounds, as this affects the interpretation of the normalized score.

Research problem 2 (P2)

How does the choice of noise schedule (e.g., linear, cosine, context-adaptive, and a simple learned schedule) affect the quality-efficiency frontier of scaled diffusion language models adapted from autoregressive checkpoints, and does this relationship differ across model families (GPT-2-based vs. LLaMA-based) and scales (127 M – 7 B)?

Rationale. The target paper shows that continual pre-training of open-source autoregressive (AR) checkpoints yields competitive diffusion language models (DLMs) such as DiffuGPT and DiffuLLaMA, but it also highlights several limitations that motivate an inference-only investigation of the noise schedule:

1. **Undertrained models (L7).** The adapted DLMs are reported to be undertrained relative to their AR counterparts, leaving a quality gap. A more effective noise schedule can reduce the difficulty of the denoising process at inference time, potentially narrowing this gap without additional training.
2. **Non-compute-normalized efficiency reporting (L9).** The original work measures only wall-clock latency, ignoring the fact that different noise schedules change the amount of computation per denoising step (e.g., by altering the proportion of tokens that remain noisy). By reporting a compute-normalized metric—approximated FLOPs = model size × steps × (effective token count) × sequence length—we obtain a fairer quality-efficiency trade-off curve for each schedule.
3. **Narrow infilling evaluation (L8).** Infilling was assessed only on single-line HumanEval with gold prefix and suffix. We will evaluate a broader suite of infilling tasks (multi-line HumanEval, MBPP, and open-ended text completion) to see whether certain schedules generalize better to realistic masking scenarios.
4. **Architectural generality (L10).** The adaptation recipe was applied only to GPT-2 and LLaMA families up to 7 B. Noise schedules interact with the model's attention pattern (causal vs. bidirectional) and may benefit one family more than the other—for instance, context-adaptive schedules that preserve bidirectional context could align better with LLaMA-based models, while simple linear schedules might suit GPT-2-based models. Testing across families and scales will reveal such interactions.
5. **Connection to alternative routes (L10, L5).** Related work shows that representation-alignment or uniform-noise adaptation can be competitive with continued denoising training. By treating the noise schedule as an inference-only "knob," we explore whether a simple change at sampling time can approach the performance of those alternative routes without retraining.
6. **Statistical rigor (L11).** We will run multiple seeds (e.g., 5) for each configuration and report mean ± standard deviation, addressing the lack of variance estimates in the original paper.
7. **Instruction-following deficiency (L5).** The authors deferred instruction tuning despite evidence that it improves performance. We will include an instruction-following benchmark (IFEval) to determine whether certain schedules better preserve the model's ability to follow prompts.

Experimental design (inference-only, feasible under constraints):

- **Models:** DiffuGPT-S/M (GPT-2-based), DiffuLLaMA-6.74 B, Dream-7B, DiffuCoder-7B, LLaDA-8B (LLaMA-based).

- **Noise schedules:** (i) linear β_t, (ii) cosine β_t (as in improved DDPM), (iii) context-adaptive schedule used in Dream 7B (noise level conditioned on token-wise entropy), (iv) a simple learned schedule (a small MLP predicting β_t from timestep, trained on a held-out slice of the adaptation corpus—this step uses only the released checkpoint's forward pass and does not modify weights).

- **Denoising steps:** 16, 32, 64 (to capture the quality-efficiency curve).

- **Sequence length:** fixed at 128 tokens (typical for infilling and reasoning prompts).

- **Metrics:**
  - *Quality:* perplexity on WikiText-103, exact-match pass@k on HumanInfilling ($k = 1,10$), MAUVE on open-ended generation (Prompts from C4), IFEval instruction-following score.

  - *Efficiency:* approximated FLOPs per generated token (model size × steps × average noisy token fraction × seq length) and wall-clock latency per token (batch size = 1).

- **Analysis:** For each model family and scale, plot quality vs. compute-normalized efficiency for each schedule; compute the area under the curve (AUC) to summarize the frontier; test whether schedule effects interact significantly with family and scale using two-way ANOVA.

Feasibility: All experiments use released checkpoints; no pre-training, continual pre-training, or weight updates are required. A single NVIDIA RTX 6000 Pro Blackwell (96 GB) can batched inference for the proposed configurations within the ten-week timeline (GPU-enabled members parallelize over models and schedules; CPU-only members handle metric computation, statistical analysis, and write-up).

Significance: This work provides the first systematic, inference-only characterization of how noise scheduling shapes the quality-efficiency trade-off of scaled DLMs across families and scales. By offering a training-free lever to mitigate undertraining, improve compute-normalized efficiency, and enhance infilling and instruction-following performance, the results give practitioners a concrete knob for deploying adapted DLMs. Moreover, uncovering family- and schedule-specific interactions can guide future DLM design (e.g., selecting schedules that better match the model's attention bias), thereby extending the adaptation paradigm introduced in the target paper without costly retraining.

ReviewingAgent scores: Clarity 4/5, Relevance 4/5, Originality 3/5, Feasibility 3/5, Significance 3/5.

All 8 ideas, ranked

Rank	Idea	Overall /5	Novelty	Feasibility	Clarity	Impact	Weakest
1	P2-M1	**4.9**	4	5	5.0	5	4
2	P2-M7	**4.9**	4	5	5.0	5	4
3	P2-M8	**4.9**	4	5	5.0	5	4
4	P2-M5	**4.8**	4	5	4.5	5	4
5	P2-M4	**4.6**	4	5	5.0	4	4
6	P2-M2	4.5	4	5	5.0	4	3
7	P2-M3	4.4	4	5	4.5	5	3
8	P2-M6	4.4	4	5	4.0	4	3

Top 5 submitted; the rest are listed so the cut is visible.

Idea 2.1 — P2-M1 (overall 4.9/5)

Method.

Inference-Only Noise Schedule Characterization via Convergence-Efficiency Frontier (CEF) Analysis

We propose a rigorous, inference-only framework to characterize how noise schedules influence the quality-efficiency trade-off of Diffusion Language Models (DLMs). We strictly separate **computational cost** (constant per step in dense implementations) from **sampling convergence** (how effectively the model refines the sequence within a fixed step budget). Our method introduces a **Convergence-Efficiency Frontier (CEF)** analysis and a standardized protocol for evaluating four distinct noise schedules: Linear, Cosine, Context-Adaptive, and Learned (MLP-based).

1. Standardized Noise Schedule Definitions & Implementation To ensure replicability, we define the noise schedule β_t (the probability of a token remaining masked at step t) for $t \in \{1, \dots, T\}$ as follows. *Note: In all schedules, "frozen" tokens remain in the attention context but are excluded from the loss/update mask to ensure bidirectional context is preserved without violating the constant-cost assumption.*

- **Linear Schedule:** $\beta_t = 1 - \frac{t-1}{T-1}$. This is the baseline, providing a uniform reduction in noise.
- **Cosine Schedule:** $\beta_t = \cos\left(\frac{\pi}{2} \cdot \frac{t-1}{T-1}\right)$. This schedule (inspired by Improved DDPM) removes noise slowly at the beginning and end, and rapidly in the middle.
- **Context-Adaptive Schedule (Dream-style):** $\beta_t^{(i)} = \max\left(\beta_t^{base}, \frac{H_t^{(i)}}{H_{max}}\right)$, where $H_t^{(i)}$ is the entropy of the model's predicted distribution for token i at step t, and $H_{max} = \log|V|$. The base schedule β_t^{base} is linear.
  - *Implementation Detail:* We implement this by modifying the `maskable_mask` in the inference loop. A token is "frozen" (excluded from updating) if its predicted entropy $H_t^{(i)}$ falls below a dynamic threshold $\tau_t = \alpha \cdot H_{max}$.
  - *Sensitivity Analysis:* Instead of a fixed $\alpha = 0.1$, we perform a small grid search for $\alpha \in \{0.05, 0.1, 0.2\}$ per model family to determine the optimal threshold, preventing premature convergence errors while avoiding oscillation.
  - *Freezing Mechanism:* We compare "permanent freeze" (once frozen, always frozen) against "soft freeze" (tokens can unmask if entropy rises again) to test if the former introduces bias.
- **Learned Schedule (MLP):** A small MLP $f_{MLP} : \mathbb{R}^3 \to [0, 1]$ predicts the target noise level $\beta_t^{learned}$ for each step.
  - **Inputs:** (1) Normalized timestep t/T, (2) Global sequence entropy $\bar{H}_t = \frac{1}{L}\sum H_t^{(i)}$, (3) Fraction of currently masked tokens M_t/L.
  - **Training Objective:** The MLP is trained to minimize the **token-wise KL-divergence** between the model's predicted distribution at step t and the **ground-truth clean sequence** from the validation set. Specifically, for a validation set of sequences, we run the full diffusion process to obtain the intermediate states. The "ground truth" target distribution p_{target} is the one-hot distribution of the *original clean tokens* (not the final sampled output x_T), ensuring the MLP learns to drive the model toward the actual data distribution rather than mimicking a potentially flawed sampling trajectory.
  - **Data & Size:** The MLP is trained on a held-out slice of 1,000 sequences from the adaptation corpus. The MLP size is kept minimal (2 hidden layers, 64 units) to ensure it acts as a lightweight adapter.
  - **Generalizability:** We report whether a single MLP trained on LLaMA-8B transfers to DiffuLLaMA-6.74B, and explicitly test cross-family transferability (GPT-2 to LLaMA) to determine if "optimal denoising trajectories" are architecture-specific or objective-specific.

2. Convergence-Efficiency Frontier (CEF) Metric We define **Convergence Efficiency** as the quality achieved per unit of *sampling effort*.

- **Quality Metrics (Q):**
  - **Teacher-Forced Perplexity (TF-PPL):** On WikiText-103. This is calculated using the model's likelihood estimation on the clean sequence (teacher-forced), separate from the generation process, to decouple model capability from sampler efficiency.
  - **Generative Perplexity (Gen-PPL):** The PPL of the *actually generated* sequences. This is reported alongside TF-PPL to identify if the noise schedule introduces sampling artifacts or incoherence that teacher-forced metrics miss.

  - **Infilling Accuracy:** Exact match pass@k on Multi-line HumanEval and MBPP.
  - **MAUVE:** On open-ended C4 prompts.
  - **IFEval:** Instruction following score.
- **Efficiency Metrics (E):**
  - **Total Steps (T):** The primary efficiency variable (16, 32, 64).
  - **Convergence Rate (CR):** The number of steps required to reach 95% of the final quality metric (e.g., 95% of max pass@k or Gen-PPL stability). A lower CR indicates faster convergence.
  - **Wall-Clock Latency:** Reported for completeness, but treated as a secondary metric since it is proportional to T for all schedules in this dense implementation.
- **Frontier Construction:** For each model and schedule, we plot **Quality (Q)** vs. **Total Steps (T)**. We compute the **Area Under the Curve (AUC)** to summarize the trade-off. A higher AUC indicates that the schedule achieves higher quality with fewer steps (faster convergence). We also report the **Step-Reduction Factor (SRF)**: the ratio of steps required by the Linear schedule to reach a target quality threshold compared to the steps required by the best-performing schedule.
- **Baseline Comparison:** We include a simple "Optimal Fixed Schedule" (found via grid search on the same held-out slice used for the MLP, tuning the parameters of a piecewise-linear function) to determine if the MLP's adaptivity provides a benefit over a well-tuned static schedule.

3. Experimental Protocol

- **Models:** DiffuGPT-S/M, DiffuLLaMA-6.74B, Dream-7B, DiffuCoder-7B, LLaDA-8B.
- **Schedules:** Linear, Cosine, Context-Adaptive (with tuned α), Learned (MLP), Optimal Fixed.
- **Steps:** 16, 32, 64.
- **Seeds:** 5 random seeds for sampling to ensure statistical significance.
- **Sequence Length:** Fixed at 128 tokens for primary analysis. *Secondary Analysis:* A subset of experiments (Linear vs. Best Schedule) will be run at 512 tokens and with varying masking ratios (10%, 50%) to test robustness to sequence length and filling density.
- **Analysis:**
  - **Two-way ANOVA:** To test for significant interactions between Schedule, Model Family (GPT-2 vs. LLaMA), and Scale.
  - **Correlation Analysis:** Between the "adaptivity" of the schedule (variance in β_t) and the Convergence Rate (CR).
  - **Statistical Testing:** Paired t-tests to determine if differences in AUC between schedules are significant.
  - **AR Baseline:** We report the wall-clock latency and quality of the corresponding AR base models (GPT-2, LLaMA) to provide context on how DLM efficiency compares to the dominant AR paradigm.

Why this method. This method directly addresses the research problem by providing a systematic, inference-only framework to isolate the effect of the noise schedule on the quality-efficiency frontier, while correcting the validity flaws identified in the previous proposal.

1. **Addressing Undertraining (L7):** By using the noise schedule as an inference-time knob, we test whether a more sophisticated schedule (e.g., Context-Adaptive or Learned) can compensate for the undertraining of the DLMs. If a Context-Adaptive schedule yields a higher AUC on the quality-efficiency frontier compared to a Linear schedule, it suggests that the model's internal representations are better utilized when the denoising process is dynamically adjusted, effectively narrowing the quality gap without additional training.

2. **Compute-Normalized Efficiency (L9) & Validity Correction:** The original work's reliance on wall-clock latency is misleading because it ignores the *convergence* behavior. The previous proposal's "Effective Denoising Steps (EDS)" was critiqued for conflating computational cost with sampling productivity. This revised method explicitly distinguishes between **Computational Cost** (constant per step) and **Sampling Convergence** (quality per step). By defining **Convergence Efficiency** and **Step-Reduction Factor (SRF)**, we provide a rigorous, valid metric that reflects how efficiently the model refines the sequence. This allows us to determine whether a schedule is truly more "efficient" in terms of achieving higher quality with

fewer denoising iterations, which is the relevant metric for inference-time deployment.

3. **Narrow Infilling Evaluation (L8):** By expanding the evaluation to multi-line HumanEval, MBPP, and open-ended text completion, we test the generalizability of the schedule effects. We hypothesize that Context-Adaptive schedules, which preserve bidirectional context by keeping high-uncertainty tokens masked longer, will perform better on complex infilling tasks where the local context is critical. This broader evaluation will reveal whether the schedule's impact is task-dependent.
4. **Architectural Generality (L10):** By testing across GPT-2-based and LLaMA-based models at multiple scales, we can identify family-specific interactions. LLaMA models have a more sophisticated attention mechanism (RoPE, SwiGLU) and were trained with a different attention bias. A Context-Adaptive schedule that modulates noise based on entropy might align better with LLaMA's ability to leverage bidirectional context. The two-way ANOVA will statistically test for these interactions, providing insights into how noise scheduling should be tailored to different architectural families.
5. **Connection to Alternative Routes (L10, L5):** By showing that a simple inference-only change (the noise schedule) can significantly improve the quality-efficiency frontier, we demonstrate that the "adaptation" process can be partially shifted to inference time. This complements alternative routes like representation alignment or uniform-noise adaptation, which require training. If the Learned schedule (trained on a small held-out slice) performs well, it suggests that a lightweight, task-specific schedule can approximate the benefits of more complex training-based adaptations.
6. **Statistical Rigor (L11):** Running 5 seeds for each configuration and reporting mean ± standard deviation addresses the lack of variance estimates in the original paper. This is crucial for determining whether the observed differences in the quality-efficiency frontier are statistically significant or due to sampling noise.
7. **Instruction-Following Deficiency (L5):** Including IFEval allows us to assess whether certain schedules better preserve the model's ability to follow instructions. We hypothesize that schedules that maintain higher uncertainty for longer (e.g., Context-Adaptive) may allow the model to "think" more about the instruction constraints, leading to better instruction-following performance.

Feasibility:

- **Inference-Only:** All experiments use released checkpoints. The "Learned Schedule" MLP is trained on a small validation set using the DLM's forward pass (to compute entropy and KL-divergence) but does not modify the DLM weights. This is feasible on CPU or a single GPU in a short time (hours, not weeks).
- **Compute:** A single NVIDIA RTX 6000 Pro Blackwell (96 GB) is sufficient for batched inference of the 6.74B-8B models. The smaller GPT-2 models can be run on the same GPU with higher batch sizes. The Context-Adaptive and Learned schedules do not increase the memory footprint (they only change the sampling logic and mask updates), so they are feasible.
- **Timeline:** Ten weeks is sufficient. Weeks 1-2: Implement the CEF metrics and the four schedules (including MLP training for Learned). Weeks 3-5: Run experiments on GPT-2 models (fast). Weeks 6-8: Run experiments on LLaMA/Dream models (slower, parallelized). Weeks 9-10: Analysis and write-up.
- **Team:** Three GPU-enabled members can parallelize over models (e.g., one on DiffuGPT, one on DiffuLLaMA, one on Dream/LLaDA). CPU-only members can handle the MLP training for the Learned schedule, metric computation, and statistical analysis.

Significance: This work provides the first systematic, inference-only characterization of how noise scheduling shapes the quality-efficiency trade-off of scaled DLMs. By introducing the CEF metric and a standardized protocol for evaluating schedules, we offer a rigorous and generalizable framework for evaluating DLMs. The results will provide practitioners with a concrete knob (the noise schedule) for deploying adapted DLMs, allowing them to trade off quality and efficiency without retraining. Furthermore, uncovering family- and schedule-specific interactions will guide future

DLM design, potentially leading to more efficient and effective adaptation methods.

Experiment design.

Phase 1: Infrastructure, Standardized Schedule Implementation & Calibration (Weeks 1–2)**

1. **Infrastructure Setup & Tokenizer Consistency:**
   - Deploy the `DiffuLLaMA` repository for DiffuGPT-S/M and DiffuLLaMA-6.74B.
   - Deploy the `dLLM` framework (or specific model repositories) for Dream-7B, LLaDA-8B, and DiffuCoder-7B to ensure standardized inference interfaces.
   - **Tokenizer Protocol:** To address vocabulary discrepancies, all external metrics (MAUVE, IFEval) and perplexity calculations will use the **specific tokenizer associated with each model checkpoint** for tokenization. For cross-family comparisons, quality metrics will be reported per-model, and only normalized metrics (e.g., AUC, SRF) will be compared directly. No arbitrary tokenizer swaps will be performed.
   - **AR Baseline Setup:** Load the corresponding AR base models (GPT-2 Small/Medium, LLaMA-7B/13B equivalents).
     - *Alignment Protocol:* The AR baseline will be evaluated on the **exact same 128-token prompts** and **same sequence length** as the DLMs to ensure a perfectly controlled comparison. AR perplexity will be computed via teacher-forced likelihood in a single forward pass, while DLM TF-PPL will be computed via the model's likelihood estimation on the clean sequence.
     - *Inference Optimization:* The AR baseline will be run using optimized inference kernels (e.g., FlashAttention-2 if supported by the base model, or standard HuggingFace `generate` with `use_cache=True`) to ensure a fair wall-clock comparison. We will explicitly report whether KV-caching was utilized for the AR baseline in the final report.
2. **Schedule Implementation:**
   - **Linear & Cosine:** Implement standard β_t schedules directly in the denoising loop.
   - **Context-Adaptive:** Implement the Dream-style entropy-thresholding mechanism. Modify the `maskable_mask` logic to freeze tokens where predicted entropy $H_t^{(i)} < \alpha \cdot H_{max}$.
     - *Freezing Mechanism (Primary Comparison):* We elevate **Soft Freeze** to a primary comparison against **Permanent Freeze**.
       * *Soft Freeze Logic:* A token is unmasked if its entropy rises above the threshold τ_t in a subsequent step. To prevent oscillation, we implement a **hysteresis band**: a token is frozen if $H_t^{(i)} < 0.9\tau_t$ and unmasked only if $H_t^{(i)} > 1.1\tau_t$. The values 0.9 and 1.1 are initial hyperparameters; if preliminary tests show significant instability, we will perform a small grid search on $\{0.8, 0.9\}$ and $\{1.1, 1.2\}$ on the Calibration Set.
       * *Oscillation Monitoring:* We will explicitly log the number of state flips (masked $\leftrightarrow$ unmasked) per token to quantify oscillation and its impact on wall-clock latency.
   - **Learned (MLP):**
     - *Architecture:* Small MLP (2×64 units, ReLU activation, sigmoid output).
     - *Inputs:* (1) Normalized timestep t/T, (2) Normalized global sequence entropy $\bar{H}_t / \log|V_{model}|$, (3) **Normalized variance of token-wise entropy** $\sigma_H^2 / \log|V_{model}|$ (added to distinguish uniform uncertainty from "hard" token-driven uncertainty), (4) Fraction of currently masked tokens M_t/L.
     - *Training Data:* A dedicated **Calibration Set** of 1,000 sequences. To address domain bias, this set will be a stratified sample: 50% from WikiText-103 (validation), 30% from C4 (open-ended), and 20% from HumanEval/MBPP (code/infilling) to ensure the MLP generalizes across the evaluation domains.
     - *Training Objective:* Minimize the token-wise KL-divergence between the model's predicted distribution at step t and the **one-hot distribution of the original clean tokens** (ground truth).
     - *Optimization:* AdamW optimizer, learning rate 10^{-3}, batch size 32, trained for 50 epochs or until convergence.
     - *Caching Strategy:* During the data preparation phase, we will run the DLM forward passes on the Calibration Set to generate the intermediate states. We will cache the **full vocabulary logits** (or log-probabilities) for each intermediate step and token to

disk (HDF5 or Parquet format). The MLP training itself will be performed on CPU using these cached tensors, requiring **zero GPU calls** during the optimization loop. This ensures the MLP training does not contend with inference jobs.
    - *Family Specificity:* Train separate MLPs for GPT-2-based and LLaMA-based models. Additionally, test cross-family transfer (e.g., LLaMA MLP on GPT-2 model) to assess architecture specificity.
  - **Optimal Fixed:** Perform a grid search on a piecewise-linear schedule (2 segments). The break point t_{break} is optimized as a **discrete variable** over $\{0.2T, 0.4T, 0.6T, 0.8T\}$, and the slopes are optimized to match the start and end noise levels. This ensures the "Learned" schedule is compared against a truly competitive static baseline that is not arbitrarily constrained to continuous optimization.
3. **Hyperparameter Tuning:**
  - For Context-Adaptive, perform a grid search for $\alpha \in \{0.05, 0.1, 0.2\}$ on a subset of 50 prompts from the Calibration Set to select the optimal α per model family.

Phase 2: Primary Inference Experiments (Weeks 3–8)

Conduct inference on the following grid:

- **Models:** DiffuGPT-S, DiffuGPT-M, DiffuLLaMA-6.74B, Dream-7B, LLaDA-8B, DiffuCoder-7B.
- **Schedules:** Linear, Cosine, Context-Adaptive (tuned α, Soft Freeze primary), Learned (MLP), Optimal Fixed.
- **Steps (T):** 16, 32, 64.
- **Seeds:** 5 distinct random seeds for sampling.
- **Tasks & Metrics:**
  1. **Generative Quality (Gen-PPL & MAUVE):** Generate text on 500 prompts from C4 and WikiText-103. Compute Generative Perplexity (using the model's own tokenizer) and MAUVE scores.
    - *Normalization:* To facilitate cross-family comparison, we will also report **Bits-Per-Character (BPC)** for Gen-PPL, calculated as $\log_2(PPL) \times \frac{\text{num_chars}}{\text{num_tokens}}$. This normalizes for vocabulary size differences between GPT-2 and LLaMA tokenizers.
  2. **Infilling Accuracy:** Evaluate on Multi-line HumanEval and MBPP infilling tasks (50 samples each). Compute exact-match pass@1 and pass@10.
  3. **Instruction Following:** Evaluate on IFEval (50 samples). Compute instruction-following accuracy.
  4. **Token-Level Stability:** For the final 10% of steps (last $\lfloor 0.1T \rfloor$ steps), measure the **Token Jitter Rate**: the fraction of positions where the predicted token ID changes between consecutive steps. This metric identifies "jitter" or "collapse" near convergence.
  5. **Efficiency:**
    - **Wall-Clock Latency:** Record total time for T steps (batch size = 1).
    - **Dense FLOPs:** Calculate actual computational cost as $F \approx 2 \times P \times T \times L$, where L is the fixed sequence length (128). *Note: We use full L because the implementation performs a dense forward pass on the entire sequence regardless of mask status.*
    - **Information Update Density (IUD):** *Secondary metric only.* Calculate the average fraction of tokens actually updated per step (M_t/L) to analyze "useful" work, but explicitly label this as a proxy for information processing, not hardware FLOPs.
  6. **Teacher-Forced Baseline (TF-PPL):** Compute TF-PPL on WikiText-103 for each model *once* (independent of schedule) to establish the model's inherent capability ceiling.
    - *Method Specification:* TF-PPL is computed by running the DLM in "teacher-forced" mode on the clean sequence. At each position i, the model predicts the next token given the clean prefix $x_{<i}$ and the clean suffix $x_{>i}$ (if bidirectional attention is used) or just the prefix (if causal). We report the average negative log-likelihood of the ground-truth tokens. This is distinct from the generative PPL which involves iterative denoising.

Phase 3: Robustness & Secondary Analysis (Weeks 7–8, Parallel to Primary)

- **Sequence Length Robustness:** For the best-performing schedule per family and the Linear baseline, run inference at sequence length 512 on 50 prompts to test generalization.

- **Masking Ratio Robustness:** For the best-performing schedule per family and the Linear baseline, run infilling with varying masking ratios (10%, 50%) on HumanEval to test sensitivity to filling density.
- **Cross-Family Transfer Nuance:** Explicitly document that MLP inputs are normalized by $\log |V_{model}|$ to ensure scale-invariance. Verify that the MLP does not rely on any token-specific IDs that would differ between GPT-2 and LLaMA by inspecting the input pipeline. Additionally, analyze the distribution of the MLP inputs (entropy, variance) for the source and target families to check for Out-of-Distribution (OOD) issues. If the input distributions diverge significantly, we will report the transfer performance with a caveat regarding input distribution shift.

Phase 4: Statistical Analysis & Frontier Construction (Weeks 9–10)

1. **CEF Frontier Construction:** For each model-schedule pair, plot Quality (Gen-PPL, pass@1) vs. Total Steps (T). Calculate the Area Under the Curve (AUC) and Step-Reduction Factor (SRF).
2. **Statistical Testing:**
   - **Normality Check:** Perform Shapiro-Wilk tests on the quality metrics (Gen-PPL, pass@k) across seeds.
   - **ANOVA:** If metrics are normally distributed, perform two-way ANOVA to test interactions between **Schedule** and **Model Family** (GPT-2 vs. LLaMA). If metrics are skewed (common for pass@k), use the **Scheirer-Ray-Hare test** or Kruskal-Wallis test as a non-parametric alternative.
   - **Pairwise Comparisons:** Perform paired t-tests (or Wilcoxon signed-rank tests if non-parametric) to compare AUC values between the best adaptive schedule and the Linear baseline for each model.
   - Report mean ± standard deviation across the 5 seeds.
3. **Correlation Analysis:** Correlate the "adaptivity" of the schedule (variance in β_t) with the Convergence Rate (steps to reach 95% of final quality).
4. **AR Baseline Comparison:** Report the quality (PPL, pass@k) and latency of the corresponding AR base models (GPT-2, LLaMA) for context, ensuring tokenizer and prompt alignment as defined in Phase 1.

Resource Management Strategy:

- **Queue Prioritization:** To mitigate shared GPU contention, implement a strict priority queue. **8B models** (LLaDA, Dream, DiffuLLaMA) are prioritized during off-peak hours as they dominate VRAM and compute time. **GPT-2 models** are scheduled during peak contention or used for pipeline validation due to their smaller footprint.
- **Parallelization:** Three GPU-enabled members will parallelize over models (e.g., one on DiffuGPT, one on DiffuLLaMA, one on Dream/LLaDA). CPU-only members will handle the MLP training (using cached forward passes), metric computation, and statistical analysis.
- **Compute Budgeting:** The 500 prompts for Gen-PPL/MAUVE are fixed. For the 8B models, batch size will be set to the maximum that fits in 96GB VRAM (likely 1-4 depending on sequence length and attention implementation). We will monitor GPU utilization and adjust batch sizes dynamically to prevent OOM errors while maximizing throughput.

Why this design. This revised experiment design addresses the specific feedback from the review while maintaining the rigor and feasibility of the original proposal.

1. **Addressing Clarity & Precision (Soft Freeze & AR Baseline):**
   - **Soft Freeze Logic:** The previous ambiguity regarding "soft freeze" is resolved by elevating it to a primary comparison and defining a concrete **hysteresis band** (freeze at $< 0.9\tau$, unmask at $> 1.1\tau$). This prevents rapid oscillation, which could otherwise distort wall-clock latency and introduce noise into the efficiency metrics. The explicit logging of state flips allows for a direct analysis of the cost-benefit of bidirectional context updates. The values for the hysteresis band are now explicitly noted as initial hyperparameters with a plan for grid search if instability is observed, enhancing transparency.
   - **AR Baseline Alignment:** The AR baseline is now strictly aligned with the DLM experiments in terms of prompt selection, sequence length (128 tokens), and tokenizer us-

age. This ensures that any observed differences in the quality-efficiency frontier are attributable to the diffusion paradigm and noise schedule, not confounding factors like prompt length or vocabulary mismatch. The review's point about AR inference optimization is addressed by explicitly stating that the AR baseline will use optimized inference (KV-cache/FlashAttention) and that this will be reported, ensuring a fair wall-clock comparison.
    - **TF-PPL Specification:** The review's minor point regarding the "likelihood estimation" method for DLM TF-PPL is addressed by explicitly defining it as the average negative log-likelihood of ground-truth tokens in a teacher-forced pass, distinct from the iterative generative process.
2. **Enhancing Validity (MLP Inputs, Data & Tokenizer Handling):**
    - **MLP Data Diversity:** To address the concern about overfitting the MLP to WikiText-103, the Calibration Set is now a stratified sample including WikiText-103, C4, and Code (HumanEval/MBPP) sequences. This ensures the learned schedule is robust across the domains used in evaluation.
    - **MLP Caching & Feasibility:** The caching strategy is refined to specify that **full vocabulary logits** are cached to disk in a structured format (HDF5/Parquet). This allows the MLP training to run entirely on CPU with zero GPU contention, addressing the feasibility concern about redundant GPU calls and disk I/O bottlenecks (structured formats are efficient for this purpose).
    - **Tokenizer Consistency:** The design explicitly mandates using the **model's own tokenizer** for all perplexity and MAUVE calculations. To address the cross-family comparison issue raised in the reproducibility review, we now also report **Bits-Per-Character (BPC)**, which normalizes perplexity by character count, allowing for a more fair comparison of "efficiency" (information per character) across models with different vocabulary sizes (GPT-2 vs. LLaMA).
    - **Optimal Fixed Baseline:** The "Optimal Fixed" schedule grid search is now specified to use discrete break points, ensuring the static baseline is competitive and not arbitrarily constrained.
3. **Improving Robustness (Cross-Family Transfer & Metrics):**
    - **Cross-Family Transfer:** The design now explicitly documents the normalization of MLP inputs by $\log|V_{model}|$ and verifies that no token-specific IDs are used. Additionally, we will analyze the input distributions for OOD issues, providing a more nuanced interpretation of cross-family transfer results.
    - **Oscillation Monitoring:** By logging state flips in the Soft Freeze mechanism, the experiment can directly measure the "cost" of bidirectional context updates.
    - **Token-Level Stability:** As suggested in the robustness review, we add a **Token Jitter Rate** metric (fraction of tokens changing in the last 10% of steps). This provides deeper insight into convergence behavior and whether certain schedules cause instability near the end of generation.
4. **Ensuring Feasibility (Resource Contention & Statistical Rigor):**
    - **GPU Queueing:** The strict priority queue for 8B models during off-peak hours directly addresses the risk of shared GPU contention.
    - **Statistical Testing:** The analysis plan now includes a **normality check** (Shapiro-Wilk) before ANOVA. If metrics are skewed (likely for pass@k), we will use non-parametric alternatives (Scheirer-Ray-Hare/Kruskal-Wallis), addressing the validity review's concern about ANOVA assumptions.
    - **Compute Budgeting:** The plan explicitly acknowledges the 500-prompt bottleneck and monitors GPU utilization to optimize batch sizes, ensuring the timeline is met.
5. **Maintaining Reproducibility (Detailed Specifications):**
    - The detailed specifications for the MLP (architecture, inputs, objective, normalization, caching format) and the Soft Freeze mechanism (hysteresis band values and tuning plan) ensure that an external researcher can replicate the experiment exactly. The separation of the Calibration Set from evaluation sets remains a key feature for preventing data leakage. The explicit reporting of BPC and the definition of TF-PPL enhance the reproducibility and comparability of the results.

This revised design provides a robust, valid, and feasible path to systematically characterize the impact of noise schedules on the quality-efficiency frontier of scaled DLMs, addressing all critical feedback points while preserving the scientific rigor of the original proposal.

Method	Clarity	Validity	Rigorousness	Innovativeness	Generalizability
rating	5	5	5	4	5

Experiment	Clarity	Validity	Robustness	Feasibility	Reproducibility
rating	5	5	5	5	5

Weakest point per the reviewers (Innovativeness, 4/5). 1. **MLP Target Distribution:** The decision to train the MLP against the "ground-truth clean sequence" rather than the model's own final sampled output is a strong methodological choice that avoids "drift" or mimicking a flawed sampler.

2. **Computational Validity:** The method correctly acknowledges the "constant-cost assumption" of dense implementations, avoiding the previous error of claiming FLOP reductions through masking. This ensures the results will be technically sound.
3. **Potential Refinement:** While the "soft freeze" vs. "permanent freeze" comparison is mentioned, the researcher could further strengthen the innovativeness by exploring if the MLP can predict *token-specific* noise levels (similar to the Context-Adaptive approach) rather than just a global β_t.
4. **Baseline Strength:** The addition of an "Optimal Fixed Schedule" (via grid search) is critical; it ensures that the MLP's success is attributed to its *adaptivity* rather than just better parameter tuning.

Idea 2.2 — P2-M7 (overall 4.9/5)

Method.

Method: Schedule-Conditioned Token Trajectory Analysis (SCTTA) with Compute-Accurate Frontier Mapping**

We propose a rigorous, inference-only methodology to characterize the quality-efficiency frontier of diffusion language models (DLMs) by analyzing the **temporal stability** of individual tokens across the denoising trajectory. Unlike previous methods that attempt to optimize the schedule via counterfactual perturbations (which suffer from distribution shift and high computational cost) or rely on global entropy proxies, SCTTA treats the noise schedule as a **filter for token commitment**. Our core insight is that the "efficiency" of a schedule is determined by how quickly high-confidence tokens "lock in" (stop changing) while low-confidence tokens continue to refine.

The method introduces two key innovations:

1. **Token Commitment Curve (TCC):** A metric that tracks the fraction of tokens that have reached a stable state (defined by a confidence threshold) at each denoising step, providing a direct measure of the "effective" progress of the denoising process.
2. **Compute-Accurate Frontier (CAF):** A rigorous efficiency metric that accounts for the actual computational cost of the dense forward pass (which is constant per step in the verified implementation) but normalizes it by the **Effective Denoising Yield (EDY)**, which measures the actual reduction in sequence uncertainty per step. This avoids the fallacy of assuming sparse computation while accurately reflecting the "usefulness" of each step.

1. Core Mechanism: Token Commitment Curve (TCC)

In discrete diffusion language models (like DiffuLLaMA), the state at step t is a sequence of token IDs. A token is considered "committed" if the model's predicted probability for that token is sufficiently high and stable. We define the **Confidence Score** $C_i(t)$ for token i at step t as the negative log-likelihood of the *sampled* token:

$$C_i(t) = -\log p_\theta(x_i^{(t)} | x_{-i}^{(t)}, t)$$

Where $x_i^{(t)}$ is the token ID at position i after step t, and p_θ is the model's predicted probability distribution over the vocabulary. *Note: We use the sampled token's NLL to reflect the actual trajectory of the generation process, acknowledging that early steps may exhibit higher variance due to stochastic sampling.*

- **Relative Commitment Threshold:** To ensure generalizability across different vocabulary sizes (e.g., GPT-2 vs. LLaMA) and model scales, we define commitment using a **relative confidence gap** rather than a fixed nats value. A token i is "committed" at step t if:

$$\frac{p_\theta(x_i^{(t)}|\cdot) - \max_{v \neq x_i^{(t)}} p_\theta(v|\cdot)}{p_\theta(x_i^{(t)}|\cdot)} > \tau_{rel}$$

  Where $\tau_{rel} \in [0, 1)$ represents the required dominance of the top token over the second-best candidate. We will validate τ_{rel} on a small held-out validation subset (disjoint from test sets) by selecting the value that maximizes the correlation between "committed" status at step t and the final token identity at step T. We will also test sensitivity to $\tau_{rel} \in \{0.1, 0.3, 0.5\}$ to ensure robustness.

- **Stability Constraint:** To avoid counting "confident but oscillating" tokens as committed, a token is only marked as committed if it has remained at the same token ID for the last $K = 2$ consecutive steps. This ensures that "commitment" reflects genuine stabilization, not just a transient spike in confidence.

- **Token Commitment Curve (TCC):** For a sequence of length L, we define the fraction of committed tokens at step t as:

$$TCC(t) = \frac{1}{L} \sum_{i=1}^{L} \mathbb{1}[\text{Committed}_i(t)]$$

  This curve starts near 0 (all tokens noisy/uncertain) and approaches 1 (all tokens confident and stable) as denoising progresses.

- **Interpretation:**
  - **Fast Rising TCC:** Indicates the model quickly resolves most tokens. This is efficient if the final quality is high.
  - **Slow/Plateauing TCC:** Indicates the model struggles to commit to tokens, suggesting high uncertainty or a mismatch between the noise schedule and the model's capability.
  - **Late Spikes:** If TCC rises sharply only in the last few steps, it suggests the earlier steps were "wasted" on low-confidence wandering.
  - **Premature Commitment:** If TCC rises quickly but final quality drops, it indicates "overconfidence" where the model locks in incorrect tokens too early.

2. Compute-Accurate Frontier (CAF) Metric

To address the non-compute-normalized efficiency limitation rigorously, we reject metrics that assume variable FLOPs per step (since the verified implementation performs a dense forward pass regardless of token state). Instead, we define efficiency based on **Yield per Fixed Compute Unit**.

- **Fixed Compute Unit (FCU):** One denoising step (forward pass + sampling) for a sequence of length $L = 128$. The cost is $FCU = 1$ unit.

- **Effective Denoising Yield (EDY):** The reduction in average uncertainty per step. We define the **Sequence Uncertainty** $U(t)$ as the average negative log-likelihood across all tokens:

$$U(t) = \frac{1}{L} \sum_{i=1}^{L} C_i(t)$$

  The yield at step t is $\Delta U(t) = U(t-1) - U(t)$.

- **Total Effective Yield:** $Y_{total} = \sum_{t=1}^{T} \max(0, \Delta U(t))$. Note: We use $\max(0, \cdot)$ because uncertainty can sometimes increase (wandering).

- **Compute-Accurate Efficiency (CAE):**

$$CAE = \frac{Y_{total}}{T}$$

  This metric measures how much "actual work" (uncertainty reduction) is done per step. A schedule with high CAE is efficient because each step significantly reduces the model's uncertainty. A schedule with low CAE is inefficient because steps are "wasted" on steps where the model does not reduce uncertainty (or increases it).

- **Frontier Construction:** We plot **Quality** (Q, e.g., Perplexity or pass@1) vs. $1/CAE$ (inverse efficiency). Higher CAE means lower $1/CAE$. The Area Under the Curve (AUC) of this frontier summarizes the trade-off. A schedule that achieves high quality with high CAE (low $1/CAE$) is superior. To avoid tautology, the Quality axis uses task-specific metrics (pass@1, MAUVE) rather than uncertainty reduction itself.

3. Experimental Protocol

- **Models:** DiffuGPT-S/M, DiffuLLaMA-6.74B, Dream-7B, DiffuCoder-7B, LLaDA-8B.
- **Schedules:**
  1. **Linear:** $\beta_t = 1 - t/T$.
  2. **Cosine:** $\beta_t = \cos(\frac{\pi}{2}\frac{t}{T})$.
  3. **Context-Adaptive:** As in Dream 7B (token-wise entropy conditioned).
  4. **Learned:** A 2-layer MLP (hidden dim 64) predicting β_t from t. *Implementation Detail:* The MLP is trained on a separate validation slice of the adaptation corpus, strictly disjoint from the test sets used for Quality metrics, to prevent data leakage.
- **Steps:** 16, 32, 64.
- **Sequence Length:** Fixed at 128 tokens.
- **Metrics:**
  - **Quality:** Perplexity (WikiText-103), pass@1 (HumanEval/MBPP), MAUVE, IFEval.
  - **Efficiency: CAE** (Compute-Accurate Efficiency), Wall-clock latency.
  - **Diagnostics: TCC** curves (Commitment vs. Step) for each schedule/model.
- **Analysis:**
  - **TCC Shape Comparison:** Plot TCC(t) for GPT-2-based vs. LLaMA-based models. We specifically test for a "wave" of commitment (sequential-like) in GPT-2 models versus a "global" commitment in LLaMA models, which would provide a concrete empirical signature of the underlying attention bias.
  - **Family Interaction:** Perform a two-way ANOVA on the **Slope of TCC at 50% Commitment** with factors *Model Family* and *Schedule Type*. Does the Context-Adaptive schedule flatten the TCC (reducing commitment speed) or sharpen it?
  - **Frontier Mapping:** Plot Quality vs. $1/CAE$ for all schedules. Identify the Pareto optimal schedule for each model.
  - **Premature Commitment Analysis:** Correlate the **Time to 90% Commitment** with the **Final Quality**. We specifically look for "premature commitment" (where TCC rises quickly but quality drops), providing insight into the "over-confidence" vs. "refinement" trade-off.

4. Feasibility and Rigor

- **Inference-Only:** TCC and CAE are computed from the logits and token IDs during the standard inference loop. No weight updates or extra forward passes are required. The confidence score $C_i(t)$ is already available from the model's output logits at each step.
- **Compute:** The overhead is negligible (calculating $-\log p$ for sampled tokens and tracking token IDs). The total number of runs is 5 models $\times$ 4 schedules $\times$ 3 steps $\times$ 5 seeds = 300 runs. This is highly feasible on a 96GB GPU within the ten-week timeline.
- **Statistical Rigor:** 5 seeds per configuration. Report mean $\pm$ std for CAE and the key TCC metrics (e.g., Time to 50% Commitment). Threshold sensitivity is analyzed by reporting results for $\tau_{rel} \in \{0.1, 0.3, 0.5\}$.

Innovation:

1. **Token-Level Commitment Dynamics:** Moves beyond global metrics to analyze *when* and *how* individual tokens stabilize, providing a mechanistic view of the denoising process.
2. **Compute-Accurate Efficiency:** Defines a rigorous efficiency metric (CAE) that respects the dense forward pass constraint of the implementation, measuring "useful" compute (uncertainty reduction) rather than assuming sparse operations.
3. **Architectural Diagnostic:** The TCC shape serves as a diagnostic tool for architectural biases (causal vs. bidirectional), directly addressing the "Architectural Generality" research question by detecting sequential vs. global commitment patterns.
4. **Actionable Insight:** Identifies whether "fast commitment" is beneficial or detrimental (premature commitment), guiding practitioners on whether to use aggressive or conservative schedules for specific model families.

Why this method. This method directly addresses the research problem by investigating how noise schedules affect the quality-efficiency frontier and whether this relationship differs across families and scales. The rationale is grounded in the following observations:

1. **Architectural Generality (L10):** The target paper notes that the adaptation recipe was applied to GPT-2 and LLaMA, but does not analyze the *temporal dynamics* of denoising. Our TCC metric explicitly tests the hypothesis that causal (GPT-2) and bidirectional (LLaMA) models have different "commitment" patterns. For example, bidirectional models might resolve tokens more uniformly ("global commitment"), while causal models might resolve them in a more sequential manner ("wave of commitment"), even within a parallel diffusion framework. This provides a concrete empirical signature of architectural bias.

2. **Undertrained Models (L7):** The target paper highlights that adapted DLMs are undertrained. We hypothesize that undertraining manifests as a "slow" or "noisy" TCC, where the model fails to commit to tokens efficiently. By identifying schedules that produce a sharper TCC (faster commitment) without triggering premature commitment, we can find a training-free lever to mitigate the impact of undertraining.

3. **Non-Compute-Normalized Efficiency (L9):** The original work uses wall-clock latency, which does not account for the "usefulness" of the compute. Our CAE metric addresses this by measuring the actual reduction in uncertainty per step, providing a more accurate measure of the quality-efficiency trade-off that is consistent with the dense forward pass implementation. By using a relative confidence threshold, we ensure this metric is generalizable across different vocabulary sizes and model scales.

4. **Connection to Alternative Routes (L10, L5):** Related work on representation alignment and uniform-noise adaptation suggests that the *geometry* of the latent space matters. Our method provides a complementary view by analyzing the *temporal dynamics* of token commitment. By identifying the optimal schedule shape for each family, we offer a training-free lever to improve performance, potentially approaching the benefits of more complex adaptation techniques.

5. **Statistical Rigor (L11):** The method includes multiple seeds and rigorous statistical analysis (ANOVA, correlation tests, threshold sensitivity analysis), addressing the lack of variance estimates in the original paper. The use of a relative threshold and stability constraint enhances

the robustness of the metrics across diverse models.

6. **Feasibility:** The method is strictly inference-only, uses released checkpoints, and requires only negligible additional computation for metric calculation, making it feasible within the given resource constraints. The "Learned Schedule" MLP is trained on a separate validation slice to avoid data leakage, ensuring the validity of the comparison.

This method is substantively different from previous proposals because it focuses on the **temporal stability of individual tokens** rather than relying on global proxies or counterfactual perturbations. It does not attempt to optimize the schedule via a complex LP problem or assume sparse computation, but rather measures the "effective" progress of the denoising process in a way that is consistent with the verified implementation. This provides a rigorous, data-driven basis for understanding how noise schedules interact with model architecture to shape the quality-efficiency frontier.

Experiment design.

Phase 1: Implementation, Calibration, and Schedule Definition (Weeks 1–2)**

1. **Environment & Memory-Safe Inference Hooks:**
   - Initialize environments for `DiffuGPT-S/M`, `DiffuLLaMA-6.74B`, `Dream-7B`, `DiffuCoder-7B`, and `LLaDA-8B`.
   - **Precision Standardization:** All inference is performed in **BF16** (BFloat16) to ensure numerical stability and consistency across models, particularly for the 7B-8B parameter models. Logits are converted to FP32 on-GPU immediately before computing probabilities to prevent underflow/overflow issues in the exponential function, ensuring precise calculation of p_{top1}, p_{top2}, and NLL.
   - **Memory-Safe Hooking:** Modify generation loops to intercept logits at step t. **Crucially**, do not store full logit tensors in VRAM. Instead, on the GPU, immediately compute:
     1. p_{top1} and p_{top2} (probabilities of the top-2 tokens).
     2. The negative log-likelihood (NLL) of the sampled token $x_i^{(t)}$.
     3. The Shannon entropy $H_i(t)$ of the distribution at position i. Move only these scalar values (size $L \times 4$ per step) to CPU for storage.
   - **CPU Memory Footprint Estimate:**
     - Per run: 128 tokens × 64 steps × 4 floats × 4 bytes/float ≈ 131 KB.
     - Total for 1,440 runs: ≈ 189 MB.
     - This is negligible for standard CPU RAM, confirming the feasibility of storing full trajectories for all runs.
   - **Metric Computation Module (CPU):** Implement the SCTTA metrics:
     - $C_i(t) = -\log p_\theta(x_i^{(t)}|\cdot)$.
     - **Commitment Status:** $\text{Gap}_i(t) = \frac{p_{top1}-p_{top2}}{p_{top1}}$. Committed if $\text{Gap}_i(t) > \tau_{rel}$ AND $x_i^{(t)} = x_i^{(t-1)} = x_i^{(t-2)}$.
     - **TCC & CAE:** Compute $TCC(t)$, $U(t)$, and CAE as defined in the method.
     - **Stability Failure Rate (SFR):** Compute the fraction of tokens that meet the confidence threshold ($\text{Gap}_i(t) > \tau_{rel}$) but fail the $K = 2$ stability check (i.e., $x_i^{(t)} \neq x_i^{(t-1)}$). This metric quantifies "wandering" behavior directly.
     - **Dual-Track TCC Reporting:** For trajectories with $T = 16$, compute and report TCC both **with** the $K = 2$ stability constraint and **without** it (i.e., $K = 1$) to assess the impact of the stability filter on short-horizon commitment estimates.
2. **Threshold Calibration (τ_{rel}):**
   - Run a pilot study ($N = 100$ sequences) on `DiffuLLaMA-6.74B` (Linear schedule, $T = 32$).
   - Calculate Pearson correlation between "Committed at step t" and "Final Token Identity at step T".
   - Select $\tau_{rel} \in \{0.1, 0.3, 0.5\}$ maximizing this correlation. Fix this global τ_{rel} for all models to ensure comparability, but report sensitivity for all three values in the final analysis.
3. **Schedule Implementation & Standardization:**
   - **Linear:** $\beta_t = 1 - t/T$.

- **Cosine:** $\beta_t = \cos(\frac{\pi}{2}\frac{t}{T})$.
- **Context-Adaptive (Standardized Entropy-Modulated Schedule - SCAS):** To ensure reproducibility and fair comparison across models where Dream 7B's specific code may not be directly transferable, we define a **Standardized Context-Adaptive Schedule (SCAS)**:

$$\beta_t^{pos} = \beta_t^{base} \cdot \left(1 + \lambda \cdot \frac{H_i(t)}{H_{max}}\right)$$

  Where β_t^{base} is the Linear schedule, $H_i(t)$ is the Shannon entropy of the logits at position i (computed in Phase 1 hooks), $H_{max} = \log(V)$, and $\lambda = 0.5$ (a fixed hyperparameter validated in Phase 1). This explicitly modulates the noise level based on local uncertainty, consistent with the "context-adaptive" concept, without relying on proprietary Dream 7B internal states.
- **Learned Schedule (Defined Objective & Implementation):**
  - **Architecture:** 2-layer MLP (Input: t/T, Output: β_t), hidden dimension 64, **GELU activation**.
  - **Optimization Strategy:** To adhere to "inference-only" constraints and ensure reproducibility without complex differentiable surrogates, we use a **Coarse-to-Fine Grid Search** over a **low-dimensional parameterization** of the schedule.
  - **Parameterization:** Instead of searching over all MLP weights (high-dimensional), we fix the MLP architecture and optimize only the **output bias terms** and a single **global scaling factor** α. The schedule is defined as $\beta_t = \alpha \cdot \text{MLP}_{fixed}(t) + b_t$, where b_t is a small per-step bias vector.
  - **Grid Search Range:**
    * Coarse Phase: Search $\alpha \in [0.5, 1.5]$ with step 0.1 (11 values) and $b_t \in [-0.1, 0.1]$ with step 0.05 (5 values per step, but we constrain b_t to be smooth, e.g., a quadratic function $b_t = c_1 t^2 + c_2 t$, searching $c_1, c_2 \in [-0.1, 0.1]$ with step 0.05).
    * Fine Phase: Refine the best 3 candidates from the coarse phase with step size 0.01.
  - **Objective:** The MLP parameters are selected to minimize the **Average Uncertainty Reduction Loss**:

$$\mathcal{L} = \frac{1}{N_{val}} \sum_{j=1}^{N_{val}} \left(U_j(0) - U_j(T)\right)$$

    Evaluated on a fixed validation set ($N_{val} = 50$ sequences, disjoint from test sets) using the model's forward pass. This is fully deterministic and reproducible.

4. **Dataset Curation (Reproducibility Focus):**
   - **WikiText-103:** Use the standard test split. Split into non-overlapping 128-token windows using a fixed stride of 128. Use **500 windows**.
   - **AR Baseline:** Compute teacher-forced perplexity for the corresponding AR models (GPT-2, LLaMA) on the **same 500 WikiText-103 windows** to establish a "gold standard" quality ceiling for quantifying the undertraining gap.
   - **Infilling (HumanEval/MBPP):** Select 100 problems. Create prompts with **varying mask ratios**: 20%, 50%, and 80% of the middle tokens masked. **Crucially**, for each problem, the "gold" prefix and suffix tokens are kept **constant** across all mask ratios; only the number of masked tokens in the middle varies. This isolates the effect of the noise schedule from the effect of available context length.
   - **C4 Prompts:** Select 500 prompts (length ≤ 64 tokens) for MAUVE.
   - **IFEval:** Select 100 instruction-following prompts.
   - **Stop-Word List:** For the Commitment Variance Analysis, use the **NLTK English Stop-Word List** (version 2.0) plus a predefined set of punctuation tokens (e.g., `.`, `,`, `!`, `?`, `;`, `:`, `(`, `)`) to define the "trivial" tokens to be excluded. This ensures exact reproducibility of the filter.
   - *Note:* All data splits and random seeds for data selection are fixed and documented in the repository for replication.

Phase 2: Main Experiment Execution (Weeks 3–7)

1. **Grid Search Execution:**

- **Models:** DiffuGPT-S, DiffuGPT-M, DiffuLLaMA-6.74B, Dream-7B, DiffuCoder-7B, LLaDA-8B.
- **Schedules:** Linear, Cosine, SCAS (Standardized Context-Adaptive), Learned.
- **Steps:** $T \in \{16, 32, 64\}$.
- **Seeds:** 5 random seeds per configuration.
- **Total Runs:** 6 models $\times$ 4 schedules $\times$ 3 steps $\times$ 5 seeds $\times$ 4 datasets $\approx$ 1440 inference runs.

2. **Resource Allocation & Parallelization (Prioritized for Feasibility):**
   - **GPU (3 members):**
     - **Priority:** 7B-8B models (DiffuLLaMA, Dream, DiffuCoder, LLaDA) are prioritized first to mitigate VRAM/latency bottlenecks.
     - Member A: DiffuLLaMA-6.74B, DiffuCoder-7B.
     - Member B: Dream-7B, LLaDA-8B.
     - Member C: DiffuGPT-S/M (Small models, fast iterations, can run multiple seeds in parallel if VRAM allows, otherwise sequential). *Note: Member C can assist with 7B models if A/B are blocked.*
     - *Strategy:* Each member runs their assigned models sequentially across schedules and steps. The memory-safe hook ensures no VRAM spikes.
   - **CPU (4 members):**
     - Member D: Preprocessing (Data splitting, prompt construction, AR baseline perplexity computation).
     - Member E: Metric Computation (TCC, CAE, Perplexity, MAUVE, IFEval scoring, SFR calculation).
     - Member F: Statistical Analysis (ANOVA, Correlations, Domain Consistency checks).
     - Member G: Documentation & Reproducibility (Code versioning, data logging, Stopword list management).
3. **Data Collection:**
   - Save for each run: Final generated text, TCC(t) trajectory (both with and without $K = 2$ constraint for $T = 16$), U(t) trajectory, CAE, SFR, Wall-clock latency, and Quality scores.

Phase 3: Analysis & Reporting (Weeks 8–10)

1. **Quality-Efficiency Frontier Mapping:**
   - Plot Quality (pass@1, MAUVE) vs. 1/CAE for each model/schedule.
   - Compute AUC of the frontier for each model to summarize efficiency.
   - **Undertraining Gap Quantification:** For each DLM configuration, compute the ratio of DLM Perplexity to the corresponding AR Baseline Perplexity. Analyze whether specific schedules (e.g., SCAS) significantly reduce this ratio, indicating a mitigation of the undertraining gap.
2. **TCC Shape & Architectural Diagnostics:**
   - **Commitment Variance Analysis:** To distinguish "sequential" (GPT-2) from "global" (LLaMA) commitment, compute the **Variance of Commitment Times** across positions for each sequence.
     - *Control:* Exclude positions corresponding to tokens in the **NLTK Stop-Word List** and **punctuation tokens** from this variance calculation to avoid artificial variance from trivially easy tokens.
     - *Hypothesis Test:* GPT-2 models should exhibit higher variance in commitment times (sequential wave), while LLaMA models should exhibit lower variance (global resolution).
   - **Positional Control:** For infilling tasks, analyze commitment times separately for "left context," "masked region," and "right context" to ensure mask position does not confound the architectural diagnostic.
   - **Short-Horizon Analysis:** Compare the TCC curves for $T = 16$ with and without the $K = 2$ stability constraint to determine if the stability filter is too aggressive for short trajectories.
   - **Stability Failure Rate (SFR) Analysis:** Analyze SFR across schedules. High SFR indicates "wandering" where tokens are confident but unstable. Test if SCAS or Learned schedules reduce SFR compared to Linear/Cosine, suggesting better noise matching.
3. **Statistical Testing:**

- **Two-Way ANOVA:** Factor 1: Model Family (GPT-2, LLaMA). Factor 2: Schedule Type. Dependent Variable: Slope of TCC at 50% Commitment. **Crucially**, perform ANOVA on the **aggregated metrics per configuration** (mean across seeds) to avoid inflating degrees of freedom.
- **Premature Commitment Analysis:** Correlate **Time to 90% Commitment** with **Final Quality** (Perplexity/pass@1). A negative correlation (fast commitment, low quality) indicates premature commitment.
- **Mask Ratio Interaction:** Explicitly check if the "optimal" schedule shifts as the mask ratio increases from 20% to 80%. Analyze if higher noise levels require more conservative schedules (e.g., Linear) to avoid premature commitment, while lower noise levels benefit from aggressive schedules (e.g., SCAS).
- **Domain Consistency Check:** Compare the ranking of schedules (by CAE and Quality) on WikiText-103 vs. HumanEval/MBPP. If the "optimal" schedule differs significantly across domains, report this as a limitation of the universality of the schedule-family interaction.

4. **Sensitivity & Robustness:**
   - Report results for $\tau_{rel} \in \{0.1, 0.3, 0.5\}$.
   - Report results for Infilling mask ratios (20%, 50%, 80%) to see if schedule performance depends on missing information density.
   - Report the impact of the $K = 2$ constraint on $T = 16$ TCC values.
5. **Final Report:**
   - Summarize the optimal schedule for each family/scale.
   - Validate the TCC diagnostic as a tool for architectural bias detection.
   - Provide actionable insights for practitioners on selecting inference schedules.
   - Discuss the extent to which schedule optimization mitigates the undertraining gap relative to the AR baseline.

Why this design. This revised experiment design rigorously addresses the feedback from the reviews while maintaining the core SCTTA methodology.

1. **Reproducibility & Clarity (Addressing Review 5 & 1):**
   - **Learned Schedule:** The ambiguous "gradient-free search" is replaced with a clear, deterministic **coarse-to-fine grid search** over a **low-dimensional parameterization** (global scaling α and quadratic bias b_t). This ensures the search space is manageable and fully reproducible, avoiding the instability of REINFORCE or complex differentiable approximations of non-differentiable sampling. The exact ranges and step sizes are specified.
   - **Context-Adaptive Schedule:** The proprietary Dream 7B logic is replaced with a **Standardized Context-Adaptive Schedule (SCAS)** defined by an explicit formula (β_t^{pos} modulated by local entropy). This ensures fair comparison across all models and eliminates the "temperature modulation" inconsistency flagged in the reviews.
   - **Dataset Specification:** Specific numbers (500 WikiText windows, 100 infilling problems) and methods (fixed stride, varying mask ratios) are provided to ensure identical data splits for replication.
   - **Stop-Word List:** The "Commitment Variance Analysis" now explicitly references the **NLTK English Stop-Word List** and a predefined punctuation set, ensuring that different researchers do not use different filters.
   - **Precision & Memory:** Explicitly states **BF16** inference with FP32 logit conversion for precision, and provides a **CPU memory footprint estimate** (~189 MB total) to complete the feasibility analysis.
2. **Validity & Robustness (Addressing Review 2 & 3):**
   - **Increased Sample Size:** WikiText windows increased to 500 and infilling problems to 100 to improve statistical power for ANOVA.
   - **Varying Mask Ratios:** Infilling evaluation now includes 20%, 50%, and 80% mask ratios, addressing the "single point of failure" concern and testing schedule robustness to information density. **Crucially**, the gold prefix/suffix are kept constant across ratios to isolate the schedule effect.
   - **Controlled TCC Variance:** The analysis of commitment time variance now explicitly controls for trivial tokens (NLTK stop words/punctuation) and mask position, ensuring that the

"sequential vs. global" diagnostic reflects architectural bias rather than token difficulty or position.
    - **Baseline Comparison:** The addition of **AR teacher-forced perplexity** on the same WikiText windows provides a "gold standard" quality ceiling, allowing direct quantification of the "undertraining gap" and testing whether specific schedules mitigate it.
    - **Short-Horizon TCC:** The plan to report TCC both with and without the $K = 2$ stability constraint for $T = 16$ addresses the concern that the constraint might underestimate commitment in short trajectories.
    - **Stability Failure Rate (SFR):** Added SFR metric to directly quantify "wandering" behavior, providing deeper insight into the necessity of the stability constraint and the effectiveness of schedules in reducing oscillation.
    - **Mask Ratio Interaction:** Explicitly analyzes if the optimal schedule shifts with mask ratio, addressing the reviewer's suggestion.
    - **ANOVA Power:** Ensures ANOVA is performed on **aggregated metrics per configuration** (mean across seeds) to avoid inflating degrees of freedom, as recommended by the reviewer.
3. **Feasibility (Addressing Review 4):**
    - **Memory-Safe Hooks:** The design explicitly mandates computing NLL/Gap on-GPU and moving only scalars to CPU, preventing VRAM overflow during trajectory tracking. This ensures the 7B models can run safely within the 96GB constraint.
    - **Standardized Schedules:** By defining SCAS and a clear grid-search objective for the learned schedule, the implementation burden is reduced, ensuring the team can complete the setup in Weeks 1-2 as planned.
    - **MLP Training:** The specification of the grid search approach for the learned schedule avoids the computational cost and complexity of training a differentiable surrogate, ensuring feasibility within the timeline.
    - **Prioritization:** The 7B models are prioritized for GPU execution to mitigate potential VRAM/latency bottlenecks, ensuring the timeline is met.
4. **Alignment with Scientific Method:**
    - The experiment remains strictly inference-only, using released checkpoints.
    - The TCC and CAE metrics are computed exactly as defined in the SCTTA method.
    - The analysis directly tests the research problem's hypotheses regarding family/scale interactions and the quality-efficiency frontier.
    - The addition of the AR baseline and domain consistency check strengthens the validation of the method's insights into undertraining and generalizability.

This design is robust, reproducible, and feasible within the 10-week timeline and single-GPU constraint, providing a rigorous answer to the research problem.

Method	Clarity	Validity	Rigorousness	Innovativeness	Generalizability
rating	5	5	5	4	5

Experiment	Clarity	Validity	Robustness	Feasibility	Reproducibility
rating	5	5	5	5	5

Weakest point per the reviewers (Innovativeness, 4/5). The method is highly innovative in its diagnostic approach. By linking the shape of the TCC to architectural biases (e.g., "wave" vs. "global" commitment), the researchers move beyond simple benchmarking toward a mechanistic understanding of how causal vs. bidirectional priors manifest in diffusion trajectories. To further strengthen the method, the authors should ensure that the "Learned Schedule" MLP is sufficiently constrained to remain an "inference-only" lever, as any significant complexity in the MLP could introduce a new variable that complicates the comparison with simple linear/cosine schedules. Additionally, while the relative confidence gap τ_{rel} is a clever way to handle vocabulary differences,

the authors should explicitly document if τ_{rel} is kept constant across model families or tuned per family, as the baseline confidence distributions of GPT-2 and LLaMA may differ significantly.

Idea 2.3 — P2-M8 (overall 4.9/5)

Method.

Schedule-Conditioned Likelihood Drift (SCLD) Analysis with Token-Role Stratification**

We propose a rigorous, inference-only methodology to characterize the quality-efficiency frontier of diffusion language models (DLMs) by decoupling the **temporal dynamics** of the noise schedule from the **spatial heterogeneity** of token roles. Unlike previous methods that aggregate global metrics (entropy, token flips, stability) across the entire sequence, SCLD posits that the effectiveness of a noise schedule is determined by its alignment with the distinct denoising difficulties of **Contextual** (fixed/gold) versus **Generative** (masked/predicted) tokens.

Our method introduces two key innovations:

1. **Token-Role Stratification:** We separate the analysis of the denoising trajectory into two distinct streams:
   - **Contextual Integrity (CI):** How well the model preserves the likelihood of the *fixed* tokens (prefix/suffix in infilling, prompt in completion) as the noise level β_t changes. This measures the model's ability to "condition" on context without being disrupted by the noise schedule.
   - **Generative Resolution (GR):** How effectively the model reduces the uncertainty of the *masked* tokens. This measures the "denoising" capability.
2. **Schedule-Conditioned Likelihood Drift (SCLD) Metric:** A compute-normalized efficiency metric that quantifies the "cost" of maintaining Contextual Integrity while achieving Generative Resolution. It penalizes schedules that cause high likelihood drift in the context (indicating poor conditioning or architectural mismatch) relative to the gain in generative quality.

1. Core Mechanism: Token-Role Stratified Likelihood Tracking

Standard DLM inference treats all tokens in the sequence uniformly in terms of forward pass computation, but their *roles* are fundamentally different. In infilling, the prefix and suffix are "ground truth" context, while the middle is the target. In open-ended generation, the prompt is context, and the rest is target.

Step-wise Analysis Protocol: For each denoising step t (from T down to 1) for a given model m, schedule s, and seed k:

1. **State Definition:** Let x_t be the sequence at step t. Let $\mathcal{C}$ be the set of indices for Contextual tokens (fixed) and $\mathcal{G}$ be the set of indices for Generative tokens (masked/variable).
   - *Clarification on Contextual Tokens:* In all tasks (Infilling, Open-Ended, IFEval), the Contextual tokens are **strictly fixed** (gold) throughout the entire denoising process. They are initialized to their true values and are never masked or updated. The "drift" measured is not a change in the token identity, but a change in the model's *confidence* (log-probability) in those fixed tokens given the evolving noisy state of the generative tokens. This reflects how well the model "reconciles" the fixed context with the current hypothesis of the generation.
2. **Likelihood Calculation:**
   - Perform a single forward pass of the DLM to obtain logits $L_t \in \mathbb{R}^{L \times V}$.
   - **Logit Extraction & Masking:** The logits used for metric calculation are the raw output logits from the final linear head of the transformer, **before** the softmax operation. Crucially, to ensure the model is not "cheating" by attending to the target tokens during the *metric calculation* phase (though it must attend to them during the *sampling* phase to update them), we do **not** apply any additional masking to the logits themselves. The model's attention mask (causal or bidirectional as defined by the specific DLM implementation, e.g., `src_mask` in DiffuLLaMA) determines which tokens it "sees." The metric simply reads

the resulting confidence scores. Since the contextual tokens are fixed inputs, their log-probability reflects the model's internal state consistency.

- **Probability Derivation:** The probabilities $p_\theta(v|x_t)$ are derived by applying a standard softmax to the raw logits L_t over the vocabulary V. No temperature scaling is applied during the metric calculation to ensure consistency with the model's native training objective; however, we will report sensitivity to temperature $T = 1.0$ vs $T = 0.7$ in the appendix to verify robustness.
- **Contextual Log-Likelihood (LLC_t):** Compute the average log-probability of the *true* tokens at positions $\mathcal{C}$:

$$LLC_t = \frac{1}{|\mathcal{C}|} \sum_{i \in \mathcal{C}} \log p_\theta(x_i^{true}|x_t, \theta)$$

Note: Since x_i^{true} is fixed, this measures how "confident" the model is in the context given the current noisy state of the rest of the sequence. A well-matched schedule should keep LLC_t stable or increasing (as the model becomes more coherent).

- **Generative Entropy (H_t):** Compute the average entropy of the predicted distribution at positions $\mathcal{G}$:

$$H_t = \frac{1}{|\mathcal{G}|} \sum_{j \in \mathcal{G}} H(p_\theta(\cdot|x_t, \theta)_j)$$

Note: This measures the uncertainty in the generation. A good schedule should reduce H_t monotonically.

3. **Drift Calculation:**
   - **Contextual Drift (ΔLLC_t):** The change in Contextual Log-Likelihood from step $t-1$ to t:

$$\Delta LLC_t = LLC_t - LLC_{t-1}$$

Positive drift indicates the context is becoming "more consistent" with the evolving sequence. Negative drift indicates the noise schedule is disrupting the model's understanding of the context.

   - **Resolution Gain (ΔH_t):** The reduction in Generative Entropy:

$$\Delta H_t = H_{t-1} - H_t$$

Positive gain indicates successful denoising.

2. Schedule-Conditioned Likelihood Drift (SCLD) Metric

To address the non-compute-normalized efficiency limitation and the architectural generality question, we define **SCLD** as a ratio of "useful work" to "disruptive side effects."

Definition:

$$SCLD = \frac{\sum_{t=1}^{T} \max(0, \Delta H_t)}{\sum_{t=1}^{T} \max(0, -\Delta LLC_t) + \epsilon}$$

- **Numerator (Total Resolution):** The cumulative entropy reduction in the generative tokens. This represents the "quality gain."
- **Denominator (Total Contextual Disruption):** The cumulative *negative* drift in contextual likelihood. This represents the "cost" or "instability" introduced by the schedule. If the context likelihood drops, it implies the model is struggling to reconcile the fixed context with the noisy generation, which is a sign of poor schedule-model alignment.
- **Handling ϵ and Stability:** To prevent extreme spikes in the ratio when the denominator is near zero (i.e., exceptionally stable schedules), we define $\epsilon = 10^{-6}$. Furthermore, to ensure robustness in statistical analysis, we report the **Log-SCLD** ($\log(SCLD)$) for ANOVA tests, which mitigates the non-normality of the ratio distribution. We also provide a "Smoothed SCLD" variant where the denominator is replaced by $\sqrt{\sum_{t=1}^{T} (\max(0, -\Delta LLC_t))^2 + \epsilon}$ (Root Mean Square of disruption) to penalize catastrophic single-step failures more heavily than many small drifts. **The primary reporting metric for all comparisons will be the Smoothed SCLD** to ensure consistency and reduce sensitivity to infinitesimal disruption values.

- **Interpretation:**
  - **High SCLD:** The schedule efficiently reduces uncertainty in the generation without disturbing the model's confidence in the context. This suggests a good match between the noise dynamics and the model's attention mechanism.
  - **Low SCLD:** The schedule either fails to reduce uncertainty (low numerator) or causes significant disruption to the context (high denominator). This suggests a mismatch (e.g., a schedule that is too aggressive for a causal model, causing it to "forget" the prompt).
- **Behavior with Positive Drift:** If ΔLLC_t is consistently positive (model becomes *more* confident in context), the denominator approaches ϵ. To prevent score inflation in this "ideal" case, we introduce a **Stability Saturation Cap**. If the Total Contextual Disruption (denominator) is below a threshold τ_{stable} (defined as the 5th percentile of disruption values across all runs for that model family), we define the "Disruption Cost" as τ_{stable} instead of the raw value. This ensures that "perfect" stability yields a high but bounded score, while "bad" stability yields a low score.

Compute Normalization: Since the forward pass cost is constant per step in the dense implementation, we normalize SCLD by the number of steps T to get **SCLD per Step**:

$$SCLD_{norm} = \frac{SCLD}{T}$$

This allows fair comparison across different step budgets (16, 32, 64). A schedule with high $SCLD_{norm}$ achieves high quality with minimal disruption per unit of compute. *Note on Overhead:* For the Context-Adaptive schedule, the entropy calculation required to determine β_t is performed on the logits from the *previous* step. This computation is negligible ($O(L \cdot V)$) compared to the transformer forward pass ($O(L^2 \cdot d + L \cdot d^2)$) and is included in the wall-clock latency but does not significantly alter the FLOPs approximation used for normalization, as it does not involve additional matrix multiplications with the model weights.

Baseline Comparison for Innovativeness: To demonstrate that SCLD identifies "bad" schedules that global metrics miss, we will compute a **Naive Efficiency Metric (NEM)** defined as $\frac{\sum \max(0, \Delta H_t)}{T}$ (Total Resolution / Steps). We will correlate SCLD with NEM. We hypothesize that while NEM may rank schedules similarly for high-performing models, SCLD will diverge for undertrained models or mismatched schedules, correctly identifying those that achieve resolution via "contextual chaos" (high NEM, low SCLD). Additionally, we will compare SCLD against a **Fixed-Step Linear Baseline** to see if SCLD consistently identifies the "Pareto optimal" schedule relative to this standard.

3. Experimental Protocol

Models:

- **GPT-2 Family:** DiffuGPT-S, DiffuGPT-M.
- **LLaMA Family:** DiffuLLaMA-6.74B, Dream-7B, LLaDA-8B, DiffuCoder-7B.

Schedules:

1. **Linear:** $\beta_t = 1 - t/T$.
2. **Cosine:** $\beta_t = \cos(\frac{\pi}{2}\frac{t}{T})$.
3. **Context-Adaptive:** As implemented in Dream 7B (entropy-conditioned).
4. **Learned:** A lightweight, external schedule predictor.
   - *Implementation Details:* The "Learned" schedule is a small Multi-Layer Perceptron (MLP) with 2 hidden layers (64 units, ReLU activation) that predicts the scalar noise level β_t for the current step.
   - *Input Features:* The MLP takes as input: (1) the normalized timestep t/T, (2) the current average entropy of the generative tokens H_t (computed from the previous step's logits), and (3) the current average log-likelihood of the contextual tokens LLC_t.
   - *Standardization Statistics:* To address the scale difference between 127M and 7B models, the standard-normalization statistics (mean and variance) for H_t and LLC_t are computed

per-model on the calibration set. When testing "Transferred" schedules (e.g., applying the GPT-2-S MLP to GPT-2-M), we **re-compute** the normalization statistics for the target model but keep the MLP weights fixed. This isolates the schedule logic from the logit scale shift.

- *Training Protocol:* The MLP is trained **externally** (not integrated into the DLM weights) on a small, held-out validation set.
  - *Data Split:* We sample 100 sequences stratified by task type (33 Infilling, 33 Open-Ended, 34 IFEval) from the adaptation corpus. These sequences are **strictly disjoint** from the final evaluation sets (HumanEval/MBPP/C4/IFEval test splits) to prevent data leakage.
  - *Meta-Training Cost:* Generating the training data for the MLP requires running the DLM inference on these 100 sequences for 32 steps. This is a one-time cost of 100×32 forward passes, which is negligible compared to the main experimental runs.
  - *Loss Function:* The loss function is negative SCLD (to maximize SCLD).
  - *Generalization Test:* To test generalizability, we will train one MLP per model family (GPT-2 and LLaMA) using the small models (DiffuGPT-S/M and DiffuLLaMA-6.74B) and apply the family-level MLP to the larger models (e.g., LLaDA-8B). We will compare this "Transferred" schedule against a "Native" schedule trained specifically for the larger model to quantify the cost of transfer.
- *Heuristic Baseline:* To validate the necessity of the MLP, we will also implement a **Heuristic Adaptive Schedule** where β_t is decreased if H_t falls below a fixed threshold (e.g., 50th percentile of the initial entropy). This serves as a simple baseline to demonstrate that the MLP's learned capacity provides a significant improvement over simple thresholding.

Tasks & Data:

- **Infilling:** Multi-line HumanEval, MBPP. (Context = prefix/suffix, Generation = middle).
- **Open-Ended:** C4 prompts. (Context = prompt, Generation = completion).
- **Instruction Following:** IFEval. (Context = instruction, Generation = response).
- **Sensitivity Analysis:** We will also run a subset of experiments with varying **Context-to-Generation Ratios** (e.g., 25% context/75% gen vs. 75% context/25% gen) to ensure the SCLD metric remains stable and informative across different masking scenarios.

Steps: 16, 32, 64. **Seeds:** 5 independent sampling seeds per configuration.

Metrics:

- **Quality:**
  - Perplexity (WikiText-103).
  - Pass@1/Pass@10 (HumanEval/MBPP).
  - MAUVE (Open-ended).
  - IFEval Score.
- **Efficiency/Diagnostics:**
  - **Smoothed SCLD** and **SCLD_norm** (and Log-SCLD for stats).
  - **Naive Efficiency Metric (NEM)** for baseline comparison.
  - **Contextual Stability Index (CSI):** $1 - \frac{\text{Std}(\Delta LLC_t)}{\text{Mean}(\Delta LLC_t)+\epsilon}$. Measures the consistency of context preservation. **CSI will be reported as a primary diagnostic alongside SCLD** to ensure that high SCLD is driven by high resolution and not just an infinitesimally small disruption denominator.
  - **Boundary Drift Analysis:** We will specifically calculate ΔLLC_t for the tokens in $\mathcal{C}$ that are **immediately adjacent** (the 3 tokens on each side) to the $\mathcal{G}$ boundary. This precise definition allows us to test the hypothesis that drift occurs primarily at the context/generation interface without dilution from the global average.
  - **Raw ΔLLC_t Trajectories:** We will explicitly plot and report the raw drift trajectories alongside the SCLD score to verify that high scores are not artifacts of near-zero denominators in failing models.
  - **Wall-Clock Latency.**

Analysis:

1. **Frontier Mapping:** Plot Quality (e.g., Pass@1) vs. $SCLD_{norm}$ for each model. Identify if high SCLD correlates with high quality.
2. **Family Interaction:** Perform a two-way ANOVA on **Log-SCLD** with factors *Model Family* (GPT-2 vs. LLaMA) and *Schedule Type*.
   - *Hypothesis:* LLaMA-based models (bidirectional attention) will exhibit higher SCLD with Context-Adaptive schedules because they can better integrate bidirectional context, leading to less contextual disruption. GPT-2-based models may show lower SCLD with aggressive schedules due to causal bias causing "context drift."
3. **Scale Interaction:** Test if SCLD scales with model size. Do larger models exhibit more "stable" contextual likelihood (lower disruption) across all schedules?
4. **Schedule Optimization:** Use the SCLD metric to select the "best" schedule for each model. Then, test if this schedule generalizes to other models in the same family.
5. **SCLD vs. NEM Divergence:** Analyze cases where SCLD and NEM rankings differ. These cases represent schedules that are "efficient" in denoising speed but "inefficient" in conditioning stability, highlighting the unique value of SCLD.
6. **Boundary vs. Global Drift:** Compare the drift magnitude at the context/generation boundary vs. the average drift across the entire context. A significant difference would support the hypothesis that interface alignment is the primary driver of schedule effectiveness.
7. **Shuffled Context Control:** To further validate the "disruption" metric, we will run a control experiment where the context tokens are randomly shuffled (breaking structural conditioning). We expect SCLD to drop significantly in these runs, confirming that the metric is sensitive to the *quality* of the context-generation alignment, not just the magnitude of entropy reduction.

4. Feasibility and Rigor

Inference-Only:

- All metrics (LLC_t, H_t, ΔLLC_t, ΔH_t) are computed from the logits obtained during the standard inference forward pass. No additional forward passes are required for the metrics themselves.
- The "Learned" schedule requires training a small MLP. This is a lightweight CPU/GPU task (seconds to minutes) on a small dataset (100 sequences). It does **not** modify the DLM weights, adhering to the "inference-only" constraint for the main model. The meta-training cost is explicitly accounted for as a one-time overhead.

Compute:

- **Runs:** 6 models × 4 schedules × 3 steps × 5 seeds = 360 inference runs.
- **Overhead:** Computing LLC_t and H_t involves gathering log-probabilities and calculating entropy, which are negligible compared to the transformer forward pass.
- **GPU:** A single 96GB GPU can handle batch size 1 for 7B models comfortably. The 360 runs are feasible within the 10-week timeline, especially with parallelization across the 3 GPU-enabled members.

Statistical Rigor:

- 5 seeds per configuration allow for mean ± std reporting.
- We use **Log-SCLD** for ANOVA to ensure normality assumptions are met, and we report bootstrap confidence intervals for the raw SCLD ratio to account for its skewed distribution.
- We explicitly test the correlation between SCLD and NEM to validate the added value of the disruption metric.
- We report **CSI** as a primary diagnostic to guard against denominator sensitivity issues.
- The **Stability Saturation Cap** ensures that the metric behaves well in the "perfect stability" regime, preventing inflation.

5. Innovation

1. **Context-Generation Decoupling:** Moves beyond global sequence metrics to explicitly analyze the *interaction* between the fixed context and the noisy generation. This directly addresses

the "infilling" and "instruction-following" aspects of the research problem, which are critical for practical DLM deployment.
2. **Disruption-Aware Efficiency:** Introduces "Contextual Disruption" as a cost metric. This provides a mechanistic explanation for why certain schedules fail: they don't just denoise slowly, they actively *destabilize* the model's conditioning on the prompt/context.
3. **Architectural Insight:** By comparing SCLD across GPT-2 and LLaMA families, we can directly test the hypothesis that bidirectional attention (LLaMA) is more robust to noise schedules that might disrupt causal context (GPT-2). The **Boundary Drift Analysis** further refines this by localizing the disruption to the interface, providing a concrete, quantifiable answer to the "architectural generality" question.
4. **Actionable Schedule Selection:** SCLD provides a single, interpretable scalar metric that can be used to automatically select the best schedule for a new DLM checkpoint without running full quality benchmarks, accelerating deployment.
5. **Rigorous Learned Schedule:** The specific definition of the external MLP (inputs: t/T, H_t, LLC_t; loss: -SCLD; per-model standardization; family-level transfer testing; heuristic baseline) ensures the "learned" schedule is a reproducible, lightweight, and truly inference-only adaptation, distinct from fine-tuning. The explicit separation of training/evaluation data prevents leakage, and the comparison against a heuristic baseline validates the necessity of the learned component.

Why this method. The proposed method, **Schedule-Conditioned Likelihood Drift (SCLD) Analysis with Token-Role Stratification**, is substantively different from the seven previously proposed methods in the following ways:

1. **Focus on Context-Generation Interaction vs. Global Dynamics:**
   - *Previous Methods:* SAPD, TSNSC, CSI, ASP, CST-PDM, NCS, and CPD all focus on global properties of the denoising trajectory (entropy flow, token flips, stability, phase decomposition, or schedule interpolation). They treat the sequence as a monolithic object.
   - *SCLD:* Explicitly stratifies the sequence into **Contextual** and **Generative** tokens. It measures the *drift* in the likelihood of the context as a function of the noise schedule. This is a unique angle that directly addresses the practical reality of DLMs being used for infilling and instruction following, where preserving the context is as important as generating the target.
2. **Metric Definition: Disruption as a Cost:**
   - *Previous Methods:* Metrics like EDS (SAPD, TSNSC), EIY (CSI), DFS (ASP), IYF (CST-PDM), SAEF (NCS), and PWY (CPD) focus on "gain" or "yield" per unit of compute. They penalize inefficiency by measuring low gain.
   - *SCLD:* Introduces **Contextual Disruption** ($\max(0, -\Delta LLC_t)$) as a *denominator* (cost). It penalizes schedules that *actively harm* the model's conditioning. This is a fundamentally different efficiency perspective: not just "how much did you gain?" but "how much did you break?" This is crucial for understanding why undertrained models might fail with certain schedules.
3. **Mechanistic Insight into Architectural Bias:**
   - *Previous Methods:* While CPD and ASP touch on architectural differences (causal vs. bidirectional), they do so through phase weights or directional sensitivity.
   - *SCLD:* Directly links the **Contextual Drift** to the attention mechanism. A causal model (GPT-2) might exhibit high negative drift if the noise schedule prevents it from "seeing" the future context effectively, whereas a bidirectional model (LLaMA) might not. SCLD quantifies this *mismatch* directly. The addition of **Boundary Drift Analysis** further refines this insight by testing if the disruption is localized to the interface between context and generation, providing a more granular mechanistic explanation.
4. **Inference-Only Rigor and Specificity:**
   - All metrics are derived from the single forward pass logits, adhering strictly to the "inference-only" constraint. The method does not require multiple seeds per step for the core metric calculation (unlike NCS's SSI), making it more computationally efficient while still allowing for statistical rigor via seed aggregation. The specific definition of the "Learned" schedule as an external MLP trained on SCLD, with standardized inputs,

family-level transfer testing, and a heuristic baseline, ensures reproducibility and strict adherence to the no-weight-update constraint.

5. **Generalizability and Practicality:**
   - SCLD provides a single, interpretable score that can be used for **automatic schedule selection**. This is a practical tool for practitioners who need to deploy DLMs without extensive manual tuning, directly addressing the "practitioner knob" significance mentioned in the research problem. The inclusion of a Naive Efficiency Metric (NEM) baseline and the Contextual Stability Index (CSI) diagnostic further strengthen the method by quantifying the specific added value of considering contextual stability and guarding against metric sensitivity. The cross-task normalization via standardized MLP inputs ensures the learned schedule is comparable across different prompt lengths and complexities. The addition of a **Shuffled Context Control** and **Context-to-Generation Ratio Sensitivity Analysis** further solidifies the method's robustness and generalizability across different masking scenarios and tokenizers.

In summary, SCLD offers a fresh perspective by focusing on the *stability of conditioning* rather than just the *speed of denoising*, providing a deeper, more practical understanding of the quality-efficiency frontier for adapted DLMs.

Experiment design.

1. Pre-Experiment Verification & Adapter Development (Weeks 1–2)** Before executing the main grid, the team must resolve the critical incompatibility between the conceptual "continuous noise schedule" and the discrete token ID implementation of DiffuLLaMA/DiffuGPT (Verified Fact 1).

- **Schedule Adapter Implementation:** Implement a `ScheduleAdapter` class that maps the scalar noise level β_t (ranging from 0 to 1, where 1 is fully noisy/masked and 0 is clean/unmasked) to the discrete masking operations required by the models.
  - **Mapping Logic:** The adapter determines the target number of masked tokens M_t at step t using deterministic rounding: $M_t = \lceil \beta_t \times L_{gen} \rceil$, where L_{gen} is the number of generative positions.
  - **Token Selection Mechanism (Confidence-Based):** To ensure reproducibility and test the schedule's impact on *which* tokens are resolved, we use **Confidence-Based Unmasking**. At each step t, the model predicts logits for all positions. The tokens in the current masked set $\mathcal{G}_{t-1}$ are ranked by their predicted probability (or log-likelihood). The top $M_{t-1} - M_t$ highest-confidence tokens are unmasked (fixed for subsequent steps).
    - **Tie-Breaking Rule:** To ensure absolute bit-level reproducibility across different hardware and seeds, if multiple tokens have identical predicted probabilities (within floating-point precision $\approx 10^{-7}$), the tie is broken by selecting the token with the **lowest sequence index** first. This deterministic rule prevents stochastic drift in the trajectory.
    - **Re-noising Handling:** If $M_t > M_{t-1}$ (re-noising, only relevant if the schedule is non-monotonic, which our proposed schedules are not, but the adapter must handle it), the lowest-confidence unmasked tokens are re-masked.
  - **Contextual Integrity:** The indices $\mathcal{C}$ (Contextual tokens) are strictly fixed in `src_mask` and `maskable_mask` as "unmasked/known" from $t = 0$ to $t = T$. They are never candidates for masking.
  - **Random Unmasking Baseline:** To isolate the effect of the schedule from the model's internal confidence bias, we will run a subset of the main grid (specifically the Linear and Cosine schedules on DiffuLLaMA-6.74B and DiffuGPT-M) using **Random Unmasking** (selecting $M_{t-1} - M_t$ tokens uniformly at random from $\mathcal{G}_{t-1}$). This allows us to distinguish whether SCLD improvements are driven by the *timing* of denoising (schedule) or the *selection* of tokens (confidence).
- **Logit Extraction & Memory Management Hook:** Modify the inference loop to capture the raw logits $L_t \in \mathbb{R}^{L \times V}$ at every step t.
  - **Immediate Reduction Strategy:** To prevent VRAM exhaustion (Verified Fact 2/Feasibility), we do **not** store the full $L \times V$ tensor in GPU memory across steps. Instead, immedi-

ately after the forward pass at step t:

1. Compute LLC_t and H_t on the GPU **in FP32 precision** to ensure numerical stability during accumulation and entropy calculation, even if the model weights are in BF16/FP16.
2. Move only the scalar values LLC_t, H_t and the index array of unmasked tokens to CPU.
3. **Raw Logit Storage:** If raw logits are needed for post-hoc analysis (e.g., Boundary Drift verification, temperature sensitivity), save them to a compressed `.pt` file on local NVMe SSD in a streaming manner.
   * **Precision & Format:** Logits are saved in **BF16** format to balance storage size and numerical fidelity (matching the model's internal precision). Files are compressed using `torch.save` with `pickle_protocol=4`.
   * **Disk Footprint Calculation:** For a 128-token sequence and 32K vocabulary, one forward pass is $128 \times 32000 \times 2$ bytes ≈ 8 MB. For 64 steps, one run is ≈ 512 MB. With 360 runs, the total footprint is ≈ 184 GB. This is feasible on a standard 500GB NVMe SSD partition dedicated to this experiment.
4. Clear the GPU tensor.

- **VRAM Monitoring:** Implement a `torch.cuda.memory_stats()` monitor that logs peak memory usage per step. If the `ScheduleAdapter`'s ranking process (sorting $L \times V$ logits) causes spikes exceeding 90% of available VRAM on 7B models, the sorting will be chunked over sequence positions (e.g., sort in batches of 32 tokens) to mitigate spikes.
- **Checkpointing:** Implement a job queue system (e.g., `slurm` or `docker` with restart policies) that saves the current step index, RNG state, and adapter state to disk every 5 steps. If the GPU is preempted, the job resumes from the last checkpoint, ensuring no data loss in the 360+ run grid.

2. Main Experimental Grid (Weeks 3–8) Execute the following configurations on a single NVIDIA RTX 6000 Pro Blackwell (96 GB).

- **Models (6) & Checkpoint Versions:**
  - GPT-2 Family: DiffuGPT-S (127M), DiffuGPT-M (355M).
  - LLaMA Family: DiffuLLaMA-6.74B, Dream-7B, LLaDA-8B, DiffuCoder-7B.
  - *Reproducibility Note:* The specific HuggingFace Hub commit hashes for all checkpoints will be pinned in the configuration file (e.g., `diffusionfamily/diffullama:commit_abc123`) to ensure weight initialization is identical across replications.
- **Schedules (5):**
  1. **Linear:** $\beta_t = 1 - t/T$.
  2. **Cosine:** $\beta_t = \cos(\frac{\pi}{2}\frac{t}{T})$.
  3. **Context-Adaptive (Dream):** Replicate Dream 7B's logic. β_{t+1} is adjusted based on the entropy H_t of the *current* masked tokens. Specifically, if H_t is high, β_{t+1} is increased (slower denoising); if low, β_{t+1} is decreased (faster denoising).
  4. **Random Schedule (Control):** β_t is sampled uniformly from $[0, 1]$ at each step, subject to the constraint that $\beta_T = 1$ and $\beta_1 = 0$. **Fixed Seeds:** The random seed for the schedule generation is fixed to **`seed=42`** across all models. The trajectory is generated **once** per (Model, Step) pair and shared across the 5 sampling seeds to ensure each model responds to the exact same "chaotic" trajectory, isolating the model's architectural response.
  5. **Learned (MLP):**
     - *Phase A (Data Generation):* Run inference on 100 held-out sequences using **three** static schedules (Linear, Cosine, Random) for 32 steps.
       * *Data Source & Disjointness:* The 100 sequences are sampled from the **RedPajama** dataset (v1.0), specifically the `code` and `web` subsets. To ensure strict disjointness from evaluation benchmarks:
         · Infilling samples: Randomly select 33 code snippets from RedPajama `code` subset that are **not** present in the HumanEval/MBPP test sets (verified via AST-based deduplication).
         · Open-Ended samples: Randomly select 33 text segments from RedPajama `web` subset that are **not** in the C4 test split.
         · IFEval samples: Randomly select 34 instruction-response pairs from RedPajama

web subset that are **not** in the IFEval test split.
    - *Sampling Seed:* The stratified sampling of these 100 sequences uses **seed=123** to allow exact replication of the MLP training data.
  - *Phase B (MLP Training):* Train a small MLP (2 hidden layers, 64 units, ReLU) to predict the optimal β_t.
    - *Inputs:* Normalized t/T, H_t, LLC_t.
    - *Target:* The β_t value that maximized the step-wise SCLD proxy $\frac{\Delta H_t}{\max(0, -\Delta LLC_t) + \epsilon}$ in the Phase A data.
    - *Target Smoothing:* To prevent overfitting to noise, the target β_t^* is smoothed using a **moving average window of 3 steps** before regression.
    - *Loss Function:* **Huber Loss** (delta=1.0) between predicted β_t and smoothed target β_t^*, to robustly handle outliers in the SCLD proxy.
  - *Phase C (Inference):* Use the trained MLP to dynamically set β_t during the main runs.
  - *Variance Monitoring:* During the main grid, we will log the variance of the MLP's predicted β_t across the 5 seeds for each step. If the variance exceeds a threshold (indicating high sensitivity to specific sequence content), we will report the mean β_t trajectory and discuss the generalizability limits.
- **Parameters:**
  - **Steps (T):** 16, 32, 64.
  - **Seeds:** 5 independent random seeds for sampling (e.g., 42, 123, 456, 789, 1011).
  - **Sequence Length:** 128 tokens.
  - **Batch Size:** 1 (to ensure consistent latency and memory usage).
- **Tasks (3):**
  1. **Infilling:** Multi-line HumanEval and MBPP. Context = Prefix/Suffix (Fixed), Generation = Middle.
  2. **Open-Ended:** C4 Prompts. Context = Prompt (Fixed), Generation = Completion.
  3. **Instruction Following:** IFEval. Context = Instruction (Fixed), Generation = Response.

3. Metric Computation (CPU-Only, Parallel with GPU) For every inference run, compute the following from the captured/reduced logits:

- **Token Role Stratification:**
  - Identify indices $\mathcal{C}$ (Contextual) and $\mathcal{G}$ (Generative) for each step.
  - **Contextual Log-Likelihood (LLC_t):** $\frac{1}{|\mathcal{C}|} \sum_{i \in \mathcal{C}} \log p_\theta(x_i^{true} | x_t)$.
  - **Generative Entropy (H_t):** $\frac{1}{|\mathcal{G}|} \sum_{j \in \mathcal{G}} H(p_\theta(\cdot | x_t)_j)$.
- **SCLD Metrics:**
  - Calculate ΔLLC_t and ΔH_t.
  - Compute **Smoothed SCLD**: $\frac{\sum \max(0, \Delta H_t)}{\sqrt{\sum (\max(0, -\Delta LLC_t))^2} + \epsilon}$.
  - Compute **SCLD_norm**: Smoothed SCLD / T.
  - Compute **Log-SCLD** for statistical testing.
  - Compute **NEM**: $\frac{\sum \max(0, \Delta H_t)}{T}$.
  - Compute **CSI**: $1 - \frac{\text{Std}(\Delta LLC_t)}{\text{Mean}(\Delta LLC_t) + \epsilon}$.
  - **Boundary Drift:** Calculate ΔLLC_t specifically for tokens in $\mathcal{C}$ adjacent to $\mathcal{G}$. We will test windows of **3, 5, and 10** tokens.
    - **Boundary-to-Global Ratio:** Compute $\frac{\text{Mean Boundary Drift}}{\text{Mean Global Drift}}$ to normalize for model scale and sequence length, enabling direct comparison between 127M and 7B models.
    - **Overlap Check:** For sequences where $|\mathcal{C}| < 2 \times$ window size, the boundary window is truncated to the available context tokens, and the ratio is adjusted accordingly.
- **Quality Metrics:**
  - Perplexity (WikiText-103 subset).
  - Pass@1/Pass@10 (HumanEval/MBPP).
  - MAUVE (Open-Ended).
  - IFEval Score.
- **Efficiency Metrics:**

  - **Wall-Clock Latency:** Measure time per token.
  - **Approximated FLOPs:** $2 \times P \times L \times T$. *Note: Since the dense forward pass is performed on the entire sequence regardless of mask state (Verified Fact 2), the "effective token count" does not reduce FLOPs in this implementation. The FLOPs are constant per step. Therefore, efficiency is primarily driven by the number of steps T required to reach a quality threshold, or the latency.*

4. Control & Sensitivity Experiments (Week 8)

- **Shuffled Context:** For a subset of runs (e.g., Linear schedule, 32 steps, 1 seed), shuffle the Contextual tokens $\mathcal{C}$ before inference. Expect SCLD to drop significantly, validating the metric's sensitivity to context quality.
- **Heuristic Baseline:** Implement a simple threshold-based adaptive schedule (reduce β_t if $H_t < 50\%$ of initial) to compare against the Learned MLP.
- **MLP Cross-Task Validation:**
  - *Training:* Train the MLP using only **Infilling** data from the calibration set.
  - *Validation:* Test its performance on **IFEval** and **Open-Ended** tasks (unseen task distributions for the MLP).
  - *Degradation Analysis:* Explicitly report the percentage drop in SCLD_norm when transferring from Infilling-trained to IFEval-tested, compared to Open-Ended-tested. We hypothesize that IFEval (requiring strict structural adherence) will show higher degradation, indicating task-specificity. This serves as a "Cross-Task Validation" to ensure the learned β_t logic is not overfitting to the specific prompt distributions of the training slice.
- **Random Unmasking Comparison:** Compare SCLD scores for Confidence-Based vs. Random Unmasking on the Linear/Cosine schedules. If Confidence-Based yields significantly higher SCLD, it suggests the model's internal confidence is a crucial component of the "schedule" effectiveness, not just the timing.

5. Statistical Analysis (Weeks 9–10)

- **ANOVA:** Perform two-way ANOVA on **Log-SCLD** with factors *Model Family* (GPT-2 vs. LLaMA) and *Schedule Type*. Test for interaction effects.
- **Correlation:** Compute Pearson correlation between SCLD_norm and NEM. Identify divergences where SCLD is low but NEM is high (indicating "contextual chaos").
- **Frontier Mapping:** Plot Quality (e.g., Pass@1) vs. SCLD_norm for each model. Calculate AUC for each schedule.
- **Boundary Analysis:** Compare Boundary Drift (3, 5, 10 tokens) vs. Global Drift. Use paired t-tests to determine if drift is significantly higher at the interface. Report the Boundary-to-Global Ratio to assess interface alignment across scales.

Why this design.

1. **Addressing Implementation Reality (Verified Facts 1, 2, & 5):**
   - **Discrete Mapping & Tie-Breaking:** The `ScheduleAdapter` explicitly defines how continuous β_t maps to discrete token masking via deterministic rounding and confidence-based unmasking. The addition of a **deterministic tie-breaking rule** (lowest index) ensures absolute bit-level reproducibility, addressing the Clarity feedback.
   - **FLOPs Honesty:** Verified Fact 2 confirms that freezing tokens does not skip matrix operations. Therefore, the experiment correctly rejects the "effective token count" FLOP reduction claim. Instead, it defines efficiency via **Wall-Clock Latency** and **Steps-to-Quality**, while using SCLD as a *diagnostic* for *why* a schedule is efficient (low disruption) rather than a direct measure of FLOPs saved.
   - **Compute Accounting:** Verified Fact 5 reminds us to include all forward calls. The "Learned" schedule's entropy calculation is $O(L \cdot V)$, negligible compared to the transformer forward pass $O(L^2 d + L d^2)$. We explicitly state this to justify that the FLOPs approximation $2PLT$ remains valid for comparison.
2. **Enhanced Validity & Robustness:**
   - **Random Unmasking Baseline:** Adding this baseline addresses the Validity feedback regarding "Confidence-Based Unmasking Bias." It isolates the effect of the *schedule tra-*

jectory from the *token selection strategy*, ensuring that SCLD improvements are attributed to the correct factor.
    - **MLP Target Stability:** The use of **Huber Loss** and **Moving Average Smoothing** for the MLP target addresses the Robustness and Validity feedback regarding target volatility. This prevents the MLP from overfitting to noise in the small Phase A dataset.
    - **Boundary Window & Ratio:** The addition of the **Boundary-to-Global Ratio** addresses the Robustness feedback, allowing for direct comparison across different model scales (127M vs. 7B) by normalizing out the magnitude of drift. The overlap check ensures the metric is well-defined for short contexts.
    - **Fixed Random Schedule Seeds:** Fixing the random seeds for the "Random Schedule" across models (Reproducibility feedback) ensures that differences in performance are due to the model's architecture, not different chaotic trajectories. The specific seed (42) and "generate once, share across sampling seeds" logic are now explicitly defined.
    - **Cross-Task Validation:** The addition of a specific **MLP Cross-Task Validation** experiment (train on Infilling, test on IFEval/Open-Ended) addresses the Robustness feedback regarding overfitting to specific prompt distributions. This tests the generalizability of the learned schedule logic beyond its training domain.
3. **Feasibility & Resource Allocation:**
    - **VRAM Management & Precision:** The plan to chunk the sorting of $L \times V$ logits is a prudent addition. Furthermore, explicitly specifying **FP32 precision for SCLD calculations** (Robustness feedback) eliminates ambiguity regarding numerical stability, while keeping storage in BF16 to manage disk space.
    - **Storage:** The BF16 streaming storage plan is feasible. The explicit calculation of the SSD footprint (~184 GB) ensures the team can provision the necessary NVMe space.
    - **Timeline:** The 10-week timeline is maintained. Weeks 1-2 are dedicated to the critical Adapter and Memory Hook development. The remaining weeks allow for the parallel execution of the grid by the 3 GPU-enabled members, with CPU-only members handling the metric computation and statistical analysis.
4. **Reproducibility:**
    - **Explicit Sampling Mechanism:** The "Confidence-Based Unmasking" with a defined tie-break rule and the "Deterministic Rounding" are explicitly defined, allowing a third party to replicate the exact trajectory.
    - **MLP Target Definition:** The MLP target is clearly defined with smoothing and Huber loss, removing ambiguity and ensuring stability.
    - **Checkpoint & Data Provenance:**
        - **Checkpoint Versions:** Specific HuggingFace commit hashes will be pinned (Reproducibility feedback).
        - **Data Source:** The "held-out slice" is now explicitly defined as **RedPajama** with `seed=123` for sampling and AST-based deduplication against test sets (Validity/Reproducibility feedback). This allows others to replicate the MLP's training data exactly.
        - **Random Schedule Seed:** The specific seed value (42) is provided (Reproducibility feedback).
    - **Open-Sourcing:** All code for the Adapter, Metric Computation, and MLP training will be open-sourced, ensuring the "inference-only" constraint is strictly adhered to.

This revised experiment directly addresses the reviewer feedback by clarifying the adapter logic (tie-breaking), enhancing memory management (chunking, precision), adding a random unmasking control, refining the MLP training (Huber loss, smoothing, cross-task validation), defining data provenance (RedPajama, seeds), and normalizing boundary analysis for cross-scale comparison. It remains strictly inference-only, feasible on the provided hardware, and rigorously aligned with the SCLD scientific method.

Method	Clarity	Validity	Rigorousness	Innovativeness	Generalizability
rating	5	5	5	4	5

Experiment	Clarity	Validity	Robustness	Feasibility	Reproducibility
rating	5	5	5	5	5

Weakest point per the reviewers (Innovativeness, 4/5). 1. **Metric Sensitivity:** The use of a "Stability Saturation Cap" and "Smoothed SCLD" shows a strong awareness of the mathematical instability of ratios. However, the authors should be cautious about the ϵ value and the 5th percentile threshold (τ_{stable}); these hyperparameters could potentially bias the results if not cross-validated across different model scales.

2. **Boundary Analysis:** The "Boundary Drift Analysis" is a particularly strong addition. To further enhance this, the authors could consider analyzing whether the drift is symmetric (prefix vs. suffix) in infilling tasks, as this would provide even deeper insight into the causal vs. bidirectional attention mismatch.
3. **Computational Cost:** While the method is "inference-only," the requirement for 5 seeds across 360 configurations is substantial. The authors have addressed feasibility, but providing a "fast-track" version of SCLD (e.g., using fewer seeds for initial schedule screening) would increase the method's practical utility for practitioners.

Idea 2.4 — P2-M5 (overall 4.8/5)

Method.

Schedule-Response Surface Mapping (SRSM) via Discrete Perturbation Probes and Compute-Aware Frontier Projection**

We propose a rigorous, inference-only methodology to characterize the quality-efficiency frontier by treating the noise schedule not as a static function to be optimized, but as a **controllable input signal** whose local response can be mapped via discrete perturbations. Unlike previous methods that rely on continuous gradients (which are undefined for discrete token sampling) or global interpolation (which assumes convexity), our method uses **Discrete Perturbation Probes** to estimate the **Local Sensitivity Coefficient (LSC)** at specific timesteps. This allows us to construct a **Response Surface Map** of the denoising trajectory, identifying "high-yield" and "low-yield" regions of the schedule. We then use this map to project a **Compute-Aware Frontier** that respects the dense-forward compute reality of the DiffuLLaMA implementation, and we quantify cross-family generality using a **Schedule Shape Correlation Index (SSCI)** based on the geometry of these response surfaces.

1. Core Mechanism: Discrete Perturbation Probes and Local Sensitivity Coefficient (LSC)

The key innovation is the definition of the **Local Sensitivity Coefficient (LSC)**, which measures how much the final output quality changes when the noise level β_t at a specific timestep t is locally perturbed, while keeping the rest of the schedule fixed. This provides a causal, ground-truth-anchored diagnostic of schedule efficiency that is robust to the discrete nature of token sampling.

- **Base Schedule Selection:** We select a neutral base schedule $S_{base}(t)$ (e.g., Linear) for each model.
- **Perturbation Probe Definition:** For each timestep $t \in \{1, \dots, T\}$, we define a **Perturbation Probe** P_t^{δ} that modifies the base schedule by shifting the noise level at step t by a magnitude δ, while compensating the noise at a reference step t_{ref} (e.g., the final step T or a low-impact early step) to preserve the total "noise budget" $\sum \beta_t$.

$$\beta_{t'} = \begin{cases} \beta_t + \delta & \text{if } t' = t \\ \beta_{t_{ref}} - \delta & \text{if } t' = t_{ref} \\ \beta_{t'} & \text{otherwise} \end{cases}$$

  We use a small, fixed perturbation magnitude $\delta = 0.05$ (in normalized noise units) to ensure we are measuring local sensitivity rather than global regime changes.

- **Local Sensitivity Coefficient (LSC):** We compute the **Local Sensitivity Coefficient** at step t as the change in a ground-truth-anchored quality metric Q when applying the probe P_t^δ:

$$\text{LSC}(t) = \frac{Q(S_{base}) - Q(P_t^\delta)}{\delta}$$

  Where Q is the **Negative Log-Likelihood (NLL)** of the ground-truth tokens on a fixed evaluation set (WikiText-103 validation segments).

  - **Interpretation:**
    - **High Positive LSC:** Increasing noise at step t *decreases* quality (i.e., the model relies on low noise at this step to resolve tokens correctly). This step is "critical" for quality.
    - **Low/Negative LSC:** Increasing noise at step t has little effect or *improves* quality (i.e., the model can tolerate or even benefit from higher noise at this step, perhaps by exploring alternative tokens). This step is "robust" or "exploratory."
  - **Why this is better than Continuous Gradients:** Since token sampling is discrete, the derivative of the final NLL with respect to β_t is not well-defined. The LSC provides a finite-difference approximation that is valid for discrete samplers and directly measurable.

- **Response Surface Map:** For each model and base schedule, we compute LSC(t) for all t. This yields a **Response Surface Map** $R(t)$, a vector of length T that visualizes the "sensitivity profile" of the denoising trajectory.

  - **Peak Sensitivity Step (t^*):** The timestep with the highest LSC, indicating where the schedule most critically affects quality.
  - **Sensitivity Entropy (H_R):** The Shannon entropy of the normalized LSC distribution, indicating how "spread out" the critical steps are. A low H_R indicates a few critical steps; a high H_R indicates sensitivity is distributed across the trajectory.

2. Compute-Aware Frontier Projection

To address the "Non-compute-normalized efficiency reporting" limitation, we use the Response Surface Map to project a **Compute-Aware Frontier** that accounts for the actual cost of denoising steps.

- **Compute Cost Model:** Per the verified repository context, the denoiser performs a **dense forward pass** on the entire sequence at each step. Thus, the compute cost of a schedule with T steps is:

$$C_{total} = T \times C_{step}$$

  Where $C_{step} \approx 2 \cdot P \cdot L$ (FLOPs for one forward pass). However, not all steps contribute equally to quality. We define the **Effective Compute** C_{eff} as the sum of compute costs weighted by the LSC:

$$C_{eff} = \sum_{t=1}^{T} C_{step} \cdot \max(0, \text{LSC}(t))$$

  This metric penalizes schedules that spend compute on "low-yield" steps (where LSC is low or negative) and rewards schedules that concentrate compute on "high-yield" steps.

- **Frontier Construction:** For each model, we plot **Quality** (NLL) vs. **Effective Compute** (C_{eff}) for different schedules and step counts. The **Area Under the Curve (AUC-EFF)** summarizes the frontier. A schedule with a higher AUC-EFF is more "compute-aware" in the sense that it achieves higher quality per unit of *useful* compute.

3. Cross-Family Generality via Schedule Shape Correlation Index (SSCI)

To address the "Architectural Generality" question, we test if the **Response Surface Map** $R(t)$ is transferable across model families.

- **Mechanism:**
  1. **Anchor Models:** DiffuGPT-S (GPT-2, 127M) and DiffuLLaMA-6.74B (LLaMA, 6.74B).
  2. **Response Surface Profiling:** For each anchor, we compute the Response Surface Map $R(t)$ for the four standard schedules (Linear, Cosine, Context-Adaptive, Learned).

3. **Shape Alignment:** We define the **Schedule Shape Correlation Index (SSCI)** between two models A and B for a given schedule S as the Pearson correlation coefficient between their Response Surface Maps:

$$\text{SSCI}(A, B; S) = \text{corr}(R_A(t), R_B(t))$$

4. **Transferability Test:** We hypothesize that if the optimal schedule shape is family-invariant, the SSCI for the "best" schedule (highest AUC-EFF) should be high (> 0.8). If the SSCI is low (< 0.5), it indicates that the critical steps for quality differ significantly between GPT-2 and LLaMA models, implying family-specific bias.
5. **Random Baseline:** To ensure SSCI is meaningful, we compare it against the SSCI obtained for a **Random Schedule** (generated by sampling β_t uniformly and sorting). If the SSCI for the "best" schedule is significantly higher than for the Random Schedule, it confirms that the response surface structure is a meaningful architectural property.

4. Experimental Protocol

- **Models:**
  - *Anchors:* DiffuGPT-S (127M), DiffuLLaMA-6.74B.
  - *Targets:* DiffuGPT-M (355M), DiffuCoder-7B, LLaDA-8B, Dream-7B.
- **Schedules:**
  1. **Linear:** $\beta_t = 1 - t/T$.
  2. **Cosine:** $\beta_t = \cos(\frac{\pi}{2}\frac{t}{T})$.
  3. **Context-Adaptive:** As implemented in Dream 7B (entropy-conditioned).
  4. **Learned:** A small MLP (2 hidden layers, 64 units) predicts β_t from t and the average token entropy of the current state.
     - **Training Protocol:** The MLP is trained on a **Calibration Set** (disjoint from Evaluation). The loss function is **MSE** between the predicted β_t and the β_t that maximizes the **AUC-EFF** on the Calibration Set. This is an inference-time calibration, not a weight update of the DLM.
     - **Generalization Check:** The learned schedule is tested on a **Domain Shift** evaluation set (e.g., if Calibration is WikiText, Evaluation includes C4 and MBPP) to ensure it is not overfitting to a specific domain.
- **Steps:** 16, 32, 64.
- **Metrics:**
  - *Quality:* Perplexity (WikiText-103), pass@1 (HumanEval/MBPP), MAUVE (C4 prompts), IFEval.
  - *Efficiency:* **AUC-EFF** (Area Under Effective Compute Curve), Wall-clock latency.
  - *Diagnostics:* **Response Surface Maps** $R(t)$ for each model/schedule. **Peak Sensitivity Step** t^*. **Sensitivity Entropy** H_R.
- **Analysis:**
  - **Generality Analysis:** Compute SSCI for all cross-family pairs (GPT-2 → LLaMA, LLaMA → GPT-2) and cross-scale pairs. Compare against Random Baseline SSCI.
  - **ANOVA:** Two-way ANOVA on **AUC-EFF** with factors *Model Family* and *Schedule Type*. Does the compute-efficiency gain from the "best" schedule differ significantly between GPT-2 and LLaMA?
  - **Sensitivity Shape Correlation:** Correlate the **Peak Sensitivity Step** t^* with the model's attention pattern. Hypothesis: LLaMA (bidirectional) may have a later peak t^* because it can leverage context more effectively in later steps, whereas GPT-2 (causal) may peak earlier.
  - **Frontier Mapping:** Plot Quality vs. AUC-EFF for all models. Identify if the "Transferred" schedule lies on the Pareto frontier for the target model.
  - **Task Sensitivity:** Analyze if SSCI varies across different tasks (coding vs. open-ended text) to determine if the "optimal" denoising trajectory is task-dependent.

5. Feasibility and Rigor

- **Inference-Only:** LSC is computed from the DLM's forward pass on evaluation sequences. The "Learned" schedule MLP is a tiny network trained on CPU/GPU using pre-computed entropy

features; it does not modify the DLM weights.

- **Compute:**
  - Anchor Optimization: 2 models × 4 schedules × 3 steps × 500 calibration sequences = 12,000 forward passes.
  - Perturbation Probes: For each anchor and schedule, we compute LSC for T steps. For $T = 16$, this is 16 probes × 500 sequences = 8,000 forward passes per anchor/schedule. Total for anchors: $2 \times 4 \times 3 \times 8,000 = 192,000$ forward passes. *Correction:* This is too high. We optimize by computing LSC on a **Subset** of 100 sequences for the probe step, and full 500 for final evaluation. Total probes: $2 \times 4 \times 3 \times 16 \times 100 = 38,400$ forward passes.
  - Transfer Tests: 4 targets × 2 transferred schedules × 3 steps × 500 evaluation sequences = 12,000 forward passes.
  - Baseline Comparisons: 4 targets × 1 random schedule × 3 steps × 500 evaluation sequences = 6,000 forward passes.
  - Total: ~70,000 forward passes. On a 96GB GPU, this is highly feasible (batch size 32-64).
- **Statistical Rigor:** 5 seeds for sampling-based metrics (MAUVE, IFEval). Report mean ± std for AUC-EFF and SSCI. Confidence intervals on SSCI are computed via bootstrapping over the evaluation sequences.
- **Addressing Verified Context:**
  - We explicitly acknowledge that the dense forward pass means FLOPs are constant per step, so AUC-EFF is a fair comparison of "useful" compute.
  - We use ground-truth likelihood (NLL) to anchor the LSC metric, avoiding the pitfalls of entropy-based proxies.
  - We define the "Learned" schedule clearly as a separate MLP, trained on a disjoint calibration set to prevent data leakage.

Why this method. This method addresses the core research problem by providing a **causal, discrete-sensitive** metric (LSC) for denoising efficiency, which is more rigorous than continuous gradients or entropy-based proxies. It directly answers the question of whether schedule-performance relationships differ across families by quantifying **Shape Correlation (SSCI)**.

1. **Addresses Undertraining (L7):** LSC identifies *where* in the trajectory the model is most sensitive to noise. If a schedule aligns with the model's high-LSC region, it maximizes quality gain from the undertrained model.
2. **Addresses Non-Compute-Normalized Reporting (L9):** AUC-EFF uses "Effective Compute" weighted by LSC, providing a fair, implementation-aware efficiency metric that rewards schedules for concentrating compute on high-yield steps.
3. **Addresses Architectural Generality (L10):** SSCI directly tests if the response surface shape is family-invariant. If SSCI is low, it proves family-specific bias. The Random Baseline ensures the SSCI is not an artifact of chance.
4. **Addresses Statistical Rigor (L11):** Multiple seeds, disjoint calibration/evaluation sets, and ANOVA ensure robust conclusions.
5. **Feasibility:** The experimental design is lean and focused on high-value diagnostics (LSC, SSCI), making it highly feasible within the 10-week timeline and single-GPU constraint.

Innovation:

1. **Discrete Sensitivity:** LSC measures *causal* impact of noise on quality using finite differences, valid for discrete token sampling.
2. **Shape Correlation Index (SSCI):** A novel metric to quantify the invariance of response surface shapes across model families, validated against a random baseline.
3. **Implementation-Aware FLOPs:** AUC-EFF respects the dense forward pass reality of the DiffuLLaMA implementation, avoiding misleading "sparse" efficiency claims.
4. **Task-Sensitive Analysis:** By testing SSCI across different tasks (coding vs. text), we provide a more nuanced view of schedule generalizability.

Experiment design.

Inference-Only Characterization of Noise Schedule Impact on the Quality-Efficiency Frontier of Scaled Discrete Diffusion Language Models via Discrete Perturbation Probes

1. Experimental Setup and Constraints

- **Hardware:** Single NVIDIA RTX 6000 Pro Blackwell (96 GB VRAM).
- **Models (Released Checkpoints):**
  - *GPT-2 Family (Anchor):* DiffuGPT-S (127M).
  - *LLaMA Family (Anchor):* DiffuLLaMA-6.74B.
  - *Target Models:* DiffuGPT-M (355M), LLaDA-8B, Dream-7B, DiffuCoder-7B.
- **Noise Schedules:**
  1. **Linear:** $\beta_t = 1 - t/T$.
  2. **Cosine:** $\beta_t = \cos(\frac{\pi}{2}\frac{t}{T})$.
  3. **Context-Adaptive:** Entropy-conditioned schedule as implemented in Dream 7B.
     - *Definition:* The noise level β_t is modulated by the average token-wise entropy of the current state x_t. Specifically, $\beta_t^{CA} = \beta_t^{base} \cdot (1 + \lambda \cdot \overline{H}(x_t))$, where $\overline{H}(x_t)$ is the mean entropy of the **predicted distribution** $p(x_0|x_t)$ for the **currently masked positions only**, and λ is a fixed scaling factor (e.g., 0.1) to ensure $\beta_t \in [0, 1]$.
  4. **Learned:** A small MLP (2 hidden layers, 64 units, **GELU** activation) predicting β_t from timestep t and the current state entropy.
     - *Input Features:* A 2-dimensional vector $[t/T, \overline{H}(x_t)]$. Both features are **min-max normalized** to $[0, 1]$ using statistics from the training calibration set before entering the MLP.
     - *Output Constraint:* The final layer uses a **Sigmoid** activation to constrain $\beta_t \in [0, 1]$.
     - *Training Protocol:* The MLP is trained on a disjoint calibration set (100 sequences from WikiText-103 Validation, selected via fixed random seed 42 for reproducibility).
     - *Target Definition:* The "optimal" β_t^* target for the MSE loss is determined via a **refined grid search**. First, a coarse grid $\{0.0, 0.1, \dots, 1.0\}$ is used to find the local optimum. Then, a fine-grained local search (± 0.05 around the coarse optimum, step 0.01) is performed on the same 100 calibration sequences to refine β_t^*. The "local quality gain" is measured as the reduction in NLL at step t when perturbing β_t while keeping other steps at the base schedule.
     - *Optimizer:* **AdamW** (lr=1e-3, weight_decay=0.01) for 1000 epochs. Weights are frozen for evaluation.
- **Denoising Steps (T):** 16, 32, 64.
- **Sequence Length:** Fixed at 128 tokens.
- **Evaluation Sets:**
  - *Calibration (for Learned Schedule & LSC probes):* 100 sequences from WikiText-103 Validation (disjoint from Eval, seed 42).
  - *Evaluation:*
    - *LM Quality:* 500 segments from WikiText-103 Test.
    - *Infilling:* 100 multi-line HumanEval and 100 MBPP instances (masking middle 20% of tokens).
    - *Instruction Following:* 100 prompts from IFEval.
    - *Open-Ended:* 100 prompts from C4.

2. Methodology: Schedule-Response Surface Mapping (SRSM)

Phase 1: Response Surface Profiling (LSC Calculation)

- **Goal:** Compute the Local Sensitivity Coefficient (LSC) for each timestep t to map the "criticality" of noise levels.
- **Procedure:**
  1. For each Model $M \in$ {All 6 Models} and each Base Schedule ($S \in$ {Linear, Cosine, Context-Adaptive, L
  2. Initialize the denoising process with T steps.
  3. For each timestep $t \in \{1, \dots, T\}$:
     - **Probe P_t^δ:** Create a perturbed schedule where β_t is increased by $\delta = 0.05$. To maintain the noise budget, the noise is compensated at a reference step t_{ref}.
     - **Reference Step Selection & Sensitivity Check:**
       - *Primary Calculation:* Use $t_{ref} = T$ (final step) as the primary reference.
       - *Boundary Clipping:* If $\beta_{t_{ref}} - \delta < 0$, the perturbation is clipped such that $\beta_{t_{ref}} = 0$

and the total noise budget is not strictly preserved (documented as a boundary case). If $\beta_t + \delta > 1$, it is clipped to 1.
    * *Sensitivity Check:* For the **Anchor models only** and for **10% of the probe steps** (randomly sampled t), also compute LSC using $t_{ref} = T/2$.
    * *Trigger Condition:* If the correlation between LSC vectors computed with $t_{ref} = T$ and $t_{ref} = T/2$ is < 0.8 for a specific model/schedule, **trigger the 3-reference average** ($t_{ref} \in \{T, T/2, 1\}$) for that specific model/schedule in the full probe set. Otherwise, proceed with $t_{ref} = T$ only.
  - **Inference:** Run the full denoising trajectory (all T steps) on the **50 Calibration Sequences** using this perturbed schedule.
  - **Metric:** Calculate the **Negative Log-Likelihood (NLL)** of the **original ground-truth tokens** from the WikiText-103 validation set using the **model's final logits** before the final sampling step. This provides a stable, continuous signal for the LSC calculation.
  - **Baseline:** Calculate NLL (Q_{base}) using the unperturbed base schedule on the same sequences. **Checkpointing:** The base schedule runs are cached and reused for all perturbations at that T to avoid redundant forward passes.
  - **LSC Calculation:**

$$\text{LSC}(t) = \frac{Q(S_{base}) - Q(P_t^{\delta, t_{ref}})}{\delta}$$

4. **Interpolation for T=32:**
   - Compute LSC explicitly for $T = 16$ and $T = 64$.
   - **Pilot Verification:** Before full execution, run a pilot on DiffuGPT-S for $T = 32$ on 10 sequences. Compare the $T = 32$ LSC vector to the **linear interpolation** of the $T = 16$ and $T = 64$ LSC vectors (resampled to length 32). If the Pearson correlation > 0.95, use linear interpolation for $T = 32$ LSC for all models. If < 0.95, compute $T = 32$ explicitly for the anchor models and interpolate for targets only if correlation > 0.9.
5. **Response Surface Map ($R(t)$):** Construct the vector $R = [\text{LSC}(1), \ldots, \text{LSC}(T)]$.
6. **Diagnostics:**
   - *Peak Sensitivity Step (t^*):* $\arg\max_t \text{LSC}(t)$.
   - *Sensitivity Entropy (H_R):* Shannon entropy of the normalized LSC distribution.

Phase 2: Compute-Aware Frontier Projection

- **Goal:** Quantify efficiency-adjusted quality to determine the "Effective Compute" (C_{eff}).
- **Procedure:**
  1. For each Model and each Schedule/Step configuration:
  2. Run inference on the full Evaluation sets.
  3. Calculate **Quality Metrics:**
     - *LM:* Perplexity (from NLL).
     - *Infilling:* **Pass@1** and **Pass@10**. For Pass@10, 10 independent samples are generated per prompt for each of the 5 seeds, and the standard unbiased estimator for Pass@k is used to capture diversity.
     - *IFEval:* Instruction Following score.
     - *Open-Ended:* MAUVE score.
  4. Calculate **Effective Compute (C_{eff}):**
     - Base FLOPs per step $C_{step} \approx 2 \cdot P \cdot L$ (where P is model params, $L = 128$).
     - $C_{eff} = \sum_{t=1}^{T} C_{step} \cdot \max(0, \text{LSC}(t))$.
     - *Note:* We compute LSC for **all 6 models** on the 50 Calibration Sequences. This ensures that the "Effective Compute" is specific to each model's actual sensitivity profile.
  5. **AUC-EFF Calculation:** Plot Quality (Perplexity, Pass@1, etc.) vs. C_{eff} for $T \in \{16, 32, 64\}$. Compute the Area Under the Curve (AUC) to summarize the frontier. Higher AUC indicates better quality per unit of "useful" compute.

Phase 3: Cross-Family Generality (SSCI)

- **Goal:** Test if the Response Surface Map shape is invariant across architectures (GPT-2

vs. LLaMA).

- **Procedure:**
  1. Compute Pearson Correlation Coefficient between the Response Surface Maps of DiffuGPT-S and DiffuLLaMA-6.74B for each Schedule S:

$$\text{SSCI}(S) = \text{corr}(R_{\text{GPT-S}}(t), R_{\text{LLaMA}}(t))$$

  2. **Random Baseline:** Generate 100 random schedules (uniform β_t). Compute the average correlation between random schedules to establish a null distribution.
  3. **Significance Test:** If SSCI(S) significantly exceeds the random baseline mean (t-test, $p < 0.05$), the schedule structure is a meaningful architectural property.
  4. **Cross-Scale Check:** Also compute SSCI between DiffuGPT-S and DiffuGPT-M (within-family, cross-scale) and between DiffuLLaMA-6.74B and LLaDA-8B (within-family, cross-scale) to distinguish family effects from scale effects.

Phase 4: Statistical Analysis and ANOVA

- **Procedure:**
  1. **Reproducibility:** Run all inference experiments with 5 different random seeds for sampling (where applicable, e.g., MAUVE, IFEval, Pass@10). For deterministic metrics (Perplexity, Pass@1 with greedy decoding if supported, or fixed seed), report mean ± std.
  2. **ANOVA:** Perform Two-Way ANOVA on **AUC-EFF** with factors:
     - Factor 1: Model Family (GPT-2 vs. LLaMA).
     - Factor 2: Schedule Type (Linear, Cosine, Context-Adaptive, Learned).
  3. **Interaction Effect:** Test if the interaction term (Family × Schedule) is significant. This directly answers if the "best" schedule differs by family.
  4. **Task Sensitivity:** Correlate t^* (Peak Sensitivity Step) with task type (Coding vs. Text). Hypothesis: Coding tasks may have earlier peaks (syntax resolution) vs. later peaks (semantic coherence).

3. Feasibility and Compute Budget

- **Forward Passes:**
  - *LSC Probes (All 6 Models):*
    - *Strategy:* Compute LSC for $T = 16$ and $T = 64$ explicitly. Interpolate $T = 32$ (verified via pilot). Use 50 sequences. Use $t_{ref} = T$ primarily, with conditional 3-ref average for Anchors if variance is high.
    - *Probes per model/schedule:* $(16 + 64)$ steps $\times$ 50 seqs $\times$ 2 (base+pert) = $8,000$ forward passes.
    - *Total LSC Probes:* 6 models $\times$ 4 schedules $\times$ $8,000 = 192,000$ forward passes.
    - *Pilot Cost:* Negligible (1 model, 10 seqs, 3 T-values).
  - *Evaluation Inference:*
    - 6 Models × 4 Schedules × 3 Step Counts × (500 LM + 100 Code + 100 IFEval + 100 C4) × 5 Seeds = $6 \times 4 \times 3 \times 800 \times 5 = 288,000$ forward passes.
  - *Total:* ~480,000 forward passes.
- **Time Estimate:**
  - DiffuLLaMA-6.74B (96GB VRAM): Batch size 1-2 (BF16). ~1-2 sec/forward pass (128 tokens).
  - Smaller models (DiffuGPT-S/M): Batch size 32-64. ~0.1 sec/forward pass.
  - Weighted Average: Most passes are on 7B models (4 out of 6). Assume 1.5 sec avg.
  - Total GPU Hours: $480,000/3600 \times 1.5 \approx 200$ GPU hours.
  - With 10 weeks and 3 GPU members, this is feasible (approx. 20 GPU hours/week/member).
  - *Mitigation:* Implement a **checkpointing system** for base schedule runs. Cache the hidden states/logits of the base run to avoid re-computing the base trajectory for every perturbation probe where possible (though full trajectory re-run is required for perturbed steps, the base NLL is cached).

4. Expected Outcomes and Hypotheses

1. **LSC Distribution:** We expect LSC to be non-uniform. For LLaMA-based models, we hypothesize a later peak t^* (higher t) due to bidirectional context allowing later refinement. For GPT-2, an earlier peak due to causal constraints.
2. **Schedule Performance:** The "Learned" schedule should outperform Linear/Cosine in AUC-EFF by concentrating noise where LSC is high. Context-Adaptive should perform well on LLaMA models.
3. **SSCI:** If SSCI > 0.8 for the best schedule, it suggests a universal "optimal noise shape" across families. If SSCI < 0.5, it confirms family-specific bias, supporting the need for architecture-specific scheduling.
4. **ANOVA:** Significant interaction effect would prove that the choice of schedule must be tailored to the model family, a key practical implication for DLM deployment.
5. **Robustness:** Pass@10 should show that better schedules not only improve the mean (Pass@1) but also increase the probability of success in stochastic sampling, validating the "quality" claim beyond just average performance.

Why this design. This experiment design directly addresses the research problem by moving beyond simple wall-clock latency comparisons to a **compute-aware, causal analysis** of noise schedules.

1. **Causal Validity via LSC:** By using Discrete Perturbation Probes, we avoid the pitfalls of treating discrete token sampling as a continuous process. The Local Sensitivity Coefficient (LSC) provides a ground-truth-anchored (NLL-based) measure of *which* timesteps actually matter for quality. Using the **logit-based NLL** rather than sampled-token NLL ensures a stable signal for the finite difference calculation, addressing the validity feedback regarding discrete sampling noise.
2. **Implementation-Aware Efficiency:** The verified repository context confirms that DiffuLLaMA performs dense forward passes on the entire sequence. Therefore, standard FLOPs counting ($T \times 2PL$) is constant for a fixed T. By introducing **Effective Compute (C_{eff})** weighted by LSC, we distinguish between "wasted" compute (steps with low LSC) and "useful" compute. This directly addresses the limitation of non-compute-normalized reporting in the target paper.
3. **Architectural Generality:** The Schedule Shape Correlation Index (SSCI) provides a rigorous, quantitative test for the hypothesis that noise schedule preferences differ across GPT-2 and LLaMA families. By comparing against a random baseline and including cross-scale controls, we ensure that any observed correlation is statistically meaningful and not an artifact of the small number of timesteps or scale differences.
4. **Feasibility and Rigor:** The design strictly adheres to the "inference-only" constraint. The "Learned" schedule is a tiny MLP trained on a disjoint calibration set, ensuring no data leakage and no modification of the DLM weights. The use of 5 seeds, Pass@10, and ANOVA ensures statistical rigor, addressing the lack of variance estimates in previous work. The compute budget is carefully calculated to fit within the single-GPU, 10-week constraint, leveraging the small size of the GPT-2 models for batched inference and the large VRAM of the RTX 6000 Pro for the 7B models.
5. **Addressing Specific Feedback:**
   - **Clarity:** We explicitly defined the entropy metric (mean token-wise entropy of the **predicted distribution** $p(x_0|x_t)$ for **masked positions only**) and the MLP architecture (GELU, AdamW, **Sigmoid output**, **min-max input normalization**). We specified the **refined grid search** (coarse + fine local search) for the learned schedule target and the **fixed random seed (42)** for data selection to ensure absolute reproducibility.
   - **Validity:** We compute LSC for **all models** (not just anchors) to ensure the "Effective Compute" is model-specific. We use logit-based NLL for stability. The "Learned" schedule MLP is trained to predict the **local optimal** β_t^* (maximizing local NLL reduction) rather than the global AUC-EFF, resolving the optimization target ambiguity. We explicitly document **boundary clipping** for the noise budget compensation to handle cases where $\beta_{t_{ref}}$ might drop below 0.
   - **Robustness:** We address the reference step sensitivity by introducing a **conditional trigger**: if the correlation between $t_{ref} = T$ and $t_{ref} = T/2$ LSC vectors is < 0.8, we auto-

matically switch to the 3-reference average for that model/schedule. We include **Pass@10** calculated via the standard unbiased estimator across 10 samples per prompt to properly capture stochastic robustness and diversity.
- **Feasibility:** The budget is revised to be realistic (~200 GPU hours) by limiting LSC probes to key step counts (16, 64) and using a smaller calibration set (50 sequences) for the probes, while maintaining full evaluation sets for final metrics. We added a **checkpointing** strategy for base runs to save compute. We also noted the use of **BF16 precision** for 7B models to potentially increase batch size and reduce wall-clock time.
- **Reproducibility:** All hyperparameters (MLP architecture, optimizer, normalization, reference step trigger threshold, grid search parameters, random seeds) are explicitly specified. The interpolation strategy for $T = 32$ is defined as **linear interpolation of the LSC vector**, subject to a **pilot verification** step with a stricter threshold (0.95) to mitigate the risk of non-linear LSC profiles.

Method	Clarity	Validity	Rigorousness	Innovativeness	Generalizability
rating	4	5	5	4	5

Experiment	Clarity	Validity	Robustness	Feasibility	Reproducibility
rating	5	5	5	5	5

Weakest point per the reviewers (Clarity, 4/5). While the method is exceptionally clear, two minor areas could be further specified to ensure absolute replicability:

1. **Reference Step Selection:** The method mentions using a reference step t_{ref} (e.g., the final step T) to preserve the noise budget. It would be beneficial to specify if t_{ref} is kept constant for all probes P_t^δ or if it is chosen dynamically to avoid perturbing a high-sensitivity region.
2. **Learned Schedule Target:** The training protocol for the MLP states the loss is MSE between predicted β_t and the β_t that "maximizes the AUC-EFF on the Calibration Set." Since AUC-EFF is a global property of a schedule, it is slightly ambiguous how a point-wise β_t target is derived for the MLP's MSE loss. Clarifying whether this target is derived from a grid search or a specific optimization loop would complete the description.

Idea 2.5 — P2-M4 (overall 4.6/5)

Method.

Resolution-Phase Decomposition (RPD) with Context-Dependent Convergence Profiling**

We propose a rigorous, inference-only methodology that reframes the noise schedule not as a static temporal curve, but as a **controller for the phase transition** of token resolution. Instead of measuring global efficiency or spatial bias in isolation, RPD dissects the denoising trajectory into two distinct operational phases: **Phase 1 (Exploration/High-Entropy)** where the model is uncertain and relies heavily on context to propose candidates, and **Phase 2 (Commitment/Low-Entropy)** where the model finalizes tokens.

Our method introduces two key innovations:

1. **Entropy-Phase Partitioning (EPP):** A dynamic, data-driven algorithm that identifies the specific timestep t^* where the model transitions from "exploring" to "committing," based on the drop in output entropy. This allows us to evaluate schedules based on how well they manage the *duration* and *quality* of each phase, rather than just the total number of steps.
2. **Context-Dependent Convergence (CDC) Metric:** A measure of how effectively the model leverages bidirectional context during the Exploration phase. This directly addresses the architectural generality question by quantifying whether LLaMA-based models (bidirectional) utilize their full context window earlier and more effectively than GPT-2-based models (causal) under different schedules.

1. Core Mechanism: Entropy-Phase Partitioning (EPP)

Standard DLM inference treats all steps as equivalent units of compute. However, the "work" done in early steps (resolving high-entropy tokens) is fundamentally different from late steps (verifying low-entropy tokens). We hypothesize that an "optimal" schedule is one that minimizes the time spent in the high-entropy phase while maintaining sufficient context for correct commitment.

- **Mechanism:**
  1. **Trajectory Tracking:** During inference, for each step t from T down to 1, we record the **Mean Output Entropy** H_t across all currently masked (unresolved) positions.

$$H_t = \frac{1}{|\mathcal{M}_t|} \sum_{i \in \mathcal{M}_t} H(p_t(\cdot|x_{\neg i}))$$

     where $\mathcal{M}_t$ is the set of masked positions at step t, and $H(\cdot)$ is the Shannon entropy of the predicted token distribution.
  2. **Phase Boundary Identification (t^*):** We define the **Phase Boundary** t^* as the first timestep (starting from $t = T$) where the mean entropy H_t drops below a threshold τ. We set τ to the 25th percentile of the entropy distribution observed across all steps in a "reference" run (e.g., using the Linear schedule with $T = 64$).
     - **Phase 1 (Exploration):** $t \in [T, t^*]$. High entropy, tokens are being proposed.
     - **Phase 2 (Commitment):** $t \in [t^*, 1]$. Low entropy, tokens are being finalized.
  3. **Phase Duration Metric:** We define the **Exploration Fraction** $\phi = (T - t^*)/T$. A schedule with a smaller ϕ resolves uncertainty faster, potentially saving compute if quality is maintained.
- **Innovation:** This moves beyond static "Effective Denoising Steps" (EDS) by recognizing that *when* the model stops being uncertain is as important as *how many* times it updates. It allows us to distinguish between a schedule that quickly commits to wrong tokens (low ϕ, low quality) and one that quickly commits to right tokens (low ϕ, high quality).

2. Context-Dependent Convergence (CDC) Metric

To address the architectural generality (L10), we need to measure *how* the model resolves uncertainty. We hypothesize that bidirectional models (LLaMA) should show a steeper drop in entropy in Phase 1 because they can access both left and right context immediately, whereas causal models (GPT-2) may require more steps to "propagate" context from the known prefix to the masked region.

- **Mechanism:**
  1. **Context Utilization Score (CUS):** For each masked position i during Phase 1, we calculate the **Contextual Gain** $G_i(t)$. This is the reduction in entropy at position i compared to a "context-free" baseline.
     - *Baseline Estimation:* To estimate the "context-free" entropy without extra forward passes, we use the **Marginal Unigram Entropy** $H_{uni}(i)$ calculated from the token distribution of the entire sequence. While not a perfect proxy for "no context," it provides a lower bound on uncertainty.
     - *Alternative Rigorous Proxy:* We compute the **Attention Concentration**. If attention weights are accessible, we measure the ratio of attention mass paid to *known* (unmasked) tokens vs. *masked* tokens. A higher ratio to known tokens indicates better context utilization.
  2. **CDC Score:** We define the **Context-Dependent Convergence** score as the slope of the entropy decay curve during Phase 1:

$$\text{CDC} = \frac{H_T - H_{t^*}}{T - t^*}$$

     A higher absolute CDC indicates that the model is rapidly reducing uncertainty per step, implying efficient use of available context.
- **Hypothesis:**
  - **LLaMA-based models** should exhibit a **higher CDC** (steeper entropy drop) under Uniform schedules (Linear/Cosine) because their bidirectional attention allows immediate context integration.

- **GPT-2-based models** may exhibit a **lower CDC** initially but might benefit more from **Context-Adaptive** schedules that delay commitment on high-entropy tokens, allowing the causal context to "propagate" further before forcing a decision.

3. Compute-Normalized Frontier via "Phase-Weighted FLOPs"

To address the non-compute-normalized efficiency limitation (L9), we refine the FLOP calculation to account for the *value* of the computation. Since Phase 2 (Commitment) often involves verifying already-likely tokens, it may be "cheaper" in terms of information gain per FLOP than Phase 1.

- **Metric Definition:**
  - **Raw FLOPs:** $F_{raw} = P \times L \times T$ (Constant for a given model and step count).
  - **Phase-Weighted FLOPs (F_{pw}):** We weight the cost of Phase 1 higher because it involves high-entropy sampling which is more "risky" and computationally critical for quality.

$$F_{pw} = F_{raw} \times \left(1 + \alpha \cdot \frac{\text{Exploration Steps}}{\text{Total Steps}}\right)$$

    Where α is a tunable weight (default 1.0) reflecting the higher cognitive load of exploration.
  - **Efficiency Score:** $E = \frac{\text{Quality}}{F_{pw}}$.
- **Frontier Construction:** For each model and schedule, we plot Quality (e.g., Perplexity or Pass@1) vs. F_{pw} across step budgets (16, 32, 64). The **Area Under the Curve (AUC)** of this frontier summarizes the trade-off. A schedule that reduces ϕ (Exploration Fraction) without sacrificing Quality will have a better AUC.

4. Experimental Protocol

- **Models:**
  - **Causal Family:** DiffuGPT-S (127M), DiffuGPT-M (355M).
  - **Bidirectional/Hybrid Family:** DiffuLLaMA-6.74B, LLaDA-8B, Dream-7B.
  - *Note:* We exclude DiffuCoder-7B from the primary family comparison as it is a specialized code model, but include it as a robustness check for the "Specialized vs. General" interaction.
- **Schedules:**
  1. **Linear:** $\beta_t = 1 - t/T$.
  2. **Cosine:** $\beta_t = \cos(\frac{\pi}{2}\frac{t}{T})$.
  3. **Context-Adaptive (Dream-7B style):** Noise level β_t is modulated by the current token-wise entropy. Specifically, positions with high entropy retain higher noise longer.
  4. **Learned Schedule (MLP):**
     - **Input:** Timestep t/T and **Global Entropy** H_t (normalized).
     - **Output:** A scalar multiplier $m_t \in [0.5, 1.5]$ applied to the base Linear schedule.
     - **Training Target:** The MLP is trained to predict the **Optimal Commitment Step**. We define the target y_t for timestep t as 1 if $t < t^*$ (Exploration) and 0 otherwise, derived from the EPP analysis of a reference run. The loss is Binary Cross-Entropy. This trains the MLP to "know" when the model is ready to commit, effectively learning a dynamic threshold for the phase transition.
     - *Constraint:* The MLP is trained on a held-out validation set of 100 prompts from WikiText-103 to avoid test-set leakage.
- **Steps:** 16, 32, 64.
- **Sequence Length:** 128 tokens.
- **Metrics:**
  - **Quality:** Perplexity (WikiText-103), Pass@1 (HumanEval/MBPP), MAUVE.
  - **Efficiency:** Wall-Clock Latency, **Phase-Weighted FLOPs (F_{pw})**.
  - **Diagnostics (New):**
    - **Exploration Fraction (ϕ):** How much of the trajectory is spent in high-entropy state.
    - **CDC Score:** Rate of entropy decay.
    - **Phase Boundary (t^*):** Absolute step where commitment begins.
- **Analysis:**

1. **Family Interaction:** Two-way ANOVA on **CDC Score** with factors *Model Family* and *Schedule Type*.
   - *Hypothesis:* LLaMA models will have significantly higher CDC under Linear/Cosine schedules compared to GPT-2 models.
2. **Schedule Efficiency:** Compare **AUC of Quality vs.** F_{pw} for each model.
   - *Hypothesis:* Context-Adaptive schedules will yield higher AUC for GPT-2 models (by managing the slower context propagation) while Linear/Cosine will yield higher AUC for LLaMA models (by not delaying the fast bidirectional convergence).
3. **Learned Schedule Performance:** Does the MLP-trained schedule outperform the static archetypes in **Exploration Fraction (ϕ)** without degrading Quality?
4. **Generalizability:** Test if the **Optimal** α (weight for Phase-Weighted FLOPs) that maximizes AUC is consistent across scales within a family.

5. Feasibility and Rigor

- **Inference-Only:**
  - EPP and CDC are post-hoc calculations on the entropy trajectory. No weight updates.
  - The "Learned Schedule" MLP is a tiny network (2 hidden layers, 64 units) trained on static entropy data. It does not touch the LM weights. It is trained once on the validation set and frozen for all test runs.
- **Compute:**
  - 5 models × 4 schedules × 3 step counts = 60 configurations.
  - With 5 seeds, 300 inference runs.
  - Each run involves 16-64 forward passes on a 128-token sequence. This is highly feasible on a 96GB GPU (batching multiple prompts per forward pass where possible, or sequential if memory constrained, but 128 tokens is small).
  - Entropy calculation is $O(L \times V)$ per step, negligible compared to the transformer forward pass.
- **Statistical Rigor:**
  - 5 seeds per configuration.
  - Report mean ± std for ϕ, CDC, Quality, and F_{pw}.
  - Use paired t-tests to compare schedules within the same model.
  - Use ANOVA for family/schedule interactions.

Addressing Review Feedback from Previous Method:

1. **Formalized SSI/STM:** We replaced the ambiguous "Schedule Symmetry Index" with the **Entropy-Phase Partitioning (EPP)**. The "phase boundary" t^* is a precise, data-driven metric derived from the entropy trajectory, not a qualitative "symmetry" guess.
2. **Clarified "Resolved" Tokens:** We do not rely on "fixed" tokens in the sense of binary state changes only. We track **Entropy**. A token is considered "in the exploration phase" as long as its contribution to the mean entropy is significant. This handles the case where a token is sampled but remains uncertain (high entropy) for several steps.
3. **MLP Target Definition:** The MLP target is explicitly defined as a **Binary Classification** task: Predicting whether the current timestep is in the "Exploration" phase ($t > t^*$) or "Commitment" phase ($t < t^*$). The target labels are derived from the EPP analysis of a reference run. The loss is Binary Cross-Entropy. This is precise and replicable.
4. **Baseline for Symmetry:** We moved away from "Left/Right" symmetry, which was a proxy for architecture. We now use **Entropy Decay Rate (CDC)** as the direct measure of architectural efficiency. This is more robust than spatial bias, which can be confounded by the specific content of the prompt (e.g., a prompt with a long prefix will naturally have different resolution dynamics than one with a short prefix).

Innovation:

1. **Phase-Based Analysis:** Shifts the focus from "how many steps" to "what kind of steps." It recognizes that denoising is a two-phase process (Explore vs. Commit) and evaluates schedules based on their management of this transition.
2. **Context-Dependent Convergence (CDC):** Provides a direct, quantitative link between **Ar-**

chitectural Bias (Causal vs. Bidirectional) and **Inference Efficiency**. It tests whether bidirectional attention actually leads to faster entropy reduction.
3. **Learned Phase Controller:** The MLP schedule is not just predicting a noise level, but learning the **optimal timing for commitment**. This is a more semantically meaningful "knob" than a static noise multiplier.

Why this method. The proposed method, **Resolution-Phase Decomposition (RPD) with Context-Dependent Convergence Profiling**, directly addresses the research problem by providing a rigorous, inference-only framework that links noise schedule choice to the *dynamics* of information resolution, rather than just the *quantity* of noise.

1. **Architectural Generality (Addressing L10 & Review Feedback):** The core innovation is the **Context-Dependent Convergence (CDC)** metric. By measuring the rate of entropy decay, we directly test the hypothesis that bidirectional models (LLaMA) resolve uncertainty faster than causal models (GPT-2) due to their attention mechanism. This is a substantively different angle from the previous "Spatial Bias" approach, which was confounded by prompt content. CDC is a property of the *model's processing efficiency*, not just the *output order*.
2. **Valid Efficiency & Quality Link (Addressing L9 & L7):** By introducing **Phase-Weighted FLOPs (F_{pw})** and the **Exploration Fraction (ϕ)**, we provide a compute-normalized metric that accounts for the *difficulty* of the denoising steps. A schedule that spends 80% of its steps in high-entropy exploration is less efficient than one that spends 20%, even if the total step count is the same. This offers a concrete, training-free lever to mitigate undertraining: if a model is undertrained, it likely has a high ϕ (struggles to commit). Schedules that reduce ϕ (like Context-Adaptive) should show the biggest gains for undertrained models.
3. **Inference-Only Knob (Addressing L5 & L7):** The **Learned Schedule** (MLP) is a practical, training-free knob. By training the MLP to predict the "Commitment Phase," we allow practitioners to use a schedule that dynamically adjusts to the model's current state. This is more robust than static schedules because it adapts to the difficulty of the specific prompt.
4. **Statistical Rigor (Addressing L11):** The method includes multiple seeds, ANOVA on CDC, and correlation analysis on ϕ, ensuring statistical significance of the architectural/schedule interactions. The use of a held-out validation set for the MLP training ensures no test-set leakage.
5. **Innovation:** RPD is substantively different from previous methods (SAPD, TSNSC, CSI, STM). It does not focus on *spatial* bias (STM), *spectral* flow (TSNSC), or *interpolation* (CSI). It focuses on the **temporal phase dynamics** of the denoising process. It provides a mechanistic explanation for *why* a schedule works: by optimizing the transition from exploration to commitment. This provides a deeper, more actionable understanding of the quality-efficiency frontier.

Experiment design.

Experiment Name:** Phase-Resolved Noise Schedule Analysis (PRNSA) for Scaled Diffusion Language Models (Revised v3)

1. Experimental Setup

- **Models (5 Checkpoints):**
  1. **DiffuGPT-S** (127M, GPT-2 architecture, Causal attention).
  2. **DiffuGPT-M** (355M, GPT-2 architecture, Causal attention).
  3. **DiffuLLaMA-6.74B** (LLaMA architecture, Bidirectional attention).
  4. **Dream-7B** (LLaMA architecture, Bidirectional attention, native Context-Adaptive support).
  5. **LLaDA-8B** (LLaMA architecture, Bidirectional attention). *Note: DiffuCoder-7B is excluded from the primary family comparison to maintain clean "General Language Model" vs. "Specialized Code Model" separation, but will be included as a robustness check for the code-specific schedule effects if compute permits.*
- **Noise Schedules (4 Types):**
  1. **Linear:** $\beta_t = 1 - t/T$.
  2. **Cosine:** $\beta_t = \cos(\frac{\pi}{2}\frac{t}{T})$.
  3. **Context-Adaptive (CA):** Token-wise noise modulation based on current entropy.

- *Implementation Specification:* To ensure reproducibility and avoid architectural drift, the CA schedule is implemented via a unified entropy-thresholding logic for all models. At each step t, for each masked position i, the model computes the entropy $H_i(t)$. The position remains masked (noisy) in the next step if $H_i(t) > \theta_{CA}^{(T)}$, where $\theta_{CA}^{(T)}$ is set to the median entropy observed in the Linear $T = 64$ reference run for that specific model, **rescaled** for the current step count T using the ratio of total steps: $\theta_{CA}^{(T)} = \theta_{CA}^{(64)} \times \frac{T}{64}$. This strictly mirrors the "delay commitment on high-entropy tokens" logic of Dream-7B without relying on proprietary internal code structures, ensuring the CDC metric is not confounded by implementation differences.
- *Non-Linearity Check:* We will verify the linearity of entropy decay. If the entropy at $t = 16$ is not linearly proportional to the entropy at $t = 64$ (checked on a subset of 50 validation prompts), we will report the deviation and potentially adjust the rescaling factor to be non-linear (e.g., quadratic) if it significantly impacts the ϕ metric, though the primary analysis will stick to the linear heuristic for simplicity unless the deviation is $> 20\%$.

4. **Learned Phase Controller (LPC):** A small MLP that predicts a multiplier $m_t \in [0.5, 1.5]$ for the base Linear schedule.
   - *Architecture:* Input layer (2 units: t/T, H_t normalized) → Hidden Layer 1 (64 units, **GELU** activation) → Hidden Layer 2 (64 units, **GELU** activation) → Output Layer (1 unit, **Sigmoid** activation).
   - *Mapping Logic:* The output of the Sigmoid function is $p_{commit} \in [0, 1]$, representing the probability that the model is in the "Commitment" phase. The multiplier is linearly mapped as: $m_t = 0.5 + p_{commit} \cdot (1.5 - 0.5) = 0.5 + p_{commit}$.
   - *Interpretation:* High p_{commit} (Commitment phase) yields $m_t \approx 1.5$, effectively increasing the noise removal rate (aggressive denoising). Low p_{commit} (Exploration phase) yields $m_t \approx 0.5$, preserving noise to allow further refinement.
   - *Training Protocol:*
     1. **Reference Run:** Perform a Linear $T = 64$ inference on **500 held-out validation prompts** from WikiText-103 (distinct from the test set).
     2. **Label Generation:** For each prompt, compute the entropy trajectory H_t. Determine the phase boundary t^* using the EPP algorithm (first t where $H_t < \tau$, with τ being the 25th percentile of the entropy distribution for that specific prompt). Label each timestep t: $y_t = 1$ if $t < t^*$ (Commitment), else 0 (Exploration).
     3. **Stratification:** Ensure the 500 prompts are stratified by initial entropy (low, medium, high) to prevent the LPC from overfitting to a specific "type" of sequence.
     4. **Training:** Train the MLP to predict y_t from $(t/T, H_t)$ using Binary Cross-Entropy loss.
        * *Optimizer:* AdamW with a learning rate of 1×10^{-3} and weight decay of 1×10^{-2}.
        * *Epochs:* 100 epochs or until validation loss plateaus.
     5. **Freeze:** Freeze the MLP weights. Apply the trained LPC to the test set.
   - *Generalization Check:* The LPC trained on WikiText-103 is applied to HumanEval prompts. We will monitor for degradation in Pass@1 compared to the Linear baseline to assess cross-domain generalizability. If degradation is significant, we will train a separate LPC on 500 HumanEval prompts for the code-specific analysis.
   - *Zero-Shot Baseline:* To quantify the contribution of the *dynamic* nature of the MLP, we will also evaluate a "Constant LPC" baseline where m_t is fixed to the average p_{commit} of the validation set. This allows us to isolate the benefit of the dynamic adjustment from a simple global noise level shift.

- **Denoising Steps (T):** 16, 32, 64.
- **Sequence Length:** 128 tokens.
- **Seeds:** 5 random seeds per configuration.
- **Total Configurations:** 5 models × 4 schedules × 3 step counts × 5 seeds = 300 inference runs.

2. Data & Evaluation Benchmarks

- **Quality Metrics:**
  - **Perplexity (PPL):** WikiText-103 (sub-sampled to 1000 sequences for computational feasi-

bility, standardized across models).
  - **Infilling Accuracy:** HumanEval-Infilling and MBPP-Infilling (Exact Match Pass@1).
  - **Diversity/Quality:** MAUVE score on open-ended generation (C4 prompts).
  - **Instruction Following:** IFEval (Instruction-Following Evaluation) score.
- **Efficiency Metrics:**
  - **Wall-Clock Latency:** Time per generated token (batch size 1).
    - *Implementation Note:* All inference runs are wrapped in `torch.no_grad()` and executed in `float16` precision. **Crucially, we record the latency of the transformer forward pass separately from the entropy/sampling overhead.** The reported "Wall-Clock Latency" will be the sum, but we will also report the "Core Inference Latency" (forward pass only) to isolate model architecture efficiency from schedule-specific computational overheads.
    - *Batch Size Consistency:* Latency measurements are strictly performed at **batch size 1** to avoid non-linear scaling effects of GPU kernels. Batching is only used for throughput during the bulk data generation phase (where latency is not the primary metric of interest, but rather the final output quality), while the specific latency benchmarks are run sequentially.
  - **Phase-Weighted FLOPs (F_{pw}):** Calculated as $F_{raw} \times (1 + \alpha \cdot \phi)$, where $F_{raw} = 2 \cdot P \cdot L \cdot T$ and ϕ is the Exploration Fraction.
    - *Sparsity Acknowledgement:* We acknowledge that the Context-Adaptive schedule may reduce the number of active tokens per step. However, since the DiffuLLaMA implementation performs a dense forward pass on the entire sequence regardless of mask state (as verified in the repository context), the *actual* FLOPs remain F_{raw}. The F_{pw} metric serves as a *theoretical* efficiency proxy that accounts for the "value" of computation, rather than a measure of physical sparsity savings. We will explicitly state this distinction in the analysis to avoid conflating theoretical importance with physical compute reduction.
    - *Justification for $\alpha = 1.0$:* The default $\alpha = 1.0$ implies that Exploration steps are weighted twice as heavily as Commitment steps. This is justified by the information-theoretic assumption that resolving high-entropy tokens (Phase 1) requires significantly more "effective" compute to converge on the correct token distribution compared to verifying low-entropy tokens (Phase 2).
    - *Sensitivity Analysis:* We will report the AUC ranking for $\alpha \in \{0.5, 1.0, 1.5\}$. The primary conclusion is valid only if the relative ranking of schedules remains invariant across this range. Additionally, we will report the correlation between ϕ (Exploration Fraction) and the final Quality metric to empirically justify the weighting of Phase 1.

3. Implementation Details

- **Inference Pipeline:**
  1. **Initialization:** Load checkpoint. Initialize x_T with mask tokens. Ensure the attention mask and tokenizer match the original training configuration of the checkpoint to avoid distribution shift. **Verify Mask Token ID:** Explicitly check that the `mask_token_id` used in the inference loop matches the one used during the checkpoint's adaptation phase (e.g., by inspecting the tokenizer config or the model's initialization code) to prevent immediate divergence.
  2. **Step Loop:** For $t = T$ down to 1:
     - Compute logits $z_t = f_\theta(x_t, t)$.
     - Compute probabilities $p_t = \text{softmax}(z_t)$.
     - **Entropy Calculation:** Compute H_t for all masked positions.
       - *Specification:* H_t is the Shannon entropy of the **raw logits** (before any temperature scaling) for the masked positions. If temperature scaling is applied for sampling, the entropy metric uses the unscaled distribution to reflect the model's intrinsic uncertainty, while sampling uses the scaled distribution. This distinction is critical for the EPP metric.
       - Record H_t.
     - **Schedule Application:**

        * *Linear/Cosine:* Determine β_t.
        * *Context-Adaptive:* Determine mask update probability based on $H_i(t)$ and $\theta_{CA}^{(T)}$ (as defined in Section 1).
        * *Learned (LPC):* Input $(t/T, H_t)$ to MLP $\rightarrow$ get m_t. Adjust $\beta_t^{eff} = \beta_t^{linear} \times m_t$.
      - **Sampling:** Sample new tokens or keep masks based on the schedule-specific logic.
    3. **Termination:** Return x_0.
- **CDC Baseline Validation (Context-Free Entropy):**
    1. For a subset of 10% of the test samples, we perform a "Context-Free" forward pass.
    2. **Implementation:** We replace all *context* tokens (unmasked tokens) with the **`mask_token_id`** (not padding, to maintain positional embedding consistency and sequence length). The masked tokens remain as they are.
    3. We compute the entropy of the masked positions in this "all-masked" state. This serves as the $H_{baseline}$ (context-free entropy).
    4. **CDC Calculation:** The CDC score is refined to use this baseline: $CDC = \frac{H_{baseline} - H_{t^*}}{T - t^*}$. This provides a more rigorous measure of how much the *context* contributes to the entropy reduction, compared to the unigram proxy.
    5. **Compute Note:** This "context-free" forward pass is also wrapped in `torch.no_grad()` and `float16` to avoid memory spikes. It is performed only for the 10% validation subset, not the full test set, to save compute.
- **Learned Schedule (LPC) Training (Offline, CPU-only):**
    1. Run 500 validation prompts with Linear $T = 64$.
    2. Compute t^* and τ for each prompt.
    3. Label each timestep t: $y_t = 1$ if $t < t^*$ (Commitment), else 0.
    4. Train MLP to predict y_t from $(t/T, H_t)$ using AdamW (lr=$1e-3$, wd=$1e-2$) for 100 epochs.
    5. Freeze MLP. Use it to generate m_t during test runs.

4. Analysis Plan

1. **Quality-Efficiency Frontier:**
    - For each model, plot Quality (e.g., Pass@1) vs. F_{pw} for the 3 step counts.
    - Calculate AUC for each schedule.
    - **Hypothesis 1:** Context-Adaptive schedules will yield higher AUC for GPT-2 models (slower context propagation), while Linear/Cosine will yield higher AUC for LLaMA models (fast bidirectional convergence).
    - **Robustness Check:** Verify that the ranking of schedules by AUC is consistent across $\alpha \in \{0.5, 1.0, 1.5\}$. Report the correlation between ϕ and Quality to support the α weighting.
2. **Architectural Interaction (CDC):**
    - Perform Two-Way ANOVA on **CDC Score** (using the rigorous Context-Free baseline for the 10% subset, and the unigram proxy for the full set) with factors: *Model Family* (GPT-2 vs. LLaMA) and *Schedule Type*.
    - **Hypothesis 2:** LLaMA models will exhibit significantly higher CDC (steeper entropy drop) under Linear/Cosine schedules compared to GPT-2 models. GPT-2 models may show improved CDC under Context-Adaptive schedules.
3. **Learned Schedule Efficacy:**
    - Compare LPC against Linear/Cosine/CA on **Exploration Fraction (ϕ)** and **Quality**.
    - **Hypothesis 3:** LPC will reduce ϕ (faster commitment) without significant degradation in Quality, particularly for undertrained models (DiffuGPT-S/M).
    - **Dynamic vs. Static:** Compare the dynamic LPC against the "Constant LPC" (average m_t) to determine if the dynamic adjustment provides a significant gain over a simple global noise shift.
    - **Generalizability:** Analyze the correlation between the LPC's predicted t^* and the actual t^* on the test set to ensure the learned phase boundary is stable.
4. **Statistical Rigor:**
    - Report Mean ± Std for all metrics across 5 seeds.
    - Use paired t-tests for within-model schedule comparisons.

- Use effect sizes (Cohen's d) alongside p-values.

5. Feasibility & Resource Allocation

- **Compute:**
  - **GPU (1x RTX 6000 Pro 96GB):** Handles all 300 inference runs.
    - *Memory Management:* Use `torch.no_grad()` and `half()` precision. For 7B-8B models (~14-16 GB in FP16), batching 4-5 sequences is safe. For 127M-355M models, batching 10-20 sequences is feasible.
    - *Time:* 300 runs × 64 steps × 128 tokens. Even with sequential execution, this is highly feasible within 10 weeks. Batching will reduce this significantly.
  - **CPU (3 members):**
    - Train LPC MLP (negligible time).
    - Compute MAUVE, IFEval scores.
    - Statistical analysis (ANOVA, t-tests).
    - Plotting and visualization.
- **Timeline (10 Weeks):**
  - **Weeks 1-2:** Setup, data preparation, LPC training, baseline verification (ensure Linear schedule matches reference implementations), and the 10% CDC baseline validation run.
  - **Weeks 3-6:** Run all 300 inference configurations. Parallelize across the 3 GPU-enabled members if possible, or sequential if shared.
  - **Weeks 7-8:** Compute metrics (PPL, Pass@1, MAUVE, IFEval). Calculate RPD diagnostics (ϕ, CDC, t^*).
  - **Weeks 9-10:** Statistical analysis, frontier plotting, write-up.

Why this design. This revised experiment design (v3) rigorously addresses the research problem by implementing the **Resolution-Phase Decomposition (RPD)** methodology, while specifically incorporating the detailed feedback from the previous review to enhance clarity, validity, robustness, feasibility, and reproducibility.

1. **Enhanced Clarity & Reproducibility (Addressing Review Feedback):**
   - **LPC Hyperparameters:** Explicitly specified the optimizer (AdamW), learning rate ($1e-3$), weight decay ($1e-2$), and epochs (100) for the LPC training, ensuring that other researchers can replicate the exact "frozen" controller.
   - **Entropy Specification:** Clarified that H_t is calculated from **raw logits** (unscaled) to reflect intrinsic model uncertainty, distinct from any temperature-scaled sampling distribution. This eliminates ambiguity in the EPP metric.
   - **CDC Baseline Precision:** Specified that the "Context-Free" baseline uses `mask_token_id` (not padding) to maintain positional embedding consistency and sequence length, preventing confounding factors in the entropy measurement.
2. **Improved Validity (Addressing Review Feedback):**
   - **LPC Generalization & Dynamic Value:** Added a "Constant LPC" baseline (fixed m_t) to quantify the specific benefit of the *dynamic* MLP adjustment versus a simple global noise shift. This isolates the value of the learned phase transition.
   - **CDC Baseline Validation:** The rigorous "Context-Free" forward pass (replacing context with masks) is now explicitly defined to use `mask_token_id` and is wrapped in `torch.no_grad()`/`float16` to prevent memory issues. This provides a ground truth check for the CDC metric, moving beyond the unigram proxy.
   - α **Justification:** Added a plan to report the correlation between ϕ (Exploration Fraction) and Quality to empirically justify the weighting of Phase 1 in the F_{pw} metric, strengthening the validity of the efficiency score.
3. **Increased Robustness (Addressing Review Feedback):**
   - **Entropy Overhead & Latency:** Clarified that latency measurements are strictly at **batch size 1** to avoid non-linear GPU scaling effects, ensuring that "Core Inference Latency" is a fair comparison. The separation of "Core" vs. "Overhead" latency remains to isolate schedule-specific costs.
   - **CA Schedule Non-Linearity:** Added a check for the linearity of entropy decay. If the linear rescaling of θ_{CA} is significantly inaccurate (deviation > 20%), the deviation will be

reported, and the impact on ϕ will be analyzed, ensuring the CA schedule implementation is robust.
    - **LPC Trigger for Code:** Maintained the plan to monitor Pass@1 on HumanEval and defined the trigger for training a separate code-specific LPC if degradation is significant, preventing timeline slippage.
4. **Feasibility & Resource Management:**
    - **Compute Accounting:** The plan remains feasible within the 10-week timeline and single-GPU constraint. The explicit separation of latency metrics and the limited scope of the CDC baseline validation (10% subset) ensure that additional rigor does not compromise feasibility.
    - **Memory Management:** The explicit mention of `float16` and batching strategies, along with the `torch.no_grad()` context for all inference and validation runs, ensures the 96GB GPU is utilized safely and efficiently.

This design provides a comprehensive, mechanistic understanding of how noise schedules shape the quality-efficiency frontier of scaled DLMs, offering actionable insights for practitioners and researchers in the field.

Method	Clarity	Validity	Rigorousness	Innovativeness	Generalizability
rating	5	4	4	4	4

Experiment	Clarity	Validity	Robustness	Feasibility	Reproducibility
rating	5	5	5	5	5

Weakest point per the reviewers (Validity, 4/5). 1. **Baseline for CDC:** The use of "Marginal Unigram Entropy" as a context-free baseline is a weak point. Unigram entropy is a global statistic and does not account for the specific token distribution of the prompt. A more valid baseline would be the entropy of the model's prediction when the context is explicitly masked (e.g., replacing all context tokens with a padding/mask token), which would require a few additional forward passes but would be scientifically rigorous.

2. **Phase-Weighted FLOPs (α):** The introduction of α as a "tunable weight" for cognitive load is conceptually interesting but lacks a firm theoretical basis. Since FLOPs are a physical measure of computation, weighting them based on "risk" or "cognitive load" transforms the metric from a measure of *efficiency* to a measure of *utility*. It would be clearer to report Raw FLOPs and Exploration Fraction (ϕ) separately and let the correlation speak for itself, rather than merging them into a synthetic F_{pw} metric.
3. **MLP Target Leakage:** While the MLP is trained on a held-out slice, the target y_t is derived from a "reference run" (Linear schedule). This means the MLP is learning to mimic the Linear schedule's transition timing, not necessarily the *optimal* transition timing for quality. To truly find an "optimal" schedule, the target should be derived from the schedule that yielded the highest quality for that specific prompt.
